



Hytera Multi-mode Advanced Radio API Reference

Contents

1. Preface	1
1.1 Purpose	1
1.2 Target.....	1
1.2.1 General.....	1
1.2.2 Audience	1
1.2.3 Implementation	1
1.3 Revision History	1
1.4 Abbreviations.....	2
2. Naming Rules	4
2.1 Class Name	4
2.1.1 Prefix	4
2.1.2 Suffix	4
2.1.3 ALL_UPPER_CASE	4
3. Overview.....	5
4. Using the API	6
4.1 Create an API instance.....	6
5. HyteraManager.....	7
5.1 HyteraManager Introduction	7
5.2 HyteraManager Method.....	7
6. HyteraSDKException	9
6.1 HyteraSDKException Introduction.....	9
6.2 HyteraSDKException Construction Method	9
7. CommonManager Overview.....	10
7.1 CommonManager	11
7.1.1 CommonManager Introduction.....	11
7.1.2 CommonManager Method.....	11
7.2 CallManager	12

7.2.1 CallManagerIntroduction.....	12
7.2.2 Call Architecture.....	13
7.2.3 Call Process	14
7.2.4 Create a Call API instance	15
7.2.5 CallManager Members	16
7.2.6 Listeners.....	37
7.2.7 Calls.....	71
7.2.8 Data.....	153
7.2.9 Constants	185
7.2.10 Other	204
7.3 BasicSettingManager.....	207
7.3.1 BasicSettingManager Introduction	207
7.3.2 BasicSettingManager Method.....	207
7.3.3 BasicSettingListener Class	210
7.4 BatteryManager	211
7.4.1 BatteryManager Introduction.....	211
7.4.2 BatteryManager Method.....	212
7.4.3 BatteryListener Class.....	213
7.5 GpsManager.....	214
7.5.1 GpsManager Introduction	214
7.5.2 GpsManager Method	214
7.5.3 GpsManager Data	217
7.6 LedVibrationManager.....	219
7.6.1 LedVibrationManager Introduction	219
7.6.2 LedVibrationManager Method	219
7.6.3 LedVibrationListener Class	222
7.7 RadioManager	223
7.7.1 RadioManager Introduction.....	223
7.7.2 RadioManagerMethod	223

7.7.3 RadioListener.....	227
8. DmrManager Overview	230
8.1 DmrManager.....	230
8.1.1 DmrManager Introduction	230
8.1.2 DmrManager Method	231
8.2 DmrCommonManager.....	232
8.2.1 DmrCommonManagerIntroduction	232
8.2.2 DmrCommonManagerMethod.....	233
8.2.3 DmrCommonListener	235
8.3 ChannelCManager	236
8.3.1 ChannelCManager Introduction	236
8.3.2 ChannelCManager Method.....	236
8.3.3 ChannelCManager Data.....	241
8.4 RoamCManager.....	242
8.4.1 RoamCManager Introduction	242
8.4.2 RoamCManager Method.....	242
8.4.3 RoamCManager Data	244
8.5 SmsCManager	244
8.5.1 SmsCManager Introduction.....	244
8.5.2 SmsCManager Method	245
8.5.3 SmsCListener Class	250
8.5.4 SmsCManager Data	251
8.6 ContactTManager	254
8.6.1 ContactTManager Introduction.....	254
8.6.2 ContactTManager Method.....	254
8.6.3 ContactTListener Class.....	257
8.6.4 ContactTManager Data.....	257
8.7 RegistrationTManager	258
8.7.1 RegistrationTManagerIntroduction	258

8.7.2 RegistrationTManager Method.....	258
8.8 SmsTManager.....	268
8.8.1 SmsTManager Introduction	268
8.8.2 SmsTManager Method	268
8.8.3 SmsTListener Class	271
8.8.4 SmsTManager Data	272
9. TetraManager Overview	275
9.1 TetraManager.....	275
9.1.1 TetraManager Introduction	275
9.1.2 TetraManager Method.....	276
9.2 ConfigManager	277
9.2.1 ConfigManager Introduction	277
9.2.2 ConfigManager Method.....	277
9.3 GroupManager.....	280
9.3.1 GroupManager Introduction	280
9.3.2 GroupManager Method.....	280
9.3.3 GroupManager Data	287
9.4 RegistrationManager.....	290
9.4.1 RegistrationManager Introduction	290
9.4.2 RegistrationManager Method	290
9.5 SDSManager.....	296
9.5.1 SDSManager Introduction	296
9.5.2 SDSManager Method	297
9.5.3 SDSListener Class	307
9.5.4 SDSManager Data	308
9.6 PppOfTetraManager	315
9.6.1 PppOfTetraManager Introduction.....	315
9.6.2 PppOfTetraManager Method	315
9.6.3 PppManagerListener.....	319

10. Example Application 320

1. Preface

1.1 Purpose

This API document describes the application interfaces open based on Hytera multi-mode advanced radio, in addition to android native APIs, it adds narrowband system related interfaces, such as call service, text message and short data , etc. It supports PDT, DMR and TETRA standards.

The APIs solve some problems when attaching a narrowband system to a (public) network. It handles the Network Address Translation, encrypts the signaling and voice data sent over the network and provides Quality of Service information. Using these APIs, users are able to develop the applications which can access to pure narrowband system or pure broadband system seamlessly.

1.2 Target

1.2.1 General

This document mainly describes the APIs related to narrowband system, the native APIs is beyond of the scope.

1.2.2 Audience

The description targets people, who:

- plan and specify an application communicating with a Hytera narrowband system
- implement or update an application communicating with a Hytera narrowband system

1.2.3 Implementation

The App has to be compiled with Android SDK 16 or higher version and needs the SDK library.

1.3 Revision History

Version	Date	Description
R1.0	2017-08-04	Initial release.
R1.5	2017-09-25	Add Dmr short data API and Tetra PPP API.

1.4 Abbreviations

Abbreviation	Meaning
AIE	Air Interface Encryption
AP	Android Platform
CPS	Customer Programming Software
DMO	Direct Mode Operation
DmrC	Digital Mobile Radio Conventional
DmrT	Digital Mobile Radio Trunking
E2EE	End-to-End Encryption
ESN	Electronic Serial Number
FSSN	Fleet Specific Short Number
GPS	Global Positioning System
GSSI	Group Short Subscriber Identity
ITSI	Individual TETRA Subscriber Identity
TMO	Trunked Mode Operation
NB	Narrow Band
MCC	Mobile Country Code
MDM	Mobile Device Management
MNC	Mobile Network Code
MNI	Mobile Network Identity
PABX	Private Automatic Branch Exchange
PEI	Peripheral Equipment Interface
PSTN	Public Switched Telephone Network
PS-LTE	Public Safety-LTE

Abbreviation	Meaning
RoIP	Radio Over IP
RSSI	Received Signal Strength Indicator
SDS	Short Data Service
SDS-TL	Short Data Service - Transport Layer
SNA	Short Number Addressing
SSI	Short Scribe Identify
SwMI	Switching and Management Infrastructure
TetraD	Terrestrial Trunked RadioDMO
TetraT	Terrestrial Trunked RadioTMO
TEI	Terminal Equipment Identity
TMO	Trunked Mode Operation
TP	Third Party
TSI	TETRA Subscriber Identity

2. Naming Rules

This chapter lists a summary of naming rules.

2.1 Class Name

2.1.1 Prefix

The following table shows a list of prefixes used in the API.

Prefix	Description
I	This prefix indicates an interface.
A	This prefix indicates an abstract class.

2.1.2 Suffix

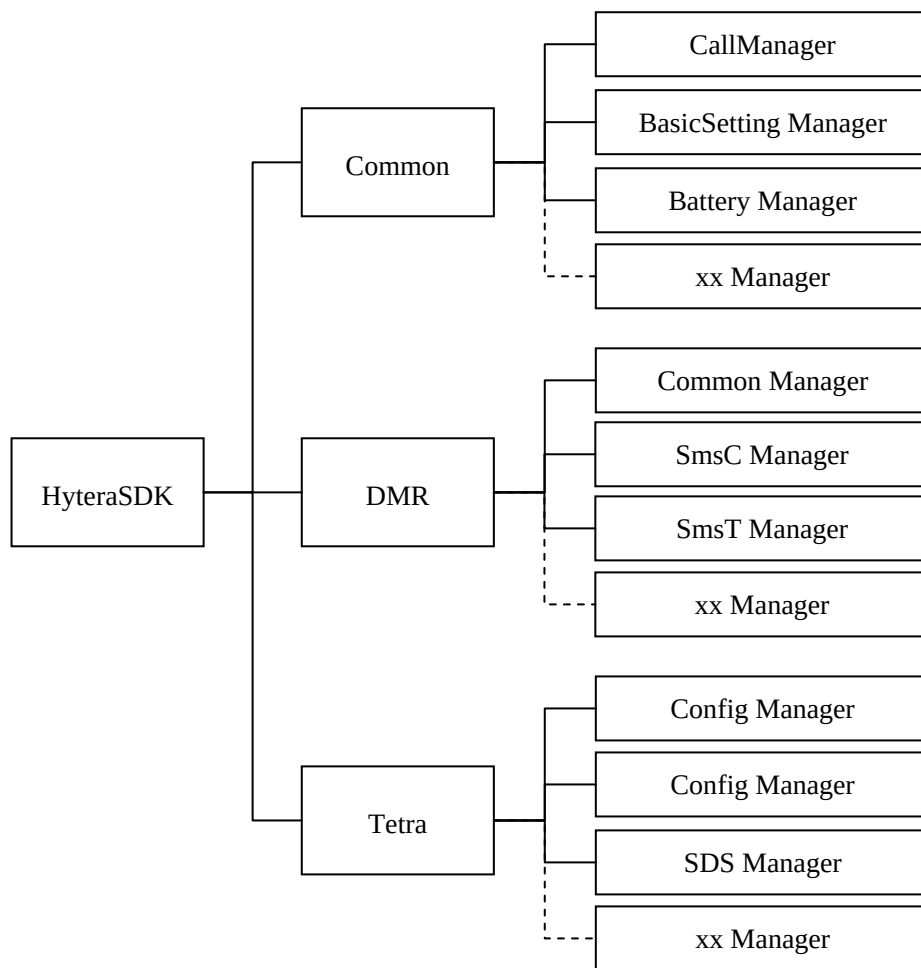
The following table shows a list of suffixes used in the SDK API.

Suffix	Description
Listener	This suffix indicates a listener. A listener is a registered interface for the API to communicate with the application. It is informed about occurred events.
Callback	This suffix indicates a callback. A callback is an interface for the API to communicate with the application. It is informed about occurred events concerning a particular method call.

2.1.3 ALL_UPPER_CASE

Classes that are written with all upper cases are constant containers. These contain public constant fields. They often provide a getXXX method e.g. getName to deliver a human readable text of the constant name.

3. Overview



The picture shows the overview of HyteraSDK. The SDK contains 3 main managers to manage different modules.

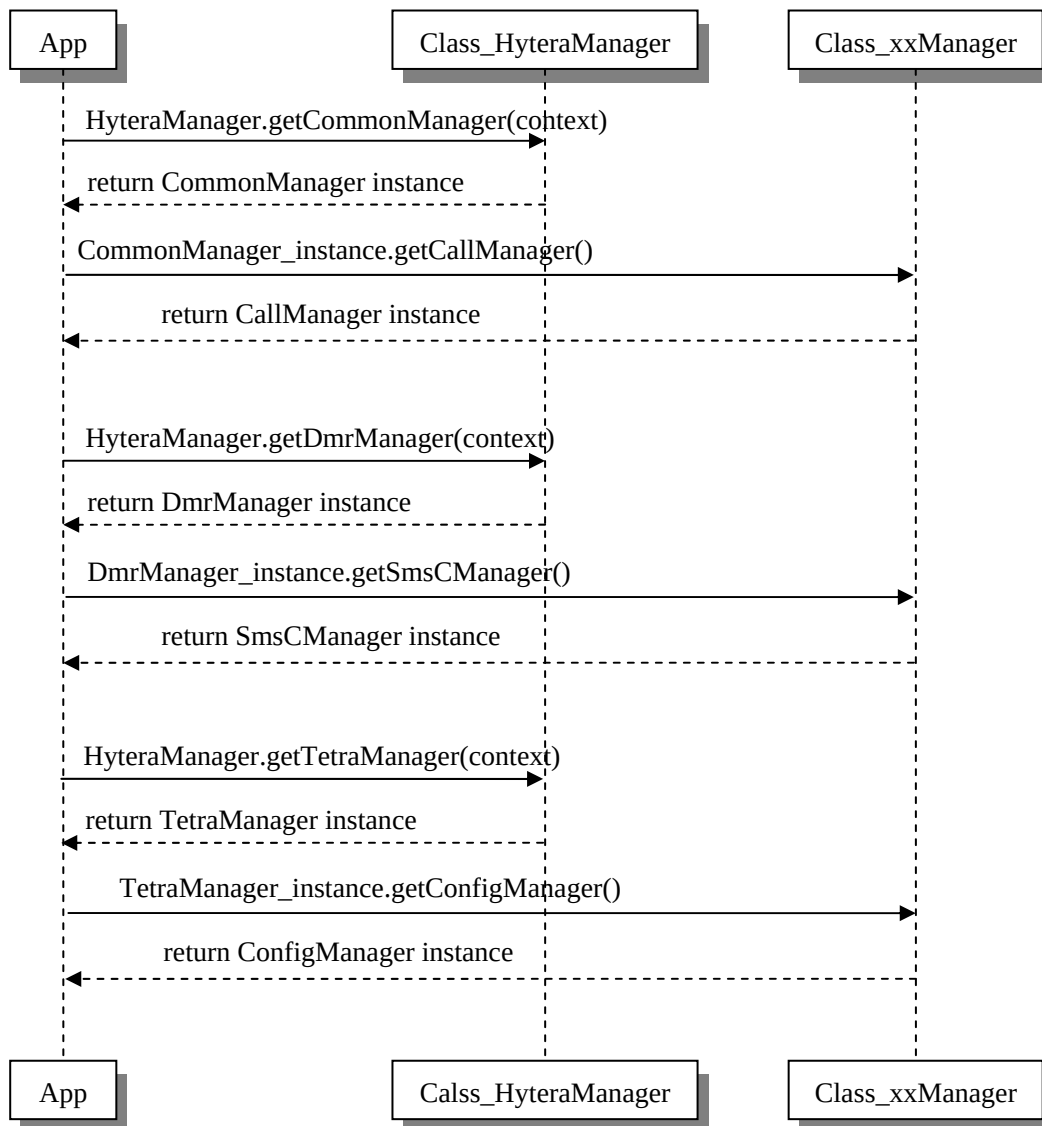
CommonManager contains methods to manage some common modules for DMR radios and Tetra radios. The Common Manager supports services of Call, Basic setting, Battery, GPS, Radio, LED and Vibrate, and all instances can be obtained easily in CommonManager.

DMR Manager provides methods to manage DMR radios, such as how to send a message in DMR Conventional mode or in DMR Trunking mode, or how to get channel or roaming information.

Similarly, TetraManager provides methods to manage Tetra radios.

4. Using the API

4.1 Create an API instance



The picture describes how to create an api instance. The App calls the `getCommonManager/getDmrManager/getTetraManager` method and hands over the context. Depending on the implementation, these methods return a singleton or a new instance respectively. After getting these instances, the App can get some specified api methods through each manager instance.

5. HyteraManager

5.1 HyteraManager Introduction

HyteraManager class contains 3 necessary methods to get 3 different managers (Common/DMR/TETRA).

Package: com.hytera.api

5.2 HyteraManager Method

getCommonManager

```
public static CommonManager getCommonManager(  
    android.content.Context context)
```

Get CommonManager instance

Parameters:

context -

Returns: CommonManager instance

getDmrManager

```
public static DmrManager getDmrManager(  
    android.content.Context context)
```

Get DmrManager instance

Parameters:

context -

Returns: DmrManager instance

getTetraManager

```
public static TetraManager getTetraManager(  

```

`android.content.Context context)`

Get TetraManager instance

Parameters:

context -

Returns: *TetraManager instance*

6. HyteraSDKException

6.1 HyteraSDKException Introduction

HyteraSDKException class is an encapsulation class of Exception, it is used to catch runtime exception.

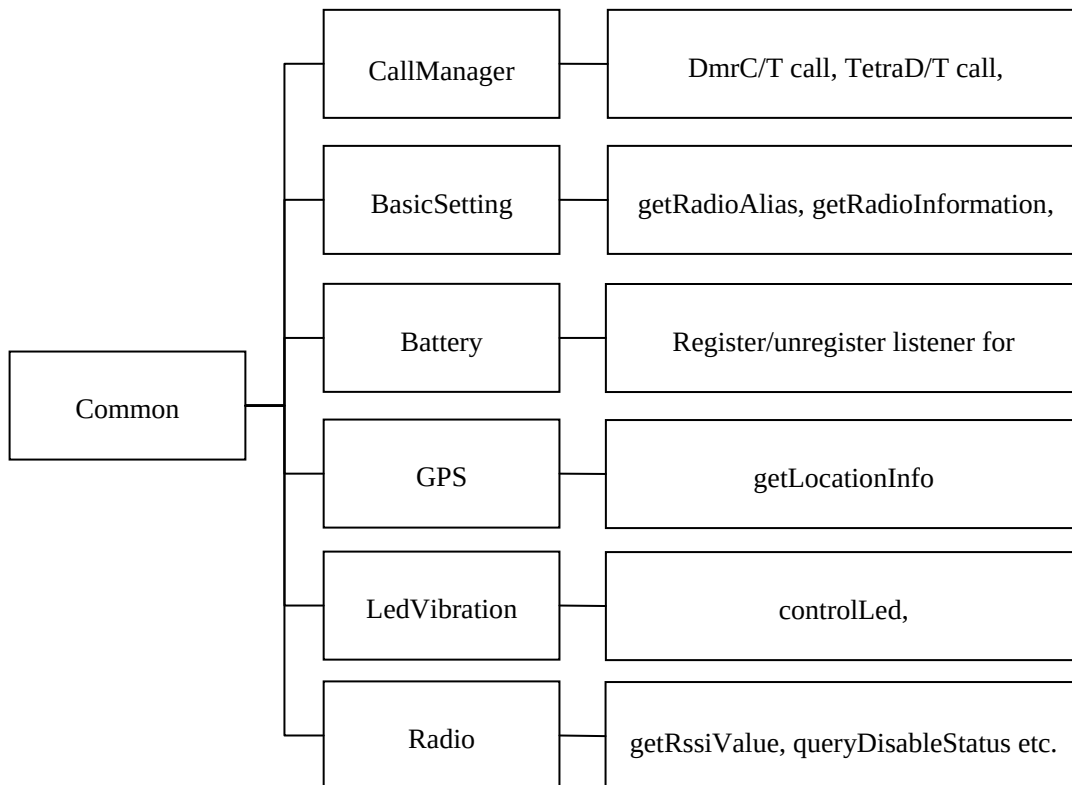
Package: com.hytera.api

6.2 HyteraSDKException Construction Method

HyteraSDKException

```
public HyteraSDKException(java.lang.String msg)
```

7. CommonManager Overview



The picture shows the overview of CommonManager. It contains 6 sub-managers to manage different modules. Each manager is independent of each other.

CallManager contains methods to setup or refuse a call.

BasicSettingManager provides methods to get information about basic setting module.

BatteryManager contains 2 methods for the listening or unlistening of the battery level.

GpsManager provides a method to get the location information of devices.

LedVibrationManager provides 2 methods to manage LED and vibration. The operation methods are described in detail in this manager.

RadioManager provides methods to manage radio modules, such as calling getRssiValue to get the RSSI value in current environment.

7.1 CommonManager

7.1.1 CommonManager Introduction

CommonManager interface is used to get some other managers, you can call its member methods to get different manager instances.

Package: com.hytera.api.base.common

7.1.2 CommonManager Method

getCallManager

`CallManager` getCallManager(CallManager.CallManagerConnectStateListener callManagerCallback) throws `HyteraSDKException`

Parameters:

callManagerCallback –

CallManager.CallManagerConnectStateListener

Returns: `CallManager` instance

Throws: `HyteraSDKException`

getBasicSettingManager

`BasicSettingManager` getBasicSettingManager()
throws `HyteraSDKException`

Returns: `BasicSettingManager` instance

Throws: `HyteraSDKException`

getGpsManager

`GpsManager` getGpsManager()throws `HyteraSDKException`

Returns: `GpsManager` instance

Throws: [HyteraSDKException](#)

getBatteryManager

[BatteryManager](#) getBatteryManager()throws [HyteraSDKException](#)

Returns: **BatteryManager instance**

Throws: [HyteraSDKException](#)

getLedVibrationManager

[LedVibrationManager](#) getLedVibrationManager()throws [HyteraSDKException](#)

Returns: **LedVibrationManager instance**

Throws: [HyteraSDKException](#)

getRadioManager

[RadioManager](#) getRadioManager()throws[HyteraSDKException](#)

Returns: **RadioManager instance**

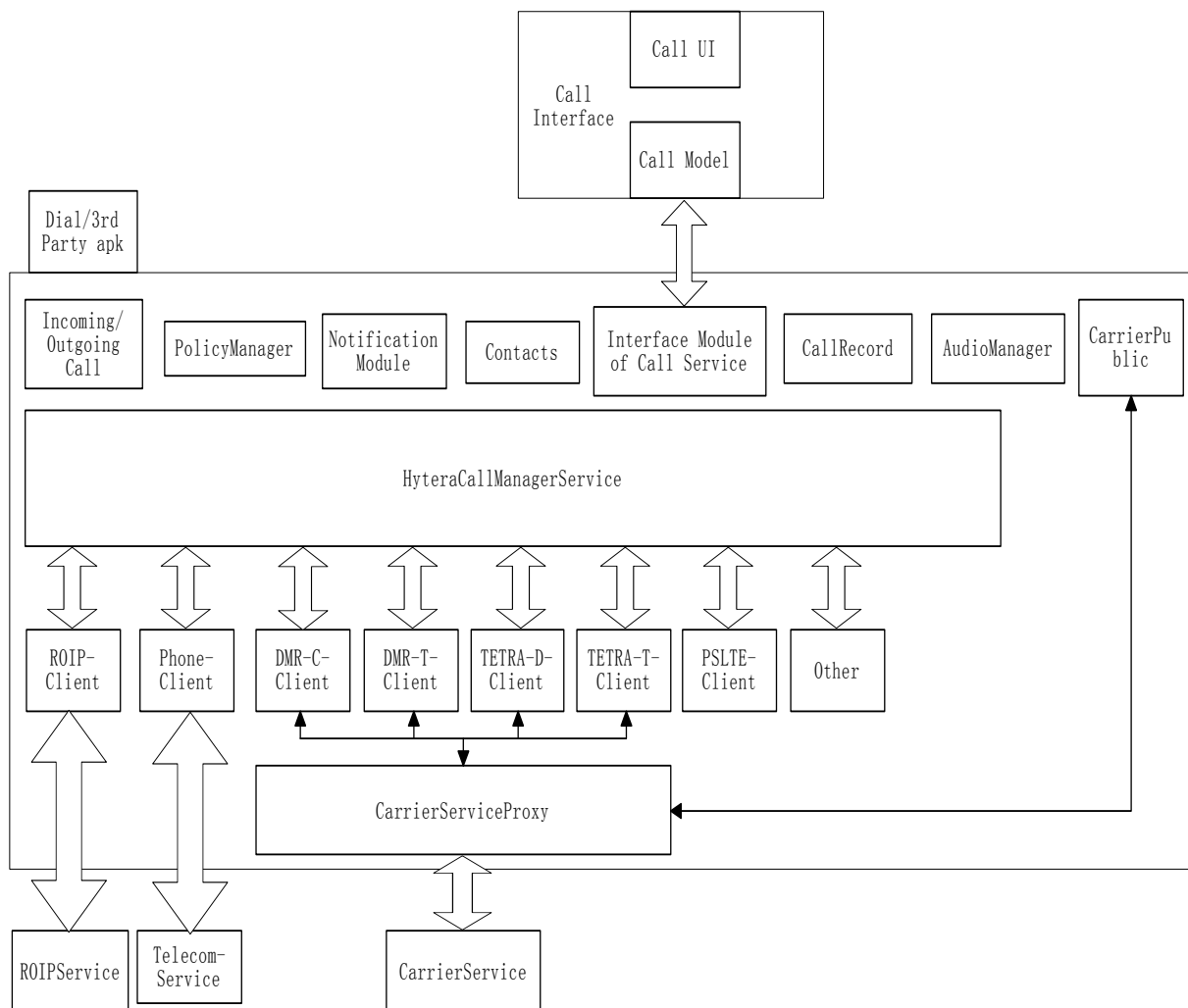
Throws: [HyteraSDKException](#)

7.2 CallManager

7.2.1 CallManagerIntroduction

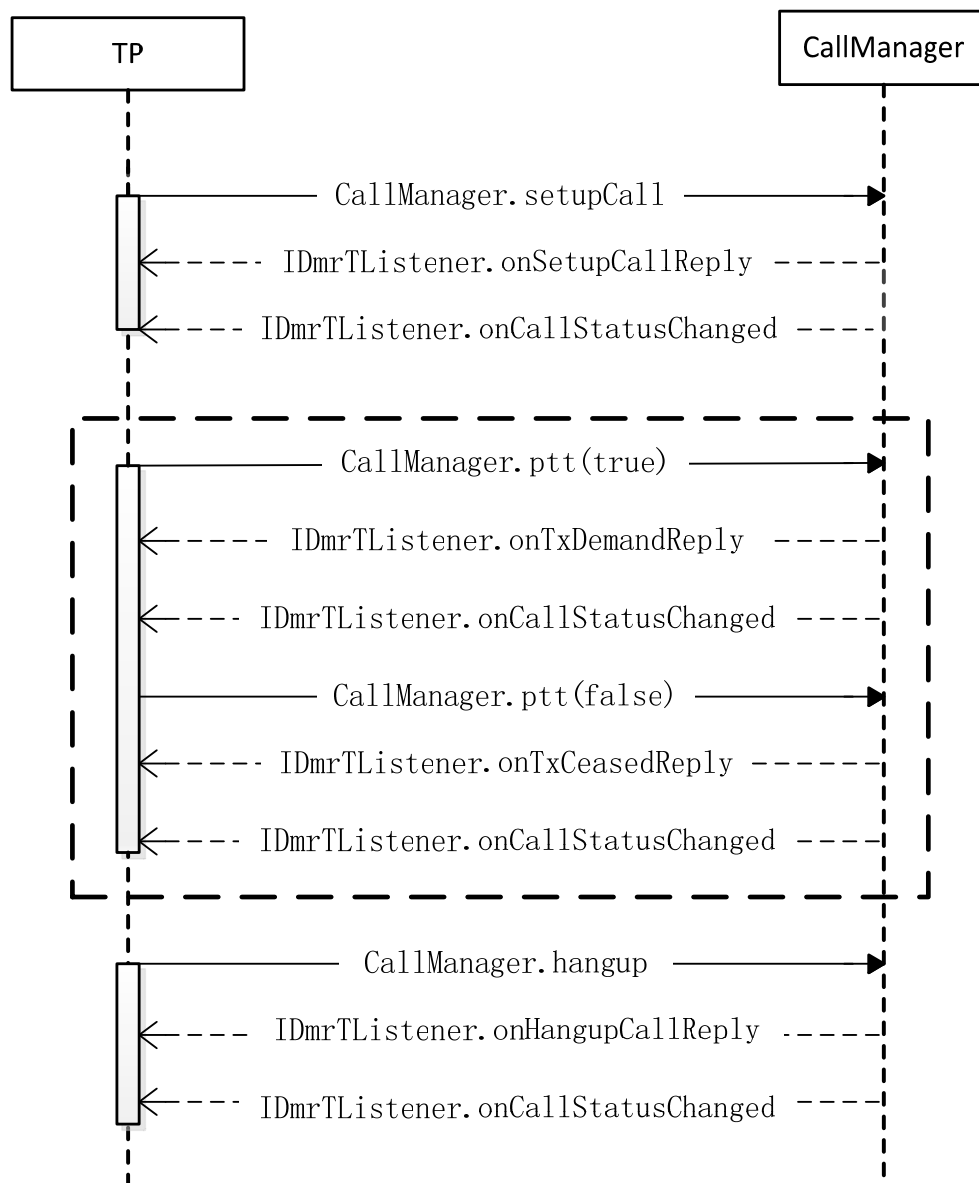
The API interface contains all necessary methods to setup or refuse a call. It is used to handle call services including DmrC/T call, TetraD/T call,ROIP call, etc.It also supports PTT response and call voice recording.

7.2.2 Call Architecture



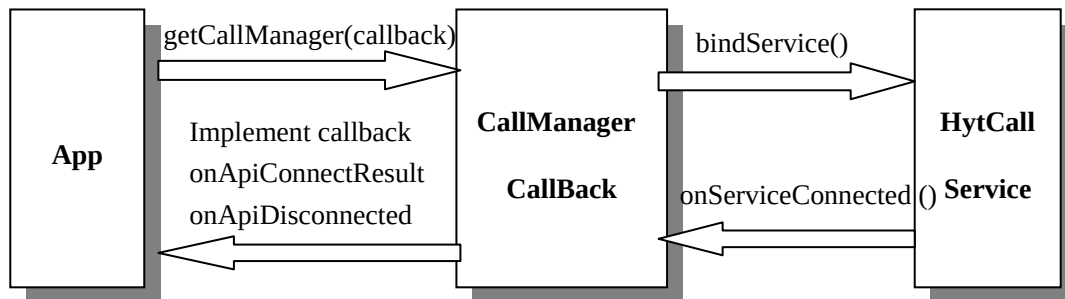
The architecture of call service can be classified into modules of Manager call, ROIP call, NB call, Phone call, UI, third party interface, Notification, etc.

7.2.3 Call Process



The picture above is a simple example to describe call process. TP can use `setupCall(OutgoingCallParam param)` of `CallManager` to set up a call and register a corresponding listener to listen the state of call. When the call sets up successfully, methods of `CallManager` can be used to control the call, such as `PTT(int netType, String callId, boolean down)`, `HangUp(int netType, String callId)`, `reject(int netType, String callId, boolean rejectWithMessage, String message)` etc. When these methods were called, the listener we register notifies TP of the operating result or the change of call state.

7.2.4 Create a Call API instance



The picture describes how to get **CallManager** instance, which has bound a **HytCallService**. The callback method returns the result of init progress and disconnected reason.

Example:

```

mCommonManager = HyteraManager.getCommonManager(context);

try {

    mCallManager =
        mCommonManager.getCallManager(new CallManager.CallManagerConnectStateListener () {

            @Override

            public void onApiConnectResult (int ret) {

                Log.d(TAG, "Api connect result, ret="+ret);

            }

            @Override

            public void onApiDisconnected(int ret) {

                Log.d(TAG, "Api disconnect result, ret="+ret);
            }
        });
  
```

```
    }

    });

    } catch (HyteraSDKException e) {

        e.printStackTrace();

    }

    Reference: CallManager.CallManagerConnectStateListener
```

7.2.5 CallManager Members

Package:com.hytera.api.base.common

7.2.5.1 CallManager Constant Fields

Qualifier and Type	Constant Fields	Value
public static final int	API_DISCONNECTED_REMOTE_SERVICE_CLOSED	0
public static final int	API_CONNECT_RESULT_SUCCESS	0
public static final int	API_DISCONNECTED_REMOTE_SERVICE_RESTART	1
public static final int	API_CONNECT_RESULT_BIND_SERVICE_ERROR	1
public static final int	API_DISCONNECTED_UNKNOWN	2
public static final int	API_CONNECT_RESULT_NULL_API_ERROR	2
public static final int	API_CONNECT_RESULT_HYTERA_SYSTEM_NOT_READY	3

7.2.5.2 CallManager Methods

operContactStarred


```
public void operContactStarred(int netType, int callType,  
                               java.lang.String contactNum,boolean isStarred)
```

Mark contact as favorite or remove contact from favorites.

Parameters:

netType- network type of call

callType - type of call

contactNum- number of contact

isStarred- whether mark the contact as favorite

Reference: [NetType](#), [CallType](#)

Example:

```
If(mCallManager.isContactUsable(mCall.getNetType(), mCall.getCallType(),  
                                mCall.getContactNumber())){  
mCallManager. operContactStarred(mCall.getNetType(), mCall.getCallType(),  
                                mCall.getContactNumber(), isChecked);  
}
```

isContactUsable

```
public boolean isContactUsable(int netType,  
                               int callType,java.lang.String contactNum  
                               )
```

Whether the contact can be marked as favorite

Parameters:

netType - network type of call

callType - type of call

contactNum - number of contact

Returns:whether the contact can be marked as favorite

Reference: [NetType](#), [CallType](#)

isContactStarred

```
public boolean isContactStarred(int netType, int callType,  
java.lang.String contactNum)
```

Whether the contact has been marked as favorite

Parameters:

netType- network type of call

callType - type of call

contactNum - number of contact

Returns: whether the contact had been marked as favorite

Reference: [NetType](#), [CallType](#)

answer

```
public void answer(int netType, java.lang.String callId)
```

Answer incoming call

Parameters:

netType-network type of call

callId-call ID

Reference: [NetType](#)

Example:

```
mCallManager.answer(mCall.getNetType(), mCall.getCallId());
```

reject

```
public void reject(int netType, java.lang.String callId,  
                  boolean rejectWithMessage, java.lang.String message)
```

Reject incoming call.

Parameters:

netType-network type of call

callId- call ID

rejectWithMessaage - whether reject the incoming call with
message

message - reject message

Reference:[NetType](#)

hangUp

```
public void hangUp(int netType, java.lang.String callId)
```

Hangup call.

Parameters:

netType-network type of call

callId- call id

Reference:[NetType](#)

hold

```
public void hold(int netType, java.lang.String callId)
```

Hold call, only for phone call.

Parameters:

netType- network type of call

callId- call id

Reference: [NetType](#)

unhold

```
public void unhold(int netType, java.lang.String callId)
```

Unhold call, only for phone call.

Parameters:

netType - network type of call

callId - call id

Reference: [NetType](#)

processDtmf

```
public void processDtmf(int netType, java.lang.String callId, char c)
```

Dtmf, only supports PhoneCall. It is recommended to call the method stopDtmf upon calling the method processDtmf.

Parameters:

netType - network type of call

callId - call id

c - key, it could be one of the group(0,1,2,3,4,5,6,7,8,9,*,#)

Reference: [NetType](#)

Example:

```
mCallManager.processDtmf(mCall.getNetType(), mCall.getCallId(), '6');
```

stopDtmf

```
public void stopDtmf(int netType, java.lang.String callId)
```

Stop Dtmf,only supportsPhoneCall

Parameters:

netType- network type of call

callId- call id

Reference:[NetType](#)

Example:

```
public boolean onTouch(View v, MotionEvent event) {  
    if (event.getAction() == MotionEvent.ACTION_UP) {  
        mCallManager.stopDtmf(mCall.getNetType(),mCall.getCallId());  
    } elseif (event.getAction() == MotionEvent.ACTION_DOWN) {  
        switch (v.getId()) {  
            case R.id.n0:  
            case R.id.n1:  
            case R.id.n2:  
            case R.id.n3:  
            case R.id.n4:  
            case R.id.n5:  
            case R.id.n6:  
            case R.id.n7:  
            case R.id.n8:  
            case R.id.n9:  
            case R.id.nAst:  
            case R.id.nPound:  
                String clickNum = ((Button) v).getTag().toString().trim();
```

```
mCallManager.processDtmf(mCall.getNetType(),mCall.getCallId(),
    clickNum.charAt(0));

    Log.d(tag, clickNum + " DOWN");

    break;

default:

    Log.d(tag, "illegal num DOWN");

    break;

}

}

return true;

}
```

ptt

```
public void ptt(int netType, java.lang.String callId, boolean down)
```

PTT request

Parameters:

netType-network type of call

callId- call id

down-PTT state

Reference: [NetType](#)

setMute

```
public void setMute(int netType, java.lang.String callId, boolean isMute)
```

Set microphone mute

Parameters:

netType- network type of call

callId- call id

isMute- whether set microphone mute

Reference: [NetType](#)

record

```
public void record(int netType, java.lang.String callId)
```

Start recording

Parameters:

netType -network type of call

callId - call id

Reference: [NetType](#)

stopRecord

```
public void stopRecord(int netType, java.lang.String callId)
```

Stop recording

Parameters:

netType -network type of call

callId - call id

Reference: [NetType](#)

pauseRecord

```
public void pauseRecord(int netType, java.lang.String callId)
```

Pause recording

Parameters:

netType- network type of call

callId- call id

Reference: [NetType](#)

getCallRecordingState

```
public void getCallRecordingState(int netType,  
                                java.lang.String callId)
```

Get the recording status returned by the callback(CallRecordListener.onGetCallRecordingState).The CallRecordListenerneeds to be registered at first.

Parameters:

netType- network type of call

callId- call id

Reference: [NetType](#), [CallRecordListener](#)

getIsCallRecorderEnabled

```
public void getIsCallRecorderEnabled(int netType,  
                                    java.lang.String callId)
```

Judge CallRecorder is enable or disable.The result is returned by the callback(CallRecordListener.onGetIsCallRecorderEnabled),so The CallRecordListenerneeds to be registered at first.

Parameters:

netType-network type of call

callId- call id

Reference: [NetType](#), [CallRecordListener](#)

getCallRecordingTime

```
public void getCallRecordingTime(int netType, java.lang.String callId)
```

Get call recording time returned by the callback(CallRecordListener.onGetCallRecordingTime). The CallRecordListener needs to be registered at first.

Parameters:

netType- network type of call

callId- call id

Reference: [NetType](#), [CallRecordListener](#)

release

```
public void release()
```

Release API

setupCall

```
public void setupCall(OutgoingCallParam param)
```

Set up a call

Parameters:

param - params of outgoing call

Reference: [OutgoingCallParam](#)

Example:

```
IDmrCCall Call = new IDmrCCall();  
Call.setCallType(CallType.DMR_C_INDIVIDUAL_CALL);  
Call.setContactNumber(number);  
Call.setIsHook(false);  
Call.setCallMode(CallMode.HALF_DUPLEX);  
Call.setCallPriority(CallPriority.DMR_C_NORMAL);
```

```
OutgoingCallParam OutgoingCallParam = new OutgoingCallParam(Call);  
mCallManager.setupCall(OutgoingCallParam);
```

openNBNetwork

```
public void openNBNetwork()
```

Open NB network

closeNBNetwork

```
public void closeNBNetwork()
```

Close NB network

releasePhysicalPtt

```
public void releasePhysicalPtt(BasePhysicalPttCallback callback)
```

Release Physical PTT callback

Parameters:

callback—Physical Ptt callback

Reference: [BasePhysicalPttCallback](#)

requestPhysicalPtt

```
public void requestPhysicalPtt(BasePhysicalPttCallback callback)
```

Request Physical PTT callback. TP implements the abstract class BasePhysicalPttCallback and passes it to Hytera Call Application by the method requestPhysicalPtt, then Hytera Call Application gets outgoing call's param to set up call by the the method of BasePhysicalPttCallback that TP implemented when physical PTT down.

Parameter:

callback– Physical Ptt callback

Reference: [BasePhysicalPttCallback](#)

addDmrCListener

```
public void addDmrCListener(IDmrCListener listener)
```

Add listener of DmrC

Parameters:

listener - listener of DmrC

Reference: [IDmrCListener](#)

Example:

```
mCallManager.addDmrCListener(new IDmrCListener() {  
  
    @Override  
    public void onDigitalSetupCallReply(int result) {  
  
    }  
  
    @Override  
    public void onDigitalHangupCallReply(int ret) {  
  
    }  
  
    @Override  
    public void onDigitalCallStatusChanged(IDmrCCall call) {  
  
    }  
  
    @Override  
    public void onAnalogStatusNotify(IDmrCCall call) {  
  
    }  
  
    @Override  
    public void onAnalogEmergencyStatusNotify(IDmrCCall call) {  
  
    }  
})
```

```
    }  
    })
```

removeDmrCListener

```
public void removeDmrCListener(IDmrCListener listener)
```

Remove listener of DmrC

Parameters:

listener - listener of DmrC

Reference:[IDmrCListener](#)

Example:

```
mCallManager.removeDmrCListener(mDmrCListener);
```

addDmrTListener

```
public void addDmrTListener(IDmrCListener listener)
```

Add listener of DmrT

Parameters:

listener - listener of DmrT

Reference:[IDmrCListener](#)

removeDmrTListener

```
public void removeDmrTListener(IDmrTListener listener)
```

Remove listener of DmrT

Parameters:

listener - listener of DmrT

Reference:[IDmrTListener](#)

addTetraDListener

```
public void addTetraDListener(IDmrTListener listener)
```

Add listener of Tetra Dmo

Parameters:

listener - listener of Tetra Dmo

Reference:[IDmrTListener](#)

removeTetraDListener

```
public void removeTetraDListener(ITetraDListener listener)
```

Remove listener of Tetra Dmo

Parameters:

listener - listener of Tetra Dmo

Reference:[ITetraDListener](#)

addTetraTListener

```
public void addTetraTListener(ITetraTListener listener)
```

Add listener of TetraTmo

Parameters:

listener - listener of TetraTmo

Reference:[ITetraTListener](#)

removeTetraTListener

```
public void removeTetraTListener(ITetraTListener listener)
```

Remove listener of TetraTmo

Parameters:

listener - listener of TetraTmo

Reference: [ITetraTListener](#)

addPhoneCallListener

```
public void addPhoneCallListener(IPhoneCallListener listener)
```

Add listener of Phone

Parameters:

listener - listener of Phone

Reference: [IPhoneCallListener](#)

removePhoneCallListener

```
public void removePhoneCallListener(IPhoneCallListener listener)
```

Remove listener of Phone

Parameters:

listener - listener of Phone

Reference: [IPhoneCallListener](#)

addRoipCallListener

```
public void addRoipCallListener(IRoipCallListener listener)
```

Add listener of Roip

Parameters:

listener - listener of Roip

Reference: [IRoipCallListener](#)

removeRoipCallListener

```
void removeRoipCallListener(IRoipCallListener listener)
```

Remove listener of Roip

Parameters:

listener - listener of Roip

Reference: [IRoipCallListener](#)

getRoipProxy

```
public RoipProxy getRoipProxy()
```

Get proxy of RoipTetra

Reference: [RoipProxy](#)

addNotificationListener

```
void addNotificationListener(UINotificationListener listener)
```

Add listener of Call Notification

Parameters:

listener - listener of Call Notification

Reference: [UINotificationListener](#)

removeNotificationListener

```
public void removeNotificationListener(UINotificationListener listener)
```

Remove listener of Call Notification

Parameters:

listener - listener of Call Notification

Reference: [UINotificationListener](#)

addCallRecordListener

```
public void addCallRecordListener(CallRecordListener listener)
```

Add listener of call record

Parameters:

listener - listener of call record

Reference:[CallRecordListener](#)

removeCallRecordListener

```
public void removeCallRecordListener(CallRecordListener listener)
```

Remove listener of call record

Parameters:

listener - listener of call record

Reference:[CallRecordListener](#)

addPhysicalPttListener

```
public void addPhysicalPttListener(PhysicalPttListener listener)
```

Add listener of Physical PTT

Parameters:

listener - listener of Physical Ptt

Reference:[PhysicalPttListener](#)

removePhysicalPttListener

```
public void removePhysicalPttListener(PhysicalPttListener listener)
```

Remove listener of Physical PTT

Parameters:

listener - listener of Physical Ptt

Reference: [PhysicalPttListener](#)

addUIChangedListener

```
public void addUIChangedListener(UIChangedListener listener)
```

Add listener of UI status

Parameters:

listener - listener of UI status

Reference: [UIChangedListener](#)

removeUIChangedListener

```
public void removeUIChangedListener(UIChangedListener listener)
```

Remove listener of UI status

Parameters:

listener - listener of UI status

Reference: [UIChangedListener](#)

getAudioModeData

```
public void getAudioModeData(int netType, java.lang.String callId,int  
                             callMode)
```

Get current audio mode and available audio mode list, returned by the callback([IAudioModeListener.onSolGetAudioModeData](#)). The [IAudioModeListener](#) needs to be registered at first.

Parameters:

netType-nettype of call

callId- call id

callMode - call mode

Reference: [NetType](#), [CallMode](#), [IAudioModeListener](#)

Example:

```
IAudioModeListener mAudioModeListener = new IAudioModeListener() {  
  
    @Override  
  
    public void onSolGetAudioModeData(IAudioModeData data) {  
  
        mCurrentAudioMode = data.mode;  
  
        mAvailableAudioModeList = (int[]) data.modelist;  
  
    }  
  
};  
  
mCallManager.addAudioModeListener(mAudioModeListener);  
  
mCallManager.getAudioModeData(mCall.getNetType(), mCall.getCallId(),  
mCall.getCallMode());
```

setAudioMode

```
public void setAudioMode(int mode)
```

Set audio mode

Parameters:

mode - audio mode, refer to CallAudioState

Example:

```
mCallManager.setAudioMode(mCall.getNetType(), mCall.getCallId(), CallAudioState
                           .ROUTE_EARPIECE);
```

addAudioModeListener

```
public void addAudioModeListener (IAudioModeListener listener)
```

Add listener of audio mode

Parameters:

listener - listener of audio mode

Reference: [IAudioModeListener](#)

removeAudioModeListener

```
public void removeAudioModeListener (IAudioModeListener listener)
```

Remove listener of audio mode

Parameters:

listener - listener of audio mode

Reference: [IAudioModeListener](#)

updateCallDisplayedMode

```
public void updateCallDisplayedMode(java.lang.String callId,int mode)
```

Update current UI display mode

Parameters:

called - call id

mode - display mode

Reference: [CallDisplayedMode](#)

registerHytKeyListener

```
public boolean registerHytKeyListener(int[] keyCodes, IHytKeyEvent event)
```

Add listener of keys

Parameters:

keyCodes- the keys need to listen, refer to HytKeyCode

event- listener of key, refer to IHytKeyEvent

Example:

```
backKeyListener = new IHytKeyEvent.Stub() {  
    @Override  
    public boolean onKey(KeyEvent arg0) throws RemoteException {  
        // TODO Auto-generated method stub  
        if(arg0.getKeyCode()==HytKeyCode.KEY_CODE_BACK){  
            //do something  
        }  
        return false;  
    }  
};  
  
int[] intArray = {HytKeyCode.KEY_CODE_BACK};  
mCallManager.registerHytKeyListener(intArray,backKeyListener);
```

unregisterHytKeyListener

```
public boolean unregisterHytKeyListener(IHytKeyEvent event)
```

Remove listener of keys

Parameters:

event- listener of key, refer to IHytKeyEvent

Example:

```
mCallManager.unregisterHytKeyListener(backKeyListener)
```

7.2.6 Listeners

7.2.6.1 CallManager.CallManagerConnectStateListener

When CallManager initializing or CallManager disconnected, this listener callback method will be called.

Package: com.hytera.api.base.common

CallManager.CallManagerConnectStateListener Methods

onApiConnectResult

```
void onApiConnectResult (int ret)
```

Result of initializing CallManager

Parameters:

ret - result status

Reference: [CallManager](#) constant fields

onApiDisconnected

```
void onApiDisconnected(int ret)
```

Reason of Api disconnected

Parameters:

ret - return the reason of disconnecting

Reference: [CallManager](#) constant fields

7.2.6.2 IDmrCListener

Package: com.hytera.call.param.dmrC

The listener for DmrC status change

IDmrCListener Methods

onDigitalSetupCallReply

```
void onDigitalSetupCallReply(int result)
```

Reply of setting up call

Parameters:

result -0x0000 Success, 0x0001 Failure

onDigitalHangupCallReply

```
void onDigitalHangupCallReply(int result)
```

Reply of hanging up call

Parameters:

result -0x00 Success , 0x01 Failure

onDigitalCallStatusChanged

```
void onDigitalCallStatusChanged(IDmrCCall call)
```

Notify digital call status

Parameters:

call-extends from Call,about handling call

Reference:[IDmrCCall](#)

onAnalogStatusNotify

```
void onAnalogStatusNotify(IDmrCCall call)
```

Notify analog call status

Parameters:

call-extends from Call,about handling call

Reference: [IDmrCCall](#)

onAnalogEmergencyStatusNotify

```
void onAnalogEmergencyStatusNotify(IDmrCCall call)
```

Notify analog emergency call status

Parameters:

call -extends from Call,about handling call

Reference: [IDmrCCall](#)

onEncryptStateChanged

```
void onEncryptStateChanged(int type, int state, int reserved)
```

Notify encryption status

Parameters:

type-

0x00 - terminal

0x01 - the current channel

state-

0x00 - disable

0x01 - enable

reserved -reserved

7.2.6.3 IDmrTListener

Package: com.hytera.call.param.dmrT

The listener for DmrT status change

IDmrTListener Methods

onSetupCallReply

```
void onSetupCallReply(int result, int reserved)
```

Reply of setting up call

Parameters:

result-

- 0x0000 Success
- 0x0001 Failure
- 0x0002 Unregister
- 0x0003 No Authority
- 0x0004 Out of Service
- 0x0005 ID Forbidden
- 0x0006 No Attached
- 0x0007 Receive Only
- 0x0008 No Encryption Devices
- 0x0009 PTT Forbidden
- 0x000A NB Network Closed
- 0x0100 Accepted
- 0x0200 Refused
- 0x8008-AFFF General Error Code

reserved- for protocol extending

onTxDemandReply

```
void onTxDemandReply(int ret,int reserved)
```

Reply for TxDemand

Parameters:

ret-

- 0x0100 Accepted
- 0x0200 Refused

reserved - for protocol extending

onTxCeasedReply

```
void onTxCeasedReply(int ret, int reserved)
```

Reply for TxCeased

Parameters:

ret-

- 0x0100 Accepted
- 0x0200 Refused

reserved - for protocol extending

onHangupCallReply

```
void onHangupCallReply(int ret, int reserved)
```

Reply of hanging up call

Parameters:

ret-

- 0x0100 Accepted
- 0x0200 Refused

reserved - for protocol extending

onAnswerCallReply

```
void onAnswerCallReply(int ret, int reserved)
```

Reply for AnswerCall

Parameters:

ret-

0x0100 Accepted

0x0200 Refused

reserved - for protocol extending

onCallStatusChanged

```
void onCallStatusChanged(IDmrTCall call)
```

Notify call status

Parameters:

call-extends from Call,about handling call

Reference:[IDmrTCall](#)

onCallSetupTimeUpdate

```
void onCallSetupTimeUpdate (IDmrTimerCounter time)
```

Notify the call duration when call setup succeeds

Parameters:

time - IDmrTimerCounter

Reference:[IDmrTimerCounter](#)

onEncryptStateChanged

```
void onEncryptStateChanged(int state, int reserved)
```

Notify encryption states

Parameters:

state-

- 0x00 - disable
- 0x01 - enable
- 0x02-0xFF reserved
- reserved-reserved

7.2.6.4 ITetraDListener

Package: com.hytera.call.param.tetrad

The listener for Tetra Dmo status change

ITetraDListener Methods**onSetupCallReply**

void onSetupCallReply(int result)

Reply of call setup

Parameters:

result-

- 0x0100 Accepted
- 0x0101-0x01FF the result of other process,not the final result
- 0x0200 Refused
- 0x8000 Error
- 0x8001-0xAFFF the error code of extending others

onCallIncoming

void onCallIncoming(ITetraDCall call)

Notify incoming call

Parameters:

call - extends from Call, about handling call

Reference: [ITetraDCall](#)

onCallConnect

```
void onCallConnect(ITetraDCall call)
```

Indicate the connected status

Parameters:

call - extends from Call, about handling call

Reference: [ITetraDCall](#)

onCallingPartyNotification

```
void onCallingPartyNotification(ITetraDCall call)
```

Indicate the CallingParty

Parameters:

call-extends from Call, about handling call

Reference: [ITetraDCall](#)

onTxDemandReply

```
void onTxDemandReply(int result)
```

Reply for TxDemand

Parameters:

result-

0x0100 Accepted

0x0101-0x01FF the result of other process,
not the final result

0x0200 Refused

0x8000 Error

0x8001-0xAFFF the error code of extending others

onTxCeaseReply

```
void onTxCeaseReply(int result)
```

Reply for TxCease

Parameters:

result–

0x0100 Accepted

0x0101-0x01FF the result of other process,
not the final result

0x0200 Refused

0x8000 Error

0x8001-0xAFFF the error code of extending others

onTxCeasedNotification

```
void onTxCeasedNotification(int ceaseCause, int priorityLevelFlag,  
                             int priorityLevel, ITetraDCall call)
```

Notify the status of TxCeased

Parameters:

ceaseCause –

0 Cause not defined or unknown

1 Normal end of transmission

2 Pre-emptive use of resource

3 Transmission time limit reached

4 Channel now in reservation

5 UpLink Transmission Ceased

priorityLevelFlag -

0 Priority Level not present

1 Priority Level present

priorityLevel - if *priorityLevelFlag* = 1

0 Normal priority call

1 High priority call

2 Pre-emptive priority call

3 Emergency priority call

call-extends from *Call*, about handling call

Reference: [ITetraDCall](#)

onChannelStatusUpdate

```
void onChannelStatusUpdate(int report,int priority,
                           int occupiedType)
```

Notify channel status

Parameters:

report -

0 Changeover or preemption failure

1 Changeover or preemption rejected

2 State change, CALL ACTIVE Tx to CALL ACTIVE Rx

3 DMCC is busy,

Please wait for the confirmation of the previous request

- 4 DMCC is busy,
DMCC waits for response to previous indication
- 5 Changeover or pre-emption aborted,
No transmission authorized
- 6 Channel Free
- 7 Channel Reserved
- 8 Channel Occupied
- 9-63 Reserved
- 64 Message not allowed/unsupported
- 65 Out of time for response message
- 66 Wrong destination device address
- 67 Device data unknown
- 68 Minimum sending power higher than allowed
- 69 Unable to send
- 70 Incorrect formatted request
- 71 Channel-state changed
- 72 Restricted service
- 73-254 Reserved
- 255 DMO-Stack was deactivated by DSP

Priority -

- 0 Normal Priority call
- 1 High priority call
- 2 Pre-emptive priority call
- 3 Emergency pre-emptive priority call
- 4 Unknown Priority

occupiedType -

- 0 Initial scanning

- 1 Sending mobile
- 2 Unknown DMO System Code
- 3 Device mismatch
- 4 Encrypted call
- 5 No signal found
- 6 Device status
- 7 Transmission on channel
- 8 Call setup transaction by gateway

onHangupReply

```
void onHangupReply(int resultCode)
```

Reply of hanging up call

Parameters:

resultCode-

- 0x0100 Accepted
- 0x0101-0x01FF the result of other process,
not the final result
- 0x0200 Refused
- 0x8000 Error
- 0x8001-0xAFFF the error code of extending others

onCallRelease

```
void onCallRelease(ITetraDCall call)
```

Notify the call release status

Parameters:

*call-*extends from Call, about handling call

Reference: [ITetraDCall](#)

onCallOutNotification

```
void onCallOutNotification(ITetraDCall call)
```

Response of outgoing call

Parameters:

call-extends from Call,about handling call

Reference: [ITetraDCall](#)

onTxGranted

```
void onTxGranted (ITetraDCall call)
```

Indicate the TxGrant status

Parameters:

call - extends from Call,about handling call

Reference: [ITetraDCall](#)

onE2EEModeChanged

```
void onE2EEModeChanged(int netMode, int e2eeType)
```

Notify E2EEstatus

Parameters:

netMode-

0 - TMO

1 - DMO

e2eeType-

0 - Not Encryption

1 - Software Encryption

2 - Hardware Encryption

7.2.6.5 ITetraTListener

Package: com.hytera.call.param.tetrat

The listener for Tetra Tmo status change

ITetraTListener Methods

onSetupCallReply

```
void onSetupCallReply(int result)
```

Reply of setting up call

Parameters:

result -

0x0100 Accepted

0x0101-0x01FF the result of other process,
not the final result

0x0200 Refused

0x8000Error

0x8001-0xAFFF the error code of extending others

onCallAnswerReply

```
void onCallAnswerReply(int result)
```

Reply of answering call

Parameters:

result -

0x0100 Accepted

0x0101-0x01FF the result of other process,

not the final result

0x0200 Refused

0x8000Error

0x8001-0xAFFF the error code of extending others

onCallIncoming

```
void onCallIncoming(ITetraTCall call)
```

Notify incoming call

Parameters:

call-extends from Call,about handling call

Reference: [ITetraTCall](#)

onCallConnect

```
void onCallConnect(ITetraTCall call)Indicate in the connected status
```

Parameters:

call- extends from Call,about handling call

Reference: [ITetraTCall](#)

onCallOutReply

```
void onCallOutReply(ITetraTCall call)
```

Reply of outgoing Call

Parameters:

call - extends from Call,about handling call

Reference: [ITetraTCall](#)

onOutgoingCallProgressNotificaiton

```
void onOutgoingCallProgressNotificaiton(ITetraTCall call)
```

Notify the status of the current outgoing call

Parameters:

call - extends from Call, about handling call

Reference: [ITetraTCall](#)

onTxDemandReply

```
void onTxDemandReply(int result)
```

Reply for TxDemand

Parameters:

result -

0x0100 Accepted

0x0101-0x01FF the result of other process,
not the final result

0x0200 Refused

0x8000 Error

0x8001-0xAFFF the error code of extending others

onTxCeaseReply

```
void onTxCeaseReply(int result)
```

Reply for TxCease

Parameters:

result -

0x0100 Accepted

0x0101-0x01FF the result of other process,
not the final result

0x0200 Refused

0x8000 Error

0x8001-0xAFFF the error code of extending others

onTxGranted

```
void onTxGranted(ITetraTCall call)
```

Notify the Tx status

Parameters:

call - extends from Call, about handling call

Reference: [ITetraTCall](#)

onHangupCallReply

```
void onHangupCallReply(int result)
```

Reply of hanging up call

Parameters:

result -

0x0100 Accepted

0x0101-0x01FF the result of other process,
not the final result

0x0200 Refused

0x8000 Error

0x8001-0xAFFF the error code of extending others

onCallRelease

```
void onCallRelease(  
    int CCInstance, int releaseCause, ITetraTCall call)
```

Notify the call release status

Parameters:

CCInstance - 0-3 default = 0

releaseCause -

- 0 Not defined or unknown
- 1 User request
- 2 Called party busy
- 3 Called party not reachable
- 4 Called party does not support encryption
- 5 Network congestion
- 6 Not allowed traffic
- 7 Incompatible traffic
- 8 Service not available
- 9 Preemption
- 10 Invalid call identifier
- 11 Called party rejection
- 12 No CC entity
- 13 Timer expiry
- 14 SwMI disconnect
- 15 No acknowledgement
- 16 Unknown TETRA identity
- 17 Supplementary Service dependent
- 18 Unknown external subscriber number
- 19 Call restoration failed
- 20 Called party requires encryption
- 21 Concurrent set-up not supported
- 22 Called party is under the same DM-GATE of the calling party
- 100 Not activated (Not for TMO)
- 101 Not registered (Not for DMO)
- 102 Default Group Not Attached
- 103 Tx Forbidden
- 104 Default Group is null
- 105 Only Receive

- 106 Group Call Forbidden
- 107 Individual Call Forbidden
- 108 All Call Forbidden
- 109 Broadcast Call Forbidden
- 110 Request Audio Resource Fail
- 111 NB Network Closed
- 112 Device Shut Down
- 113 Temporarily disabled
- 114 Program
- 115 Invalid Number
- 116 Host Deny
- 117 PS Exception
- 118 GSDM(Not for DM0)
- 119-149 Not Defined

call - extends from Call, about handling call

Reference: [ITetraTCall](#)

onAIEIndicationAck

```
void onAIEIndicationAck(
    int cCInstance, boolean isAIEEncryption, boolean isSilentMode)
```

Indicate AIE status and Silent Mode

Parameters:

cCInstance - 0-3 default = 0

isAIEEncryption - 0 No AI encryption
1 AI encryption

isSilentMode - 0 Not Silent Mode

1 Silent Mode

onEmergencyAlarmStatusChanged

```
void onEmergencyAlarmStatusChanged(int status, int endCause)
```

Notify the emergency alarm status

Parameters:

status - 0x00 EmgAlarm start

0x01 EmgCall start

0x02 EmgCall end

0x03 EmgAlarm end

endCause - 0 exit

1 NB network closed

2 Temporarily disabled

3 Not activated

4 Not register

5 Forbidden

6 Not Defined

onPreSetupCall

```
void onPreSetupCall(ITetraTCall call)
```

Notify the call is pre set up call

Parameters:

call - extends from Call, about handling call

Reference: [ITetraTCall](#)

onE2EEModeChanged

```
void onE2EEModeChanged(int netMode, int e2eeType)
```


Notify E2EE encryption states

Parameters:

netMode-

0 - TMO

1 - DMO

e2eeType-

0 - Not Encryption

1 - Software Encryption

2 - Hardware Encryption

7.2.6.6 iPhoneCallListener

Package: com.hytera.call.param.phone

The listener for Phone Call status change

iPhoneCallListenerMethods

onPhoneBind

`void onPhoneBind(boolean bind)`

Called when Phone is bound.

Parameters:

bind- bind state

onAudioMode

`void onAudioMode(int newMode)`

Called when audio mode is changed

Parameters:

newMode- the new audio mode

onMute

```
void onMute(boolean muted)
```

Called when mute state is changed

Parameters:

muted- mute state

onSupportedAudioMode

```
void onSupportedAudioMode(int modeMask)
```

Parameters:

modeMask-mode mask

onIncomingCall

```
void onIncomingCall(IPhoneCall call)
```

Notify incoming call

Parameters:

call- extends from Call, handling phonecall

Reference: [IPhoneCall](#)

onCallUpdate

```
void onCallUpdate(IPhoneCall call)
```

Called when call is updated

Parameters:

call- extends from Call, handling phonecall

Reference: [IPhoneCall](#)

onDisconnect

```
void onDisconnect(IPhoneCall call)
```

Called when call disconnects

Parameters:

call- extends from Call, handling phonecall

Reference: [IPhoneCall](#)

onBringToFrontground(not work,reserved)

```
void onBringToFrontground(boolean showDialpad)
```

Parameters:

showDiapad-whether dialpad had shown

onSetupCallReply

```
void onSetupCallReply(int reply, OutgoingCallParam param)
```

Reply of setting up call

Parameters:

reply - 0 success,

1 timeout

2 other reason

param - OutgoingCallParam of call

Reference: [OutgoingCallParam](#)

7.2.6.7 IRoipCallListener

Package Name:com.hytera.call.param.roip

The listener for RoipTetra Call status change

IRoipCallListener Methods

onCallIncoming

```
void onCallIncoming(IRoipCall call)
```

Notify incoming call

Parameters:

call- extends from Call, handling RoipTetraCall

Reference:[IRoipCall](#)

onCalledRinging

```
void onCalledRinging(IRoipCall call)
```

Called when call is ringing

Parameters:

call- extends from Call, handling RoipTetraCall

Reference:[IRoipCall](#)

onCallSetup

```
void onCallSetup(IRoipCall call)
```

Is called when call is set up

Parameters:

call- extends from Call, handling RoipTetraCall

Reference:[IRoipCall](#)

onCallQueued

```
void onCallQueued(IRoipCall call)
```

Is called when call is in queue state

Parameters:

`call`- extends from `Call`, handling `RoipTetraCall`

Reference:[IRoipCall](#)

onTxGranted

```
void onTxGranted(IRoipCall call)
```

Called when Tx is granted

Parameters:

`call`- extends from `Call`, handling `RoipTetraCall`

Reference:[IRoipCall](#)

onCallResumed

```
void onCallResumed(IRoipCall call)
```

Called when call is resumed

Parameters:

`call`- extends from `Call`, handling `RoipTetraCall`

Reference:[IRoipCall](#)

onTxDemandQueued

```
void onTxDemandQueued(IRoipCall call)
```

Called when TxDemand is in queue state

Parameters:

`call`- extends from `Call`, handling `RoipTetraCall`

Reference:[IRoipCall](#)

onTxCeased

```
void onTxCeased(IRoipCall call)
```

Called when Tx is ceased

Parameters:

call- extends from Call, handling RoipTetraCall

Reference:[IRoipCall](#)

onTxGranted2Others

```
void onTxGranted2Others(IRoipCall call)
```

Called when Tx is granted to others

Parameters:

call- extends from Call, handling RoipTetraCall

Reference:[IRoipCall](#)

onTxNotGranted

```
void onTxNotGranted(IRoipCall call)
```

will be called when Tx is not Granted

Parameters:

call- extends from Call , handling RoipTetraCall

Reference:[IRoipCall](#)

onCallEstablished

```
void onCallEstablished(IRoipCall call)
```

Called when call is established

Parameters:

call- extends from Call, handling RoipTetraCall

Reference: [IRoipCall](#)

onCallError

```
void onCallError(  
    IRoipCall call, CC_DISCONNECT_CAUSE result, java.lang.String cause)
```

will be called when call is error

Parameters:

call- extends from Call , handling RoipTetraCall

cause- error cause

Reference: [IRoipCall](#) , [CC_DISCONNECT_CAUSE](#)

onCallCanceled

```
void onCallCanceled(IRoipCall call, CC_DISCONNECT_CAUSE result)
```

will be called when call is canceled

Parameters:

call- extends from Call , handling RoipTetraCall

result- result

Reference: [IRoipCall](#) , [CC_DISCONNECT_CAUSE](#)

onTxDemanded

```
void onTxDemanded(  
    IRoipCall call, PROTOCOL_RESULT result, java.lang.String cause)
```

Will be called when demanded Tx

Parameters:

call- extends from Call , handling RoipTetraCall

result- result

cause - cause

Reference: [IRoipCall](#) ,[PROTOCOL_RESULT](#)

onTxInterrupted

void onTxInterrupted(IRoipCall call)

Called when Tx is interrupted

Parameters:

call- extends from Call, handling RoipTetraCall

Reference: [IRoipCall](#)

onCallReleased

void onCallReleased(IRoipCall call)

Called when call is released

Parameters:

call- extends from Call, handling RoipTetraCall

Reference: [IRoipCall](#)

7.2.6.8 IRoipMsgListener

Package Name:com.hytera.call.param.roip

The listener for RoipTetra Msg status change

IRoipMsgListenerMethods

onReceivedShortData

void onReceivedShortData(IRoipMsgData data)

Reference: [IRoipMsgData](#)

onShortDataSent

```
void onShortDataSent(long tmpId, SDS_RESULT resultCode, String cause)
```

Reference: [SDS_RESULT](#)

onShortDataConfirmedBySystem

```
void onShortDataConfirmedBySystem(long tmpId, long systemId)
```

onShortDataConfirmedByReceiver

```
void onShortDataConfirmedByReceiver(long systemId)
```

onReceivedStatus

```
void onReceivedStatus(IRoipMsgStatus data)
```

Reference: [IRoipMsgStatus](#)

onStatusSent

```
void onStatusSent(long tmpId, SDS_RESULT resultCode, String cause)
```

Reference: [SDS_RESULT](#)

onShortDataConfirmedByUser

```
void onShortDataConfirmedByUser(byte resultCode)
```

onShortDataNotDelivered

```
void onShortDataNotDelivered(byte resultCode)
```

onStatusConfirmedBySystem

```
void onStatusConfirmedBySystem(long tmpId, long systemId)
```

onStatusConfirmedByReceiver

```
void onStatusConfirmedByReceiver(long systemId)
```

7.2.6.9 UINotificationListener

Package Name: com.hytera.call.param.notification

The listener of Notification

UINotificationListenerMethods

onAttemptStartIncallUI

```
void onAttemptStartIncallUI(Call call)
```

Called for starting UI of Hytera Call Applicance

Parameters:

call- handling Call

Reference: [Call](#)

7.2.6.10 UIChangedListener

Package Name: com.hytera.call.param.uevent

The listener for UIstatus changes

UIChangedListenerMethods

onCurrentShowCallChanged

```
void onCurrentShowCallChanged(Call oldCall, Call newCall)
```

Called when the current call is changed

It indicates call is ended or call is null if CallId is CallIDGenerator.INVALID_CALL_ID

Parameters:

oldCall - old Call

newCall -new Call

Reference: [Call](#)

onCallDisplayedModeChanged

```
void onCallDisplayedModeChanged(  
    java.lang.String callId, int oldMode, int newMode)
```

Called when call display mode is changed

Parameters:

callId - call Id

oldMode - old mode of UI

newMode -new mode of UI

Reference: [CallDisplayedMode](#)

7.2.6.11 PhysicalPttListener

Package Name: com.hytera.call.param.dsp

The listener for PTT status change

PhysicalPttListener Methods

onPhysicalPtt

```
void onPhysicalPtt(boolean down)
```

Called when PTT is down or up

Parameters:

down-whether is down

7.2.6.12 IAudioModeListener

Package Name: com.hytera.call.param.dsp

The listener for audio mode status change

IAudioModeListener Mehods

onUnsolAudioModeChanged

```
void onUnsolAudioModeChanged (int mode)
```

Called when audio mode is changed

Parameters:

mode- audio mode, refer to CallAudioState

onUnsolModeListChanged

```
void onUnsolModeListChanged (int[] modeList)
```

Called when audio mode list is changed

Parameters:

modeList - audio mode list, refer to CallAudioState

onSolGetAudioModeData

```
void onSolGetAudioModeData (IAudioModeData data)
```

Get the data of audio mode. It is called when calling `CallManager.getAudio`.

ModeData

Parameters:

data- data of audio mode

Reference: [IAudioModeData](#)

7.2.6.13 CallRecordListener

Package Name: com.hytera.call.param.notification

The listener for callRecordstatus change

CallRecordListener Methods

onRecordError

```
void onRecordError(java.lang.String callId,int errorcode)
```

Called when recording has error

Parameters:

called- call id

errorcode- error code

Reference: [CallRecord.RecordError](#)

onRecordState

```
void onRecordState(  
    java.lang.String callId, long durationtime, int recordstate)
```

Notify the recording status

Parameters:

callId- call id

durationtime- the time of record

recordstate- the status of record

Reference: [CallRecord.RecordState](#)

onGetCallRecordingState

```
void onGetCallRecordingState(  
    java.lang.String callId,int recordstate)
```

Get the call recording status. It is called when calling `CallManager.getCallRecordingState`.

Parameters:

callId- call id

recordstate- the status of recording

Reference: [CallRecord.RecordState](#)

onGetIsCallRecorderEnabled

```
void onGetIsCallRecorderEnabled(  
    java.lang.String callId,boolean enable)
```

Whether enable call recording or not.It is called when calling `CallManager.getIsCallRecorderEnabled`

Parameters:

callId- call id

enable- whether enable to start call record or not

onGetCallRecordingTime

```
void onGetCallRecordingTime(  
    ICallRecordTimeInfo callRecordTimeInfo)
```

Get the recording time. It is called when calling `CallManager.getCallRecordingTime`

Parameters:

callRecordTimeInfo-info of recording time

Reference: [ICallRecordTimeInfo](#)

7.2.6.14 IHytKeyEvent

The listener for key status change

IHytKeyEventMethods

onKey

boolean onKey(KeyEvent event)

Notify the event of keys. Specifically, if TP returns true when the event is HytKeyCode.KEY_CODE_BACK, the HYT Call Application will not process the HytKeyCode.KEY_CODE_BACK event to achieve some operations like answering incoming call.

Parameters:

event-the info of key event

7.2.7 Calls

7.2.7.1 Call

Package Name: com.hytera.call.param

The class provides methods to deal with call

Call Direct Subclasses

IDmrCCall, IDmrTCall, IPhoneCall, IPocVoiceCall, IRoipCall, IRoipCall, ITetraDCall, ITetraTCall

Call Constant Fields

Qualifier and Type	Constant fields	value
--------------------	-----------------	-------

public static final int	COMMON_STATE_INVALID	-1
public static final int	COMMON_STATE_INCOMING	0
public static final int	COMMON_STATE_OUTGOING	1
public static final int	COMMON_STATE_RX	2
public static final int	COMMON_STATE_TX	3
public static final int	COMMON_STATE_HANGTIME	4
public static final int	COMMON_STATE_DUPLEX_VOICE	5
public static final int	COMMON_STATE_CALL_END	6

Call Methods

setCanHangup

```
public final void setCanHangup(boolean canHangup)
```

Set whether call can hangup

Parameters:

canHangup-false or true

isCanHangup

```
public final boolean isCanHangup()
```

Get hangup flag

setIsMoCall


```
public final void setIsMoCall(boolean isMoCall)
```

Set whether to be the calling party in a call.

isMoCall

```
public final boolean isMoCall()
```

Get Mocal flag

getCallId

```
public final java.lang.String getCallId()
```

Get the callidentity, each call has a unique ID.

setCallState

```
public void setCallState(int callState)
```

Set the call status

Parameters:

callState -the state of call

getCallState

```
public int getCallState()
```

Get call status

getContactIconUri

```
public java.lang.String getContactIconUri()
```

Get uri of contact Icon

setContactIconUri

```
public void setContactIconUri(java.lang.String contactIconUri)
```

Set uri of contact Icon

Parameters:

contactIconUri - uri of contactIcon

getRingDialTime

```
public long getRingDialTime()
```

Get ring time

setRingDialTime

```
public void setRingDialTime(long ringDialTime)
```

Set ring time

getConnectTime

```
public long getConnectTime()
```

Get connecting time

setConnectTime

```
public void setConnectTime(long connectTime)
```

Set connecting time

getDisConnectTime

```
public long getDisConnectTime()
```

Get disconnected time

setDisConnectTime

```
public void setDisConnectTime(long disConnectTime)
```

Set disconnected time

getPreviousCallState

```
public int getPreviousCallState()
```

Get previous call status

setCallReleaseReason

```
public final void setCallReleaseReason(int releaseCause)
```

Set reason of releasing call

Parameters:

releaseCause - reason of releasing call

getCallReleaseReason

```
public final int getCallReleaseReason()
```

Get reason of releasing call

setIsNotNormalRelease

```
public void setIsNotNormalRelease()
```

set call release abnormal and it can lead to display the reason of error in UI

setCallType

```
public void setCallType(int callType)
```

Set call type

Parameters:

callType - type of call, such as individual call, group call
and so on

Reference: [CallType](#)

getCallType

```
public int getCallType()
```

Get call type

Reference: [CallType](#)

setCallId

```
public void setCallId(java.lang.String callId)
```

Set the call identity, each call has a unique ID.

Parameters:

callId - the identity of call

setCallMode

```
public void setCallMode(int callMode)
```

Set call mode

Parameters:

callMode - call mode, such as FULL_DUPLEX, HALF_DUPLEX.

Reference: [CallMode](#)

getCallMode

```
public int getCallMode()
```

Get call mode, such as FULL_DUPLEX, HALF_DUPLEX.

Reference: [CallMode](#)

getNetType

```
public abstract int getNetType()
```

Get callnetType

Reference: [NetType](#)

setContactNumber

```
public void setContactNumber(java.lang.String number)
```

Set contact number

Parameters:

number – group number or individual number

getContactNumber

```
public java.lang.String getContactNumber()
```

Get contact number

getDisplayedContactName

```
public java.lang.String getDisplayedContactName()
```

Get displayed contact name: return contact alias, or return contact number if the alias is empty.

getDisplayedSpeakerName

```
public java.lang.String getDisplayedSpeakerName()
```

Get displayed speaker name: return speaker alias, or return number if the alias is empty.

getPreviousDisplaySpeakerName

```
public java.lang.String getPreviousDisplaySpeakerName()
```

Get the previous displayed speaker name: return speaker alias or return number if the alias is empty.

getDisplayedCallerName

```
public java.lang.String getDisplayedCallerName()
```

Get displayed caller name, return alias if the caller has, otherwise return number.

setCallerNumber

```
public void setCallerNumber(java.lang.String callerNumber)
```

Set caller number

Parameters:

number - number of caller

getCallerNumber

```
public java.lang.String getCallerNumber()
```

Get callernumber, only valid in group call

setSpeakerNumber

```
public void setSpeakerNumber(java.lang.String number)
```

Set speaker number

Parameters:

number - number of speaker

getSpeakerNumber

```
public java.lang.String getSpeakerNumber()
```

Get speaker number

getPreviousSpeakerNumber

```
public java.lang.String getPreviousSpeakerNumber()
```

Get the previous speaker number

getContactAlias

```
public java.lang.String getContactAlias()
```

Get contact alias

setContactAlias

```
public void setContactAlias(java.lang.String contactAlias)
```

Set contact alias

Parameters:

contactAlias - alias of contact

getCallerAlias

```
public java.lang.String getCallerAlias()
```

Get caller alias

setCallerAlias

```
public void setCallerAlias(java.lang.String callerAlias)
```

Set caller alias

Parameters:

contactAlias – alias of caller

getSpeakerAlias

```
public java.lang.String getSpeakerAlias()
```

Get speaker alias

getPreviousSpeakerAlias

```
public java.lang.String getPreviousSpeakerAlias()
```

Get the previous speaker alias

setSpeakerAlias

```
public void setSpeakerAlias(java.lang.String speakerAlias)
```

Set speaker alias

Parameters:

number – alias of speaker

setCallPriority


```
public void setCallPriority(int callPriority)
```

Set call priority

Parameters:

callPriority-call priority

Reference: [CallPriority](#)

getCallPriority

```
public int getCallPriority()
```

Get call priority

Reference: [CallPriority](#)

setIsE2EE

```
public void setIsE2EE(boolean isE2EE)
```

Set whether the call is E2EE encrypted

Parameters:

isE2EE-whether call is E2EE Encrypted

isE2EE

```
public boolean isE2EE()
```

Get state of E2EE

setIsHook

```
public void setIsHook(boolean isHook)
```

Set whether call is Hook

isHook

```
public boolean isHook()
```

Get hook status

isInCall

```
public abstract boolean isInCall()
```

Return true if in call, else return false.If the call is put through, then enters in call status,including process such as Tx, Rx,hangtime,queuing etc.

isCallEnd

```
public abstract boolean isCallEnd()
```

Return true if call ended, else return false

isTx

```
public abstract boolean isTx()
```

Return true if in Tx status, else return false

isRx

```
public abstract boolean isRx()
```

Return true if in Rx status, else return false

isDialing

```
public abstract boolean isDialing()
```

Return true if in dialing status, else return false

isIncomingCall

```
public abstract boolean isIncomingCall()
```

Return true if in incoming call status, else return false

isHangTime

```
public abstract boolean isHangTime()
```

Return true if in hangtime status, else return false

isPttEnable

```
public abstract boolean isPttEnable()
```

Return true if PTT enabled, else return false

isCallSetupQueuing

```
public abstract boolean isCallSetupQueuing()
```

Return true if in call queuing status, else return false

isEmergencyCall

```
public abstract boolean isEmergencyCall()
```

Return true if the call is emergency, else return false

isNormalCallEnd

```
public boolean isNormalCallEnd()
```

Return true if the call ends normally, else return false

hashCodeString

```
public java.lang.String hashCodeString()
```

Return hashCode

equals

```
public boolean equals(java.lang.Object call)
```

Whether the call is the same.

isTheSameTypeAndNumberAs

```
public boolean isTheSameTypeAndNumberAs(Call call)
```

Whether the type and number are the same

isFullDuplex

```
public boolean isFullDuplex()
```

Return true if the call is FullDuplex, else return false

hasPendingSetupCall

```
public abstract boolean hasPendingSetupCall()
```

Whether the call is pending for set up

isPhoneCall

```
public boolean isPhoneCall()
```

Whether call nettype is NetType.NET_TYPE_PHONE

isRoipCall

```
public boolean isRoipCall()
```

Whether call network type is NetType.NET_TYPE_ROIP_TETRA or NetType.NET_TYPE_ROIP_DMR

isNarrowBandCall

```
public boolean isNarrowBandCall()
```

Whether the call is a narrowband call

isThirdPartyCall

```
public boolean isThirdPartyCall()
```

Whether the call network type is NetType.NET_TYPE_THIRD_PARTY_VOICE or NetType.NET_TYPE_THIRD_PARTY_VIDEO

describeContents

```
public int describeContents()
```

Describe contents

writeToParcel

```
public void writeToParcel(Parcel out,int flags)
```

toString

```
public java.lang.String toString()
```

clone

```
public java.lang.Object clone()
```

7.2.7.2 IDmrCCall

Package Name: com.hytera.call.param.dmrcc

Extended from [Call](#), used to handle DmrCCall.

IDmrCCall Nested class

IDmrCCall.State

IDmrCCall.State field

Qualifier and Type	Constant fields	Value
public static final int	INVALID	0
public static final int	VOICE_TX	1
public static final int	HANG_TIME_TX	2
public static final int	CALL_END_TX	3
public static final int	CALL_FAIL_TX	4
public static final int	TOT	5
public static final int	TOT_PRE_ALERT_TIME	6
public static final int	CALL_CONFIRM	7
public static final int	SOURCE_INVALID	0x00
public static final int	SOURCE_CHANNEL_BUSY	0x01
public static final int	SOURCE_RX_ONLY	0x02
public static final int	SOURCE_LOW_BATTERY	0x03

public static final int	SOURCE_UNLOCK	0x04
public static final int	SOURCE_PRIVATE_CALL_NO_ACK	0x05
public static final int	SOURCE_REPEATER_WAKEUP_FAIL	0x06
public static final int	SOURCE_NO_CONTACT	0x07
public static final int	SOURCE_IGNITION_PTT_DISABLE	0x08
public static final int	SOURCE_TOT_REKEY	0x09
public static final int	SOURCE_TX_DENY	0x0a
public static final int	SOURCE_TX_INTERRUPTED	0x0b
public static final int	SOURCE_TX_ALARM_NO_ACK	0x0c
public static final int	SOURCE_NB_OFF	0x0d
public static final int	VOICE_RX	0x21
public static final int	HANG_TIME_RX	0x22
public static final int	CALL_END_RX	0x23

IDmrCCall.State methods

isInCall

```
public static boolean isInCall(
    int state, int source, int previousState)
```

Whether the call is in call status

Parameters:

state– call state

source– source

previousState– previous call state

Reference: [IDmrCCall.State](#)

isInCall

```
public static boolean isInCall(int state, int source)
```

Whether the call is in call status

Parameters:

state– call state

source– source

Reference: [IDmrCCall.State](#)

isCallEnd

```
public static boolean isCallEnd(  
                                int state, int source, int previousState)
```

Whether the call has ended

Parameters:

state– call state

source– source

previousState– previous call state

Reference: [IDmrCCall.State](#)

isCallEnd

```
public static boolean isCallEnd(int state, int source)
```


Whether the call has ended

Parameters:

state– call state

source– source

Reference: [IDmrCCall.State](#)

isFailCallEnd

```
public static boolean isFailCallEnd (  
                                int state, int source, int previousState)
```

Whether the call failed

Parameters:

state– call state

source– source

previousState– previous call state

Reference: [IDmrCCall.State](#)

isTx

```
public static boolean isTx (int state)
```

Whether the call is in Tx status

Parameters:

state– call state

Reference: [IDmrCCall.State](#)

isRx

```
public static boolean isRx (int state)
```

Whether in the call is in Rx status

Parameters:

state– call state

Reference: [IDmrCCall.State](#)

isHangTime

```
public static boolean isHangTime (int callState)
```

Whether the call is in hangtime status

Parameters:

state– call state

Reference: [IDmrCCall.State](#)

IDmrCCall.AnalogState

IDmrCCall.AnalogState field

Qualifier and Type	Constant fields	Value
public static final int	INVALID	0
public static final int	VOICE_TX	1
public static final int	AUDIO_RESET_MODE	2
public static final int	TX_CALL_FAIL	3
public static final int	TOT	4
public static final int	TOT_PRE_ALERT_TIME	5
public static final int	SOURCE_INVALID	0x00

public static final int	SOURCE_CHANNEL_BUSY	0x01
public static final int	SOURCE_UNLOCK	0x02
public static final int	SOURCE_TOT_REKEY	0x03
public static final int	SOURCE_TX_DENY	0x04
public static final int	SOURCE_NB_OFF	0x05
public static final int	VOICE_RX	0x21
public static final int	CALL_END	0x22

IDmrCCall.AnalogStateMethods

isInCall

```
public static boolean isInCall(int state, int source)
```

Whether the call is in call status

Parameters:

state– call state

source– source

Reference: [IDmrCCall.AnalogState](#)

isCallEnd

```
public static boolean isCallEnd(int state, int source)
```

Whether the call has ended

Parameters:

state– call state

source– source

Reference: [IDmrCCall.AnalogState](#)

isFailCallEnd

```
public static boolean isFailCallEnd (int state, int source)
```

Whether the call fails

Parameters:

state– call state

source– source

Reference: [IDmrCCall.AnalogState](#)

isTx

```
public static boolean isTx (int state)
```

Whether the call is in Tx state

Parameters:

state– call state

Reference: [IDmrCCall.AnalogState](#)

isRx

```
public static boolean isRx (int state)
```

Whether the call is in Rx status

Parameters:

state– call state

Reference: [IDmrCCall.AnalogState](#)

isHangTime

```
public static boolean isHangTime (int callState)
```

Whether the call is in hangtime status

Parameters:

state– call state

Reference: [IDmrCCall.AnalogState](#)

IDmrCCall.AnalogEmergencyState

Qualifier and Type	Constant fields	Value
public static final int	INVALID	0
public static final int	VOICE_TX	1
public static final int	STAY	2
public static final int	FAIL	3
public static final int	VOICE_RX	4
public static final int	CALL_END	5
public static final int	SOURCE_INVALID	0x00
public static final int	SOURCE_UNLOCK	0x01
public static final int	SOURCE_TX_DENY	0x02
public static final int	SOURCE_NB_OFF	0x03

IDmrCCall.AnalogEmergencyStateMethods

isInCall

```
public static boolean isInCall(int state, int source)
```

Whether the call is in call status

Parameters:

state– call state

source– source

Reference: [IDmrCCall.AnalogEmergencyState](#)

isCallEnd

```
public static boolean isCallEnd(int state, int source)
```

Whether the call has ended

Parameters:

state– call state

source– source

Reference: [IDmrCCall.AnalogEmergencyState](#)

isFailCallEnd

```
public static boolean isFailCallEnd (int state, int source)
```

Whether the call failed

Parameters:

state– call state

source– source

Reference: [IDmrCCall.AnalogEmergencyState](#)

isTx

```
public static boolean isTx (int state)
```

Whether the call is in Tx status

Parameters:

state- call state

Reference: [IDmrCCall.AnalogEmergencyState](#)

isRx

```
public static boolean isRx (int state)
```

Whether the call is in Rx status

Parameters:

state- call state

Reference: [IDmrCCall.AnalogEmergencyState](#)

isHangTime

```
public static boolean isHangTime (int callState)
```

Whether the call is in hangtime status

Parameters:

state- call state

Reference: [IDmrCCall.AnalogEmergencyState](#)

IDmrCCall Methods

canForceChangeToTx

```
public boolean canForceChangeToTx()
```

Whether the call could be forced to enter Tx status

getSource

```
public int getSource()
```

Get source

Reference: [IDmrCCall.State,](#)

[IDmrCCall.AnalogState,](#)

[IDmrCCall.AnalogEmergencyState](#)

setSource

```
public void setSource(int mSource)
```

Set source.

Parameters:

mSource- the reason of tx failed

Reference:[IDmrCCall.State,](#)

[IDmrCCall.AnalogState,](#)

[IDmrCCall.AnalogEmergencyState](#)

getOperationType

```
public int getOperationType()
```

OperationType-

0x00 - Normal Digital Call

0x01 - Emergency Call (Group)

0x02 - Emergency Alarm (Group)

0x03 - Remote Monitor

setOperationType

```
public void setOperationType(int mOperationType)
```

OperationType-

0x00 - Normal Digital Call

0x01 - Emergency Call (Group)

0x02 - Emergency Alarm (Group)

0x03 - Remote Monitor

isEmergencyCall

```
public boolean isEmergencyCall()
```

Return true if the call is emergency call

getChannelAlias

```
public java.lang.String getChannelAlias()
```

Get channel alias

describeContents

```
public int describeContents()
```

Override:

[describeContents](#) in [Call](#)

getNetType

```
public int getNetType()
```

Override:

[getNetType](#) in [Call](#)

isInCall

```
public boolean isInCall()
```

Override:

[isInCall](#) in [Call](#)

isAnalogNormalCall

```
public boolean isAnalogNormalCall()
```

Whether the call is normal analog call

isAnalogEmergencyCall

```
public boolean isAnalogEmergencyCall()
```

Whether the call is analog emergency call

isDigitalCall

```
public boolean isDigitalCall()
```

Whether the call is digital call

isCallEnd

```
public boolean isCallEnd()
```

Whether the call is in call end status

Override:

[isCallEnd](#) in [Call](#)

getCallMode

```
public int getCallMode()
```

Get call mode

Override:

[getCallMode](#) in [Call](#)

setIsPendingSetupCall

```
public void setIsPendingSetupCall(boolean isPendingSetupCall)
```

Set whether the call is pending for set up

Parameters:

isPendingSetupCall- whether is pending setup call

hasPendingSetupCall

```
public boolean hasPendingSetupCall()
```

Whether the call is pending for set up

getAnalogCallRelReasonStr

```
public static final java.lang.String getAnalogCallRelReasonStr(  
    Context context, int releaseCause)
```

Get the release reason of analog call

Parameters:

context- context

releaseCause-the cause code of release,can be get by the method
`Call.getCallReleaseReason`

getAnalogEmergencyCallRelReasonStr

```
public static final java.lang.String  
    getAnalogEmergencyCallRelReasonStr(  
        Context context,int releaseCause)
```

Get the release reason of analog emergency call

Parameters:

context- context

releaseCause-the cause code of release,can be get by the method
`Call.getCallReleaseReason`

getCallRelReasonStr

```
public static final java.lang.String getCallRelReasonStr(Context context,  
                                                         int releaseCause)
```

Get the release reason of call

Parameters:

context- context

releaseCause-the cause code of release,can be get by the method
`Call.getCallReleaseReason`

toString

```
public java.lang.String toString()
```

isDialing

```
public boolean isDialing()
```

Return true if the call is dialing

Override:

[isDialing](#) in [Call](#)

isIncomingCall

```
public boolean isIncomingCall()
```

Whether the call is a incoming call

Override:

[isIncomingCall](#) in [Call](#)

isTx

```
public boolean isTx()
```

Return true if the call is in Tx state

Override:

[isTx](#) in [Call](#)

isPttEnable

```
public boolean isPttEnable()
```

Return true if PTT is enabled

Override:

[isPttEnable](#) in [Call](#)

isCallSetupQueuing

```
public boolean isCallSetupQueuing()
```

Return true if the call is in setup queuing status, else return false

Override:

[isCallSetupQueuing](#) in [Call](#)

isRx

```
public boolean isRx()
```

Return true if the call is in Rx status

Override:

[isRx](#) in [Call](#)

isHangTime

`public boolean isHangTime()` *Return true if the call is in hangtime status*

Override:

[isHangTime](#) in [Call](#)

hashCodeString

`public java.lang.String hashCodeString()`

Return hashcode

Override:

[hashCodeString](#) in [Call](#)

7.2.7.3 IDmrTCall

Package: `com.hytera.call.param.dmr`

Extended from [Call](#), used to handledDmrTCall

IDmrTCall Nested class

IDmrTCall.State field

Qualifier and Type	Constant fields	Value
public static final int	IDLE	0
public static final int	CALL_SETUP_REQUESTING	1
public static final int	RANDOM_CONNECT_STATE	2
public static final int	CALL_SETUP_QUEUING	3
public static final int	CALL_RING	4

public static final int	WAITING_ENTER_TRANSACTION_CHANNEL_STATE	5
public static final int	CALL_HANG_TIME_STATE	6
public static final int	PTT_AUTHORIZED_STATE	7
public static final int	VOICE_TX_REQUESTING	8
public static final int	VOICE_TX	9
public static final int	VOICE_RX	10
public static final int	DUPLEX_TALKING	11
public static final int	CALL_FORCE_GET_SPEAKING_PERMISSION	12
public static final int	CALL_HANGUP_REQUEST_STATE	13
public static final int	CALL_FINISHED_STATE	14

IDmrTCall methods

isEmergencyCall

```
public boolean isEmergencyCall()
```

Override:

[isEmergencyCall](#) in [Call](#)

describeContents

```
public int describeContents()
```

Override:

[describeContents](#) in [Call](#)

getNetType

```
public int getNetType()
```

Override:

[getNetType](#) in [Call](#)

isInCall

```
public boolean isInCall()
```

Override:

[isInCall](#) in [Call](#)

isCallEnd

```
public boolean isCallEnd()
```

Override:

[isCallEnd](#) in [Call](#)

hasPendingSetupCall

```
public boolean hasPendingSetupCall()
```

Override:

[hasPendingSetupCall](#) in [Call](#)

toString

```
public java.lang.String toString()
```

Override:

[toString](#) in [Call](#)

getCallRelReasonStr

```
public static final java.lang.String getCallRelReasonStr(  
    Context context, int releaseCause)
```

Get the release reason of call

Parameters:

context - context

releaseCause - the cause code of release, can be get by the method
Call.getCallReleaseReason

getTimerCounter

```
public IDmrTimerCounter getTimerCounter()
```

Get the counting of timer

Reference: [IDmrTimerCounter](#)

isDialing

```
public boolean isDialing()
```

Override:

[isDialing](#) in [Call](#)

isIncomingCall

```
public boolean isIncomingCall()
```

Override:

[isIncomingCall](#) in [Call](#)

isTx

Override:

[isTx](#) in [Call](#)

isRx**Override:**

[isRx](#) in [Call](#)

isPttEnable

```
public boolean isPttEnable()
```

Override:

[isPttEnable](#) in [Call](#)

isCallSetupQueuing

```
public boolean isCallSetupQueuing()
```

Override:

[isCallSetupQueuing](#) in [Call](#)

isHangTime

```
public boolean isHangTime()
```

Override:

[isHangTime](#) in [Call](#)

getCommonReplyStr

```
public static final java.lang.String getCommonReplyStr(  
    Context context, int result)
```

Get the common results of reply

Parameters:*context* - context*result* - the result of reply**getSetupCallReplyStr**

```
public static final java.lang.String getSetupCallReplyStr(
    Context context, int result)
```

*Get the results of reply***Parameters:***context* - context*result* - the result of reply**7.2.7.4 ITetraDCall**

Package: com.hytera.call.param.tetrad

Extended from **Call**, used to handleTetraDCall**ITetraDCall Nested class****ITetraDCall.State field**

Qualifier and Type	Constant fields	Value
public static final int	TX_GRANTED_FAIL	0
public static final int	IDEL	1
public static final int	TX_GRANTED	1
public static final int	CALL_OUTGOING	2
public static final int	VOICE_TX	4

ITetraDCall.TxGrantValue field

Qualifier and Type	Constant fields	Value
public static final int	VOICE_RX	5
public static final int	HANG_TIME	6

ITetraDCall methods**setIsPendingSetupCall**

```
public final void setIsPendingSetupCall(  
                                boolean isPendingSetupCall)
```

Set whether the call is pending for setup

Parameters:

isPendingSetupCall- whether is pending set up call

setModeType

```
public final void setModeType(int modeType)
```

Set the modetype

Parameters:

modeType -the type of mode,

0-TCH/S: TETRA encoded speech

1-7.2 kbit/s unprotected data

2-Low protection 4.8 kbit/s short interleaving depth = 1

3-Low protection 4.8 kbit/s medium interleaving depth = 4

4-Low protection 4.8 kbit/s long interleaving depth = 8

- 5-High protection 2.4 kbit/s short interleaving depth = 1
- 6-High protection 2.4 kbit/s medium interleaving depth = 4
- 7-High protection 2.4 kbit/s high interleaving depth = 8
- 8-TCH/S: 7.2 kbit/s unprotected data

getModeType

```
public final int getModeType()
```

Get the modetype

Returns:

- 0-TCH/S: TETRA encoded speech
- 1-7.2 kbit/s unprotected data
- 2-Low protection 4.8 kbit/s short interleaving depth = 1
- 3-Low protection 4.8 kbit/s medium interleaving depth = 4
- 4-Low protection 4.8 kbit/s long interleaving depth = 8
- 5-High protection 2.4 kbit/s short interleaving depth = 1
- 6-High protection 2.4 kbit/s medium interleaving depth = 4
- 7-High protection 2.4 kbit/s high interleaving depth = 8
- 8-TCH/S: 7.2 kbit/s unprotected data

setCallerAlias

```
public final void setCallerAlias(java.lang.String callerAlias)
```

Set the caller alias

Override:

[setCallerAlias](#) in [Call](#)

callerAlias-the caller's alias

getCallerAlias

```
public final java.lang.String getCallerAlias()
```

Get the caller alias

Override:

[getCallerAlias](#) in [Call](#)

Returns:

caller's alias

setContactAlias

```
public final void setContactAlias(java.lang.String contactAlias)
```

Override:

[setContactAlias](#) in [Call](#)

Parameters:

contactAlias – the contact's alias

getContactAlias

```
public final java.lang.String getContactAlias()
```

Override:

[getContactAlias](#) in [Call](#)

setSpeakerAlias

```
public final void setSpeakerAlias(java.lang.String speakerAlias)
```

Override:

[setSpeakerAlias](#) in [Call](#)

getSpeakerAlias

```
public final java.lang.String getSpeakerAlias()
```

Override:

[getSpeakerAlias](#) in [Call](#)

setGwRpNumber

```
public final void setGwRpNumber(java.lang.String gwrpNum)
```

getGwRpNumber

```
public final java.lang.String getGwRpNumber()
```

getEsnNumber

```
public final java.lang.String getEsnNumber()
```

Get Esn Number

setEsnNumber

```
public final void setEsnNumber(java.lang.String esnNumber)
```

Set Esn Number

isPresenceCheck

```
public final boolean isPresenceCheck()
```

Whether the presence check is required

setIsPresenceCheck

```
public final void setIsPresenceCheck(boolean isPresenceCheck)
```

Set whether the presence check is required

setIsLateEntry

```
public final void setIsLateEntry(boolean isLateEntry)
```

Set whether the call is late entry

Parameters:

isLateEntry- whether is late entry

isLateEntry

```
public boolean isLateEntry()
```

Confirm whether the call is late entry

setCommunicationType

```
public void setCommunicationType(int type)
```

Set communication type

Parameters:

type- type of communication

- 0 Direct MS-MS operation
- 1 Operation with repeater (*)
- 2 Operation with gateway (*)
- 3 Operation with a repeater/gateway (*)

getCommunicationType

```
public int getCommunicationType()
```

Get the communicationtype

Returns:

- 0 Direct MS-MS operation
- 1 Operation with repeater (*)
- 2 Operation with gateway (*)
- 3 Operation with a repeater/gateway (*)

setInterleavingDepth

```
public void setInterleavingDepth(int interleavingDepth)
```

Set InterleavingDepth

Parameters:

interleavingDepth—interleavingDepth

- 0 Interleaving with N = 1 (i.e. short interleaving)
- 1 Interleaving with N = 4 (i.e. medium interleaving)
- 2 Interleaving with N = 8 (i.e. long interleaving)

getInterleavingDepth

```
public int getInterleavingDepth()
```

Get InterleavingDepth

Returns:

- 0 Interleaving with N = 1 (i.e. short interleaving)
- 1 Interleaving with N = 4 (i.e. medium interleaving)
- 2 Interleaving with N = 8 (i.e. long interleaving)

setTPNIrequire

```
public void setTPNIrequire(int tpnIrequire)
```

Set whether the TPNI is required

Parameters:

tpnIrequire-tpnIrequire

- 0 TPNI required
- 1 TPNI shall be suppressed
- 2 TPNI may be provided

getTPNIrequire

```
public int getTPNIrequire()
```

Get whether the TPNI is required

Returns:

- 0 TPNI required
- 1 TPNI shall be suppressed
- 2 TPNI may be provided

setGwmcc

```
public void setGwmcc(int gwmcc)
```

Set Gwmcc

Parameters:

gwmcc – 0-1023, country code, signify network is not selected if gwmcc equals 0

getGwmcc

```
public int getGwmcc()
```

Get Gwmcc

Returns:

0-1023, country code, signify network is not selected if result equals 0

setGwMnc

```
public void setGwMnc(int gwMnc)
```

Set GwMnc

Parameters:

gwMnc - 0-16383, network code, signify network is not
selected if gwMnc equals 0

getGwMnc

```
public int getGwMnc()
```

Get GwMnc

Returns:

0-16383, network code, signify network is not selected if
gwmnc equals 0

setGw

```
public void setGw(int gw)
```

Set Gw

getGw

```
public int getGw()
```

Get Gw

setArea

```
public void setArea(int area)
```

Set area

Parameters:

area-

- 0 Area selection not defined
- 1-14 Area number
- 15 All areas

getArea

```
public int getArea()
```

Get area

Returns:

- 0 Area selection not defined
- 1-14 Area number
- 15 All areas

getContactTetraCallID

```
public TetraCallID getContactTetraCallID()
```

Get the contact TetraCallID

Reference: [TetraCallID](#)

getCallerTetraCallID

```
public TetraCallID getCallerTetraCallID()
```

Get the TetraCallID of the caller

Reference: [TetraCallID](#)

getSpeakerTetraCallID

```
public TetraCallID getSpeakerTetraCallID()
```

Get the TetraCallID of the speaker

Reference:[TetraCallID](#)

setContactTetraCallID

```
public void setContactTetraCallID(TetraCallID id)
```

Set the TetraCallID of the contact speaker

Parameters:

id-TetraCallID

Reference:[TetraCallID](#)

setCallerTetraCallID

```
public void setCallerTetraCallID(TetraCallID id)
```

Set the TetraCallID of the contact caller

Parameters:

id-TetraCallID

Reference:[TetraCallID](#)

setSpeakerTetraCallID

```
public void setSpeakerTetraCallID(TetraCallID id)
```

Set the TetraCallID of the contact speaker

Parameters:

id-TetraCallID

Reference:[TetraCallID](#)

getNetType

```
public int getNetType()
```

Override:

[getNetType](#) in [Call](#)

isAllCall

```
public boolean isAllCall()
```

Whether the call is all call type

isInCall

```
public boolean isInCall()
```

Override:

[isInCall](#) in [Call](#)

isCallEnd

```
public boolean isCallEnd()
```

Override:

[isCallEnd](#) in [Call](#)

hasPendingSetupCall

```
public boolean hasPendingSetupCall()
```

Override:

[hasPendingSetupCall](#) in [Call](#)

isAIEEncryption

```
public boolean isAIEEncryption()
```

Whether the call is AIE encrypted

setAIEEncryption

```
public void setAIEEncryption(boolean isAIEEncryption)
```

Set whether the call is AIE encrypted

toString

```
public java.lang.String toString()
```

isDialing

```
public boolean isDialing()
```

Override:

[isDialing](#) in [Call](#)

isIncomingCall

```
public boolean isIncomingCall()
```

Whether the call is a incoming call

Override:

[isIncomingCall](#) in [Call](#)

getDisplayedContactName

```
public java.lang.String getDisplayedContactName()
```

Override:

[getDisplayedContactName](#) in [Call](#)

getDisplayedSpeakerName

```
public java.lang.String getDisplayedSpeakerName()
```

Override:

[getDisplayedSpeakerName](#) in [Call](#)

isEmergencyCall

```
public boolean isEmergencyCall()
```

Override:

[isEmergencyCall](#) in [Call](#)

isTx

```
public boolean isTx()
```

Override:

[isTx](#) in [Call](#)

isRx

```
public boolean isRx()
```

Override:

[isRx](#) in [Call](#)

isPttEnable

```
public boolean isPttEnable()
```

Override:

[isPttEnable](#) in [Call](#)

isHangTime

```
public boolean isHangTime()
```


Override:

[isHangTime](#) in [Call](#)

getCallRelReasonStr

```
public static final java.lang.String getCallRelReasonStr(  
    Context context, int releaseCause)
```

Get the release reason of call

Parameters:

context - context

releaseCause - the cause code of release, can be get by the method
`Call.getCallReleaseReason`

getCommonReplyStr

```
public static final java.lang.String getCommonReplyStr(  
    Context context, int result)
```

Get the common results of reply

Parameters:

context - context

result - the result of reply

isCallSetupQueuing

```
public boolean isCallSetupQueuing()
```

Override:

[isCallSetupQueuing](#) in [Call](#)

setTxGranted

```
publicfinal void setTxGranted(int txGranted)
```

Set state of Tx granted

Parameters:

txGranted - state of tx granted

Reference: [ITetraDCall.TxGrantValue](#)

getTxGranted

```
publicfinal int getTxGranted ()
```

Return the status of Tx granted

Reference: [ITetraDCall.TxGrantValue](#)

7.2.7.5 ITetraTCall

Package: com.hytera.call.param.tetrat

Extended from [Call](#), used to handleTetraTcall

ITetraTCall Nested class**ITetraTCall.State field**

Qualifier and Type	Constant fields	Value
public static final int	IDLE	0x00
public static final int	PENDING_CALL	0x01
public static final int	CALL_OUTGOING	0x02
public static final int	CALL_INCOMING	0x03
public static final int	VOICE_TX	0x04

public static final int	VOICE_RX	0x05
public static final int	HANG_TIME	0x06
public static final int	FULL_DUPLEX_TALKING	0x07
public static final int	TX_QUEUING	0x08
public static final int	TX_WAIT	0x09

ITetraTCall Nested interface

ITetraTCall.CallInStatus field

Qualifier and Type	Constant fields	Value
public final int	CALL_PROGRESSING	0
public final int	CALL_QUEUED	1
public final int	CALLED_PARTY_PAGED	2
public final int	CALL_CONTINUE	3
public final int	HANG_TIME_EXPIRED	4

ITetraTCall.TxGrantValue field

Qualifier and Type	Constant fields	Value
public static final int	TX_GRANTED	0
public static final int	TX_NOT_GRANTED	1
public static final int	TX_QUEUING	2

public static final int	TX_GRANTED_TO_OTHER	3
public static final int	TX_WAIT	4
public static final int	TX_CONTINUE	5
public static final int	TX_NOT_CONTINUE	6
public static final int	TX_CEASED	7

ITetraTCall methods

setCCInstace

```
public final void setCCInstace(int ccInstace)
```

Set CCinstance

Parameters:

ccInstace - 0-3,default=0

getCCInstace

```
public final int getCCInstace()
```

Get CCinstance

Returns:

0-3,default=0

setAIService

```
public final void setAIService(int aiService)
```

Set ai service

Parameters:

aiService -

- 0 TETRA speech
- 1 7.2 kbit/s unprotected data
- 2 Low protection 4.8 kbit/s short interleaving depth = 1
- 3 Low protection 4.8 kbit/s medium interleaving depth = 4
- 4 Low protection 4.8 kbit/s long interleaving depth = 8
- 5 High protection 2.4 kbit/s short interleaving depth = 1
- 6 High protection 2.4 kbit/s medium interleaving depth = 4
- 7 High protection 2.4 kbit/s high interleaving depth = 8

getAIService

```
public final int getAIService()
```

Get ai service

Returns:

- 0 TETRA speech
- 1 7.2 kbit/s unprotected data
- 2 Low protection 4.8 kbit/s short interleaving depth = 1
- 3 Low protection 4.8 kbit/s medium interleaving depth = 4
- 4 Low protection 4.8 kbit/s long interleaving depth = 8
- 5 High protection 2.4 kbit/s short interleaving depth = 1
- 6 High protection 2.4 kbit/s medium interleaving depth = 4
- 7 High protection 2.4 kbit/s high interleaving depth = 8

setSlots

```
public final void setSlots(int slots)
```

Set slots

Parameters:

slots -

- if AIService = 0
- slots:
 - 1- TETRA encoded speech
 - 2- reserved
 - 3- reserved
 - 4- Proprietary encoded speech
- if AIService != 0
- slots:
 - 0- 1 slot
 - 1- 2 slots
 - 2- 3 slots
 - 3- 4 slots

getSlots

```
public final int getSlots()
```

Get slots

Returns:

- if AIService = 0
- slots:
 - 1- TETRA encoded speech
 - 2- reserved
 - 3- reserved
 - 4- Proprietary encoded speech

```
if AIService != 0  
  
slots:  
  
0- 1 slot  
  
1- 2 slots  
  
2- 3 slots  
  
3- 4 slots
```

setTxGranted

```
public final void setTxGranted(int txGranted)
```

Set TxGrant status

Parameters:

txGranted -

- 0 Tx granted
- 1Tx not granted
- 2 Queuing for Tx
- 3 Tx granted to another
- 4 Tx wait
- 5 Tx continue
- 6 Tx not continue
- 7 Tx cease

getTxGranted

```
public final int getTxGranted()
```

Get TxGrant status

Returns:

- 0 Tx granted
- 1Tx not granted

- 2 Queuing for Tx
- 3 Tx granted to another
- 4 Tx wait
- 5 Tx continue
- 6 Tx not continue
- 7 Tx cease

setTxReqPermission

```
public final void setTxReqPermission(int txReqPermission)
```

Set TxReqPermission status

Parameters:

txReqPermission -

0 Allowed to request

1 Not allowed to request

getTxReqPermission

```
public final int getTxReqPermission()
```

Get TxReqPermission status

Returns:

0 Allowed to request

1 Not allowed to request

setArea

```
public final void setArea(int area)
```

Set area

Parameters:

area-

0 Area not defined

1-14 Area 1-14

15 All areas

getArea

```
public final int getArea()
```

Get area

Returns:

0 Area not defined

1-14 Area 1-14

15 All areas

setCLIR

```
public final void setCLIR(int clir)
```

Set CLIR

Parameters:

clir-

0 not implement or default

1 Reserved

2 Presentation not restricted

3 Presentation restricted

getCLIR

```
public final int getCLIR()
```

Get CLIR

Returns:

- 0 not implement or default
- 1 Reserved
- 2 Presentation not restricted
- 3 Presentation restricted

setCallOutStatus

```
public final void setCallOutStatus(int callOutStatus)
```

Set CallOutStatus

Parameters:

callOutStatus -

- 1 0 Call progressing
- 2 Call queued
- 3 Called party paged
- 4 Call continue
- 5 Hang time expired

getCallOutStatus

```
public final int getCallOutStatus()
```

Get CallOutStatus

Returns:

- 0 Call progressing
- 1 Call queued
- 2 Called party paged
- 3 Call continue
- 4 Hang time expired

setCallInStatus

```
public final void setCallInStatus(int callInStatus)
```

Set CallInStatus

Parameters:

callInStatus -

- 0 Call progressing
- 1 Call queued
- 2 Called party paged
- 3 Call continue
- 4Hang time expired

getCallInStatus

```
public final int getCallInStatus()
```

Get CallInStatus

Returns:

- 0 Call progressing
- 1 Call queued
- 2 Called party paged
- 3 Call continue
- 4Hang time expired

setIsReqTx

```
public final void setIsReqTx(boolean isReqTx)
```

Set whether to request Tx

getIsReqTx

```
public final boolean getIsReqTx()
```

Get whether to requestTx

setIsAIEEncryption

```
public final void setIsAIEEncryption(boolean isAIEEncryption)
```

Set whether the call is is AIE encrypted

isAIEEncryption

```
public final boolean isAIEEncryption()
```

Whether the call is AIE encrypted

setIsSilentMode

```
public final void setIsSilentMode(boolean isSilentMode)
```

Set whether the mode type is silent mode

isSilentMode

```
public final boolean isSilentMode()
```

Get whether the mode type is silent mode

getContactTetraCallID

```
public final TetraCallID getContactTetraCallID()
```

Get ContactTetraCallID

Reference: [TetraCallID](#)

getCallerTetraCallID

```
public final TetraCallID getCallerTetraCallID()
```

Get CallerTetraCallID

Reference: [TetraCallID](#)

getSpeakerTetraCallID

```
public final TetraCallID getSpeakerTetraCallID()
```

Get SpeakerTetraCallID

Reference: [TetraCallID](#)

setContactTetraCallID

```
public final void setContactTetraCallID(TetraCallID id)
```

Set ContactTetraCallID

Parameters:

id-TetraCallID

Reference: [TetraCallID](#)

setCallerTetraCallID

```
public final void setCallerTetraCallID(TetraCallID id)
```

Set CallerTetraCallID

Parameters:

id-TetraCallID

Reference: [TetraCallID](#)

setSpeakerTetraCallID

```
public final void setSpeakerTetraCallID(TetraCallID id)
```

Set SpeakerTetraCallID

Parameters:

id—TetraCallID

Reference: [TetraCallID](#)

getNetType

```
public final int getNetType()
```

Override:

[getNetType](#) in [Call](#)

isAllCall

```
public boolean isAllCall()
```

Whether the call type is all call

isInCall

```
public final boolean isInCall()
```

Override:

[isInCall](#) in [Call](#)

isCallEnd

```
public final boolean isCallEnd()
```

Override:

[isCallEnd](#) in [Call](#)

hasPendingSetupCall

```
public final boolean hasPendingSetupCall()
```

Override:

[hasPendingSetupCall](#) in [Call](#)

setIsPendingSetupCall

```
public void setIsPendingSetupCall(boolean isPendingSetupCall)
```

Set whether the call is pending for setup

Parameters:

isPendingSetupCall—whether is pending set up call

clone

```
public java.lang.Object clone()
```

Override:

[clone](#) in [Call](#)

toString

```
public final java.lang.String toString()
```

Override:

[toString](#) in [Call](#)

tinyString

```
public final java.lang.String tinyString()
```

getCommonReplyStr

```
public static final java.lang.String getCommonReplyStr(  
    Context context, int result)
```

Get the common results of reply

Parameters:

context - context

result - the result of reply

getCallRelReason

```
public static final java.lang.String getCallRelReason(  
    Context context,int releaseCause)
```

Get reason of call release

Parameters:

context - context

releaseCause - the second parameter of method
`ItetraTListener.onCallRelease`

isDialing

```
public final boolean isDialing()
```

Override:

[isDialing](#) in [Call](#)

isIncomingCall

```
public boolean isIncomingCall()
```

Override:

[isIncomingCall](#) in [Call](#)

isEmergencyCall

```
public boolean isEmergencyCall()
```

Override:

[isEmergencyCall](#) in [Call](#)

setIsEmergencyCall

```
public void setIsEmergencyCall(boolean isEmergencyCall)
```

Set whether the call is emergencyCall

Parameters:

isEmergencyCall – whether is EmergencyCall

isTx

```
public boolean isTx()
```

Override:

[isTx](#) in [Call](#)

isRx

```
public boolean isRx()
```

Override:

[isRx](#) in [Call](#)

isTxQueuing

```
public boolean isTxQueuing()
```

Whether the call is in Tx queuing status

isPttEnable

```
public boolean isPttEnable()
```

Override:

[isPttEnable](#) in [Call](#)

isHangTime

```
public boolean isHangTime()
```

Override:

[isHangTime](#) in [Call](#)

isCallSetupQueuing

```
public boolean isCallSetupQueuing()
```

Override:

[isCallSetupQueuing](#) in [Call](#)

setWaitingForAnswer

```
public final void setWaitingForAnswer(boolean waitingForAnswer)
```

Set whether the call is waiting for answering

Parameters:

waitingForAnswer– whether wait for answer

7.2.7.6 IPhoneCall

Package name: com.hytera.call.param. phone

Extended from **Call**, used to handle PhoneCall

IPhoneCall Constant Fields

Qualifier and Type	Constant fields	value
public static final int	UNEMERGENCY	0
public static final int	EMERGENCY	1

IPhoneCall Nested class

IPhoneCall.State field

Qualifier and Type	Constant fields	Value
public static final int	INVALID	0
public static final int	NEW	1
public static final int	IDLE	2
public static final int	ACTIVE	3
public static final int	INCOMING	4
public static final int	CALL_WAITING	5
public static final int	DIALING	6
public static final int	REDIALING	7

public static final int	ONHOLD	8
public static final int	DISCONNECTING	9
public static final int	DISCONNECTED	10
public static final int	CONFERENCED	11
public static final int	CONNECTING	13
public static final int	SELECT_PHONE_ACCOUNT	12

IPhoneCall methods

setIsEmergencyCall

```
public void setIsEmergencyCall(boolean isEmergencyCall)
```

Set whether the call is emergency call

isEmergencyCall

```
public boolean isEmergencyCall()
```

Override:

[isEmergencyCall](#) in [Call](#)

describeContents

```
public int describeContents()
```

Override:

[describeContents](#) in [Call](#)

getCallType

```
public int getCallType()
```

Override:

[getCallType](#) in [Call](#)

Returns:

callType-individual call, group call and so on.

Reference:[CallType](#)

getNetType

```
public int getNetType()
```

Override:

[getNetType](#) in [Call](#)

isInCall

```
public boolean isInCall()
```

Override:

[isInCall](#) in [Call](#)

isCallEnd

```
public boolean isCallEnd()
```

Override:

[isCallEnd](#) in [Call](#)

getCallMode

```
public int getCallMode()
```

Override:

[getCallMode](#) in [Call](#)

toString

```
public java.lang.String toString()
```

Override:

[toString](#) in [Call](#)

isDialing

```
public boolean isDialing()
```

Override:

[isDialing](#) in [Call](#)

getDisconnectCause

```
public java.lang.CharSequence getDisconnectCause()
```

Returns call disconnection cause, defined by DisconnectCause.

setDisconnectCause

```
public void setDisconnectCause(  
    java.lang.CharSequence disconnectCause)
```

Parameters:

disconnectCause - cause of disconnect

setErrorCode

```
public void setErrorCode(int code)
```

Set error codes

getErrorCode

```
public int getErrorCode()
```

Get error codes

isIncomingCall

```
public boolean isIncomingCall()
```

Override:

[isIncomingCall](#) in [Call](#)

isComming

```
public boolean isComming()
```

Whether the call is a incoming call

setIsComming

```
public void setIsComming(boolean comming)
```

Set whether the call is a incomming

isHook

```
public boolean isHook()
```

Override:

[isHook](#) in [Call](#)

7.2.7.7 IRoipCall

Package name: com.hytera.call.param.roip

Extended from **Call**, used to handle RoipCall

IRoipCall Nested class

IRoipCall.CallResult

Constant fields for IRoipCall.CallResult

Qualifier and Type	Constant fields	Value
public static final java.lang.String	ALREADY_DISCONNECTED_STR	"call already disconnected"
public static final java.lang.String	ALREADY_REQUESTED_TX_STR	"already requested Tx permission"
public static final java.lang.String	EXISTING_CALL_STR	"already in call with higher priority"
public static final java.lang.String	DUPLEX_CALL_STR	"noTxDemand/TxCease in full duplex call"
public static final java.lang.String	NO_CONNECTION_STR	"no network connection"
public static final java.lang.String	NO_TX_PERMISSION_STR	"no Tx permission to cease"
public static final java.lang.String	NOT_ACTIVE_STR	"call is not active"
public static final java.lang.String	OTHER_PARTY_TALKING_STR	"other party is talking"
public static final java.lang.String	SERVICE_CLOSED_STR	"service closed"
public static final java.lang.String	STILL_CLOSING_STR	"service is still closing"

public static final java.lang.String	SUCCESS_STR	"operation succeeded"
public static final java.lang.String	TIMER_EXPIRED_STR	"timer expired"
public static final java.lang.String	TX_REJECTED_STR	"TxDemand was rejected"
public static final java.lang.String	UNKNOWN_ERROR_STR	"unknown error"
public static final int	SUCCESS	0
public static final int	UNKNOWN_ERROR	1
public static final int	TIMEOUT_ERROR	2
public static final int	WRONG_STATE_ERROR	3
public static final int	EXISTING_CALL_ERROR	4
public static final int	DUPLEX_CALL_ERROR	5
public static final int	TX_REJECTED_ERROR	6

IRoipCall.State

Constant fields forIRoipCall.State

Qualifier and Type	Constant fields	Value
public static final int	IDLE	0
public static final int	DISCONNECTED	1
public static final int	I_SETUP	2
public static final int	O_SETUP	3

public static final int	ACTIVE	4
public static final int	ON_HOLD	5
public static final int	TRANSFER	6
public static final int	RINGING	7
public static final int	CONNECTING	8
public static final int	OUTGOING	9

IRoipCall.State methods

isConnectingOrConnected

```
public static boolean isConnectingOrConnected(int state)
```

Whether the call is in connecting or connected stause

Parameters:

`state`—IRoipCall.State

Reference: [IRoipCall.State](#)

isDialing

```
public static boolean isDialing(int state)
```

Whether the call is indialing status

Parameters:

`state`—IRoipCall.State

Reference: [IRoipCall.State](#)

IRoipCall Constructor

IRoipCall

```
public IRoipCall(java.lang.String myNum)
```

IRoipCall Methods

isCallAnswered

```
public boolean isCallAnswered()
```

Whether the call is answered

setCallAnswered

```
public void setCallAnswered(boolean callAnswered)
```

Set call answered status

setEncryptionType

```
public void setEncryptionType(ENCRYPTION_TYPE encryptionType)
```

set Encryption Type

[Reference:ENCRYPTION_TYPE](#)

getEncryptionType

```
public ENCRYPTION_TYPE getEncryptionType()
```

get EncryptionType

[Reference:ENCRYPTION_TYPE](#)

setEncodingType

```
public void setEncodingType(ENCRYPTION_TYPE encodingType)
```

set EncodingType

Reference:[ENCRYPTION_TYPE](#)

getEncodingType

```
public ENCRYPTION_TYPE getEncodingType()
```

get EncodingType

Reference:[ENCRYPTION_TYPE](#)

setEmergency

```
public void setEmergency(boolean isEmergency)
```

Set whether is emergency

isEmergency

```
public boolean isEmergency()
```

Whether is emergency

isInCall

```
public boolean isInCall()
```

Override:

[isInCall](#) in [Call](#)

hasHangup

```
public boolean hasHangup()
```

Whether the call has hanged up

setHold

```
public void setHold(boolean isHold)
```

Set the call hold status

isHold

```
public boolean isHold()
```

Whether the call is in hold status

toString

```
public java.lang.String toString()
```

Override:

[toString](#) in [Call](#)

describeContents

```
public int describeContents()
```

Override:

[describeContents](#) in [Call](#)

getMyNum

```
public java.lang.String getMyNum()
```

Get My Num

getNetType

```
public int getNetType()
```

Override:

[getNetType](#) in [Call](#)

isCallEnd

```
public boolean isCallEnd()
```

Override:

[isCallEnd](#) in [Call](#)

hasPendingSetupCall

```
public boolean hasPendingSetupCall()
```

Override:

[hasPendingSetupCall](#) in [Call](#)

isDialing

```
public boolean isDialing()
```

Override:

[isDialing](#) in [Call](#)

isIncomingCall

```
public boolean isIncomingCall()
```

Override:

[isIncomingCall](#) in [Call](#)

isEmergencyCall

```
public boolean isEmergencyCall()
```

Override:

[isEmergencyCall](#) in [Call](#)

setAudioFlowOrientation

```
public void setAudioFlowOrientation(int orientation)
```

Set the orientation of audio flow

Parameters:

orientation -

-1 no receiving and transmitting

0 transmitting

1 receiving

2 receiving and transmitting

isTx

```
public boolean isTx()
```

Override:

[isTx](#) in [Call](#)

isRx

```
public boolean isRx()
```

Override:

[isRx](#) in [Call](#)

isPttEnable

```
public boolean isPttEnable()
```

Override:

[isPttEnable](#) in [Call](#)

isHangTime

```
public boolean isHangTime()
```

Override:

[isHangTime](#) in [Call](#)

isCallSetupQueuing

```
public boolean isCallSetupQueuing()
```

Override:

[isCallSetupQueuing](#) in [Call](#)

isTxQueuing

```
public boolean isTxQueuing()
```

isAllCall

```
public boolean isAllCall()
```

isE2EE


```
public boolean isE2EE()
```

7.2.8 Data

7.2.8.1 OutgoingCallParam

Package: com.hytera.call.param

Outgoing call parameters

OutgoingCallParamConstantfields

Constant fields for OutgoingCallParam

Qualifier and Type	Constant fields	Value
public static final int	PARAM_IS_VALID	0
public static final int	PARAM_NUM_EMPTY	-1
public static final int	PARAM_INVALID_CONTACT_ID	-1
public static final int	PARAM_INVALID_CALL_TYPE	-1
public static final int	PARAM_NETTYP_ERR	-2
public static final int	PARAM_CALL_TYPE_ERR	-3

OutgoingCallParam Constructor

OutgoingCallParam

```
public OutgoingCallParam ()
```

OutgoingCallParam

```
public OutgoingCallParam(Intent intent)
```

OutgoingCallParam

```
public OutgoingCallParam(Call call)
```

OutgoingCallParam Methods

getNetType

```
public int getNetType()
```

Get call network type

Reference: [NetType](#)

setNetType

```
public void setNetType(int netType)
```

Set call network type

Parameters:

netType– net type

Reference: [NetType](#)

getContactNumber

```
public java.lang.String getContactNumber()
```

Get contact number

setContactNumber

```
public void setContactNumber(java.lang.String number)
```

Set contact number

Parameters:

number– contact number

getCallType

```
public int getCallType()
```

Get call type

Reference:[CallType](#)

setCallType

```
public void setCallType(int callType)
```

Set call type

Parameters:

callType– call type

Reference:[CallType](#)

isE2EE

```
public boolean isE2EE()
```

Whether the encryption type is E2EE

setE2EE

```
public void setE2EE(boolean e2ee)
```

Set whether the encryption type is E2EE

Parameters:

e2ee– whether is e2ee encryption

isHook

```
public boolean isHook()
```

Whether the call is hook call

setIsHook

```
public void setIsHook(boolean isHook)
```

Set whether the call is hook call

Parameters:

isHook- whether is hook call

isReqTx

```
public boolean isReqTx()
```

Return true if the call is requesting Tx

setIsReqTx

```
public void setIsReqTx(boolean isReqTx)
```

Set whether to request call Tx

Parameters:

isReqTx- whether request tx for call

getCallMode

```
public int getCallMode()
```

Get call mode

Reference:[CallMode](#)

setCallMode

```
public void setCallMode(int callMode)
```

Set call mode

Parameters:

callMode– call mode

Reference:[CallMode](#)

getCallPriority

```
public int getCallPriority()
```

Get call priority

Reference:[CallPriority](#)

setCallPriority

```
public void setCallPriority(int callPriority)
```

Set call priority

Parameters:

callPriority– call priority

Reference:[CallPriority](#)

getContactID

```
public int getContactID()
```

Get contact id

setContactID

```
public void setContactID(int contactID)
```

Set contact id

Parameters:

contacted-contactID

isAutoCompleteFullNum

```
public boolean isAutoCompleteFullNum ()
```

Return true if the contact ID is auto-completed full number

setIsAutoCompleteFullNum

```
public void setIsAutoCompleteFullNum (boolean autoComplete)
```

Set whether the contact ID is auto-completed full number

Parameters:

autoComplete- whether auto complete full num

ismForceBringUpActivity

```
public boolean ismForceBringUpActivity()
```

Whether to be forcedbringing up activity

setmForceBringUpActivity

```
public boolean setmForceBringUpActivity(  
                                     boolean mForceBringUpActivity)
```

Set whether to be forced bringing up activity

describeContents

```
public int describeContents ()
```

Get description contents

toString

```
public String toString ()
```

7.2.8.2 ICallRecordTimeInfo

Package name: com.hytera.call.param.callrecord

Contains timeinfo of call record

ICallRecordTimeInfo constructor

```
public ICallRecordTimeInfo(String callId, int state, long recordingTime)
```

java.lang.String	callId callid
int	state record state
int	recordingTime recording time

7.2.8.3 IAudioModeData

Package name: com.hytera.call.param.dsp

Contain info of audiomode

IAudioModeData constructor

```
public IAudioModeData(String callId, int[] modelist, int mode)
```

java.lang.String	callId callid
int[]	modelist available audio mode list
int	mode current audio mode

7.2.8.4 TetraCallID

Package name: com.hytera.call.param.tetrad

Indicate Tetra Call's ID

TetraCallID Constant fields

Qualifier and Type	Constant fields	Value
public static final int	ID_TYPE_SSI	0
public static final int	ID_TYPE_MCC_MNC_SSI	1
public static final int	ID_TYPE_SNA	2
public static final int	ID_TYPE_ESN	3
public static final int	ID_TYPE_ESN1	4

TetraCallID Methods

setCallIdType

```
public final void setCallIdType(int type)
```

Set call ID type

Parameters:

type-id type

getCallIdType

```
public final int getCallIdType()
```

Get call ID type

setMcc

```
public final void setMcc(int mcc)
```

Set MCC

Parameters:

mcc-Mcc

getMcc

```
public final int getMcc()
```

Get MCC

setMnc

```
public final void setMnc(int mnc)
```

Set MNC

Parameters:

mnc - Mnc

getMnc

```
public final int getMnc()
```

Get MNC

setSsi

```
public final void setSsi(int ssi)
```

Set Ssi

Parameters:

ssi - Ssi

getSsi

```
public final int getSsi()
```

Get Ssi

setSna

```
public final void setSna(int sna)
```

Get Sna

Parameters:

sna - Sna

getSna

```
public final int getSna()
```

Get Sna

setEsn

```
public final void setEsn(java.lang.String esn)
```

Set Esn

Parameters:

esn-Esn

getEsn

```
public final java.lang.String getEsn()
```

Get Esn

describeContents

```
public int describeContents()
```

Get description contents

writeToParcel

```
public void writeToParcel(Parcel dest, int flags)
```

Parameters:

dest-Parcel

flags-not used

getDisplayNumber

```
public java.lang.String getDisplayNumber()
```

Get display number

clone

```
public java.lang.Object clone()
```

Override:

[clone](#) in [Call](#)

toString

```
public java.lang.String toString()
```

7.2.8.5 IDmrTimerCounter

Package: com.hytera.call.param.dmr

Includetime data of DmrT

IDmrTimerCounterConstant fields

Constant fields for IDmrTimerCounter

Qualifier and Type	Constant fields	Value
public static final int	COUNT_DOWN_TIME	0x00
public static final int	POSITIVE_TIME	0x01
public static final int	CANCEL_TIMER	0x02

IDmrTimerCounterConstructor

IDmrTimerCounter

```
public IDmrTimerCounter ()
```

IDmrTimerCounterMethods

resetTimer

```
public void resetTimer()
```

Reset timer

setTimerFlag

```
public void setTimerFlag(int timerFlag)
```

Set timer flag

Parameters:

timerFlag-

0x00 - count down time

0x01 - positive time

0x02 - cancel timer

getTimerFlag

```
public int getTimerFlag()
```

Get timer flag

Returns:

0x00 - count down time

0x01 - positive time

0x02 - cancel timer

setTimerStartTime

```
public void setTimerStartTime()
```

Set the start time of timer

getTimerStartTime

```
public long getTimerStartTime()
```

Get the start time in seconds of timer.SystemClock.elapsedRealtime.

setTotalCallTime

```
public void setTotalCallTime(int totalCallTime)
```

Set total time

getTotalCallTime

```
public int getTotalCallTime()
```

Get total time

Returns:

total time - the remaining time or the elapsed time

7.2.8.6 IRoipMsgData

Package name: com.hytera.call.param.roip

Contain info of Roip Msg

long	mId
	id
int	mCallType

call type	
DATA_TYPE	mDataType data type
boolean	mIsEncrypted whether is Encrypted
java.lang.String	mCalled sms receiver
java.lang.String	mCaller sms sender
java.lang.String	mData data

7.2.8.7 IRoipMsgStatus

Package name: com.hytera.call.param.roip

Contain info of Roip Tetra Msg Status

long	mId id
int	mCallType call type
java.lang.String	mCalled sms receiver
java.lang.String	mCaller sms sender
SDS_STATUS_CODE	mStatusCode

S	status code
int	mSdsStatusValue

7.2.8.8 SDS_DataSequenceType

Package Name:com.hytera.call.param.roip

AddressType	calledParty
	called party
AddressType	callingParty
	calling party
Encrypt_SDS_DataType	encrypt_SDS_DataType
pe	
Integer16	sdsId
	sds Id

SDS_DataSequenceType Methods

toString

public java.lang.String toString ()

getSdsId

public Integer16 getSdsId()

getthe sdsId

Reference: Integer16

setSdsId

```
public void setSdsId(Integer16 sdsId)
```

set the sdsId

Reference: Integer16

getCalledParty

```
public AddressType getCalledParty()
```

Return the calledParty

Reference: AddressType

setCalledParty

```
public void setCalledParty(AddressType callingParty)
```

set the calledParty

Reference: AddressType

getCallingParty

```
public AddressType getCallingParty()
```

get the callingParty

Reference: AddressType

setCallingParty

```
public void setCallingParty(AddressType callingParty)
```

set the callingParty

Reference: [AddressType](#)

getEncrypt_SDS_DataType

public Encrypt_SDS_DataType getEncrypt_SDS_DataType()

getthe encrypt_SDS_DataType

Reference: [Encrypt_SDS_DataType](#)

setEncrypt_SDS_DataType

public void setEncrypt_SDS_DataType(Encrypt_SDS_DataType encrypt_SDS_DataType)

setthe encrypt_SDS_DataType

Reference: [Encrypt_SDS_DataType](#)

7.2.8.9 AddressType

Package Name:com.hytera.call.param.roip

TETRA_Address	tetraAddress
	tetra address

ISDN_Address	isdnAddress
	isdn address

AddressType Methods

getTetraAddress

public TETRA_Address getTetraAddress()

Return the tetraAddress

Reference: [TETRA_Address](#)

setTetraAddress

```
public void setTetraAddress(TETRA_Address tetraAddress)
```

setthe tetraAddress

Reference: [TETRA_Address](#)

getIsdnAddress

```
public ISDN_Address getIsdnAddress()
```

getthe isdnAddress

Reference: [ISDN_Address](#)

setIsdnAddress

```
public void setIsdnAddress(ISDN_Address isdnAddress)
```

setthe isdnAddress

Reference: [ISDN_Address](#)

7.2.8.10 TETRA_Address

Package Name:com.hytera.call.param.roip

SSIType	ssiType ssi type
MNCType	mncType mnc type
MCCType	mccType

mcc type

TETRA_Address Methods

getSsiType

```
public SSiType getSsiType()
```

Return the ssiType

Reference: [SSiType](#)

setSsiType

```
public void setSsiType(SSiType ssiType)
```

setthe ssiType

Reference: [SSiType](#)

getMncType

```
public MNCType getMncType()
```

Return the mncType

Reference: [MNCType](#)

setMncType

```
public void setMncType(MNCType mncType)
```

setthe mncType

Reference: [MNCType](#)

getMccType

```
public MCCType getMccType()
```

Return the mccType

Reference: [MCCType](#)

setMccType

```
public void setMccType(MCCType mccType)
```

setthe mccType

Reference: [MCCType](#)

7.2.8.11 ISDN_Address

Package Name:com.hytera.call.param.roip

DialingString	dialingString
	dialing string

SSIType	ssiType
	ssi type

ISDN_Address Methods**getSsiType**

```
public SSIType getSsiType()
```

Return the ssiType

Reference: [SSIType](#)

setSsi

```
public void setSsiType(SSIType ssiType)
```

setthe ssiType

Reference: **SSIType**

7.2.8.12 SSIType

Package Name:com.hytera.call.param.roip

Integer	ssi
	ssi

SSIType Methods

getSsi

```
public Integer getSsi()
```

Return the ssi

setSsi

```
public void setSsi(Integer ssi)
```

setthe ssi

7.2.8.13 MNCType

Package Name:com.hytera.call.param.roip

Integer	mnc
	mnc

MNCType Methods

getMnc

```
public Integer getMnc()
```

Return the mnc

setMnc

```
public void setMncType(Integer mnc)
```

set the mnc

7.2.8.14 MCCType

Package Name:com.hytera.call.param.roip

Integer	mcc
	mcc

MCCType Methods

getMcc

```
public Integer getMcc()
```

Return the mcc

setMcc

```
public void setMcc(Integer mcc)
```

setthe mcc

7.2.8.15 DialingString

Package Name:com.hytera.call.param.roip

java.lang.String	dialingString
	dialingString

DialingString Methods

getDialingString

```
public String getDialingString()
```

Return the dialingString

setDialingString

```
public void setDialingString(String dialingString)
```

setthe dialingString

7.2.8.16 Encrypt_SDS_DataType

Package Name:com.hytera.call.param.roip

Integer	statusType
	dstatus type
byte[]	sdsType1
byte[]	sdsType2


```
byte[]          sdsType3

staticSdsType4Sequen sdsType4SequenceType
ceType
```

Encrypt_SDS_DataType Methods

getStatusType

```
public Integer getStatusType()
```

Return the statusType

setStatusType

```
public void setStatusType(Integer statusType)
```

setthe statusType

getSdsType1

```
public byte[] getSdsType1()
```

Return the sdsType1

setSdsType1

```
public void setSdsType1(byte[] sdsType1)
```

setthe sdsType1

getSdsType2

```
public byte[] getSdsType2()
```

Return the sdsType2

setSdsType2

```
public void setSdsType2(byte[] sdsType2)
```

setthe sdsType2

getSdsType3

```
public byte[] getSdsType3()
```

Return the sdsType3

setSdsType3

```
public void setSdsType3(byte[] sdsType3)
```

setthe sdsType3

getSdsType4SequenceType

```
public static SdsType4SequenceType getSdsType4SequenceType()
```

Return the sdsType4SequenceType

Reference: [SdsType4SequenceType](#)

setSdsType4SequenceType

```
public static void setSdsType4SequenceType(SdsType4SequenceType  
sdsType4SequenceType)
```

setthe sdsType4SequenceType

Reference: [SdsType4SequenceType](#)

7.2.8.17 SdsType4SequenceType

Package Name:com.hytera.call.param.roip

byte[]	sdsData
	sds data
Integer	bitLength
	bit length

SdsType4SequenceTypeMethods

getSdsData

public byte[] getSdsData()

Return the sdsData

setSdsData

public void setSdsData(byte[] sdsData)

set the sdsData

7.2.8.18 Integer16

Package Name:com.hytera.call.param.roip

Integer	value
---------	--------------

Integer16 Methods

getValue

```
public Integer getValue()
```

Return the value

setValue

```
public void setValue(Integer value)
```

set the value

7.2.8.19 RoipToBeEncryptShortData

Package Name:com.hytera.call.param.roip

byte[]	data
byte[]	encryptedData
DATA_TYPE	dataType
boolean	encrypted
SDS_TYPE	messageType

RoipToBeEncryptShortData Methods

getData

```
public byte[] getData()
```

Return the data

setData

```
public void setData(byte[] data)
```

set the data

getEncryptedData

```
public byte[] getEncryptedData()
```

Return the encryptedData

setEncryptedData

```
public void setEncryptedData(byte[] encryptedData)
```

set the encryptedData

getDataType

```
public DATA_TYPE getDataType()
```

Return the dataType

Reference:[DATA_TYPE](#)

setDataType

```
public void setDataType(DATA_TYPE dataType)
```

set the dataType

Reference:[DATA_TYPE](#)

isEncrypted

```
public boolean isEncrypted()
```

setEncrypted

```
public void setEncrypted(boolean encrypted)
```

getMessageType

```
public SDS_TYPE getMessageType()
```

Return the messageType

Reference: [SDS_TYPE](#)

setMessageType

```
public void setDataType(SDS_TYPE messageType)
```

set the messageType

Reference: [SDS_TYPE](#)

7.2.8.20 TetraRoipEncryptMmsInfo

Package Name:com.hytera.call.param.roip

String	mDestSsi
CALL_TYPE	mCallType
byte[]	mMsgByte
DATA_TYPE	mDataType
SDS_TYPE	mSdsType

boolean

mIsEncrypted**TetraRoipEncryptMmsInfo Methods****toString**

```
public java.lang.String toString()
```

getmDestSsi

```
public java.lang.String getmDestSsi()
```

return the mDestSsi

setmDestSsi

```
public void setmDestSsi(String mDestSsi)
```

set the mDestSsi

getmCallType

```
public CALL_TYPE getmCallType()
```

return the mCallType

Reference:[CALL_TYPE](#)

setmCallType

```
public void setmCallType(CALL_TYPE mCallType)
```

set the mCallType

Reference:[CALL_TYPE](#)

getmMsgByte

```
public byte[] getmMsgByte()
```

return the mMsgByte

setmMsgByte

```
public void setmDestSsi(byte[] mMsgByte)
```

set the mMsgByte

getmDataType

```
public DATA_TYPE getmDataType()
```

return the mDataType

Reference:[DATA_TYPE](#)

setmDataType

```
public void setmDataType(DATA_TYPE mDataType)
```

set the mDataType

Reference:[DATA_TYPE](#)

getmSdsType

```
public SDS_TYPE getmSdsType()
```

return the mSdsType

Reference:[SDS_TYPE](#)

setmSdsType

```
public void setmSdsType(SDS_TYPE mSdsType)
```

set the mSdsType

Reference: **SDS_TYPE**

ismIsEncrypted

```
public boolean ismIsEncrypted()
```

setmIsEncrypted

```
public void setmIsEncrypted(Boolean mIsEncrypted)
```

7.2.9 Constants

7.2.9.1 CallType

Package Name: com.hytera.call.param

Include all call types

CallType Constant fields

Qualifier and Type	Constant fields	Value
public static final int	DMR_C_INDIVIDUAL_CALL	0
public static final int	DMR_T_INDIVIDUAL_CALL	0
public static final int	POC_GROUP_CALL	0
public static final int	TETRA_D_INDIVIDUAL_CALL	0
public static final int	ROIP_TETRA_INDIVIDUAL_CALL	0
public static final int	TETRA_T_INDIVIDUL_CALL	0
public static final int	THIRD_PARTY_GROUP_VOICE_CALL	0

public static final int	FRAMEWORK_INDIVIDUAL_CALL	0
public static final int	DMR_T_GROUP_CALL	1
public static final int	DMR_C_GROUP_CALL	1
public static final int	ROIP_TETRA_GROUP_CALL	1
public static final int	TETRA_D_GROUP_CALL	1
public static final int	TETRA_T_GROUP_CALL	1
public static final int	THIRD_PARTY_INDIVIDUAL_VOICE_CALL	1
public static final int	FRAMEWORK_GROUP_CALL	1
public static final int	POC_INDIVIDUAL_CALL	1
public static final int	ROIP_TETRA_BROADCAST_CALL	2
public static final int	THIRD_PARTY_EMERGENCY_VOICE_CALL	2
public static final int	POC_BROADCAST_CALL	2
public static final int	DMR_T_ALL_CALL	2
public static final int	FRAMEWORK_ALL_CALL	2
public static final int	DMR_C_ALL_CALL	2
public static final int	DMR_T_PSTN_CALL	3
public static final int	POC_ALL_CALL	3
public static final int	THIRD_PARTY_AMBIENCE_VOICE_LISTENING	3
public static final int	DMR_C_ANALOG_CALL	3

public static final int	TETRA_T_BROADCAST	3
public static final int	THIRD_PARTY_BROADCAST_VOICE_CALL	4
public static final int	DMR_C_ANALOG_EMERGENCY_CALL	4
public static final int	DMR_T_PABX_CALL	4
public static final int	THIRD_PARTY_ALL_VOICE_CALL	5
public static final int	DMR_T_BROADCAST_GROUP_CALL	17
public static final int	DMR_T_BROADCAST_ALL_CALL	18

7.2.9.2 CallDisplayedMode

Package Name: com.hytera.call.param

Include all call display mode

CallDisplayedMode Constant fields

Qualifier and Type	Constant fields	Value
public static final int	INVALID	-1
public static final int	UI	0
public static final int	NORMAL_NOTIFICATION	1
public static final int	HEADUP_NOTIFICATION	2
public static final int	TOUCH_TO_RETURN_NOTIFICATION	3
public static final int	PHONE_HOLD_NOTIFICATION	4
public static final int	UI_AND_HEADUP_NOTIFICATION	5

Qualifier and Type	Constant fields	Value
public static final int	UI_AND_NORMAL_NOTIFICATION	6

CallDisplayedModemethods

hasUI

```
public static boolean hasUI(int mode)
```

Whether the call has UI display

Parameters:

mode– CallDisplayedMode

Reference:[CallDisplayedMode](#)

getString

```
public static String getString(int mode)
```

Parameters:

mode– CallDisplayedMode

Reference:[CallDisplayedMode](#)

7.2.9.3 CallEndReason

Package Name: com.hytera.call.param

CallEndReason Constant fields

Qualifier and Type	Constant fields	Value
public static final int	CALL_FAIL_REASON_NUMBER_FORMAT_ERROR	100

7.2.9.4 CallMode

Package Name: com.hytera.call.param

Include all call mode

CallMode Constant fields

Qualifier and Type	Constant fields	Value
public static final int	READ_FROM_RCDB	-1
public static final int	HALF_DUPLEX	0
public static final int	FULL_DUPLEX	1

7.2.9.5 CallParam

Package Name: com.hytera.call.param

Include call param

CallParam Constant fields

Qualifier and Type	Constant fields	Value
public static final String	NET_TYPE	net_type
public static final String	CALL_TYPE	call_type
public static final String	CONTACT_ID	contact_id
public static final String	NUMBER	number
public static final String	HOOK	hook
public static final String	CALL_MODE	call_mode

public static final String	PRIORITY	priority
public static final String	TX_DEMAND	tx_demand
public static final String	FORCE_BRINGUP_ACTIVITY	force_bringup_activity
public static final String	AUTO_COMPLETE_FULL_NUMBER	auto_complete_full_number
public static final String	E2EE	e2ee

7.2.9.6 CallPriority

Package Name: com.hytera.call.param

Include all call priority

CallPriority Constant fields

Qualifier and Type	Constant fields	Value
public static final int	DMR_C_NORMAL	0
public static final int	DMR_C_EMERGENCY_CALL	1
public static final int	DMR_C_EMERGENCY_ALARM	2
public static final int	DMR_C_REMOTE_MONITOR	3
public static final int	DMR_T_NORMAL	0
public static final int	DMR_T_PRE_EMPTIVE	1
public static final int	DMR_T_EMERGENCY	2
public static final int	TETRA_T_NORMAL_LOWEST	1
public static final int	TETRA_T_NORMAL_HIGHEST	14

Qualifier and Type	Constant fields	Value
public static final int	TETRA_T_PRE_EMPTIVE	14
public static final int	TETRA_T_EMERGENCY	15
public static final int	TETRA_D_NORMAL	0
public static final int	TETRA_D_HIGHT	1
public static final int	TETRA_D_PRE_EMPTIVE	2
public static final int	TETRA_D_EMERGENCY	3
public static final int	ROIP_PRIORITY_NOT_DEFINED	0
public static final int	ROIP_PRIORITY_1	1
public static final int	ROIP_PRIORITY_2	2
public static final int	ROIP_PRIORITY_3	3
public static final int	ROIP_PRIORITY_4	4
public static final int	ROIP_PRIORITY_5	5
public static final int	ROIP_PRIORITY_6	6
public static final int	ROIP_PRIORITY_7	7
public static final int	ROIP_PRIORITY_8	8
public static final int	ROIP_PRIORITY_9	9
public static final int	ROIP_PRIORITY_10	10
public static final int	ROIP_PRIORITY_11	11
public static final int	ROIP_PRE_EMPTIVE_1	12
public static final int	ROIP_PRE_EMPTIVE_2	13
public static final int	ROIP_PRE_EMPTIVE_3	14
public static final int	ROIP_PRE_EMPTIVE_EMERGENCY	15

Qualifier and Type	Constant fields	Value
public static final int	POC_NORMAL	0
public static final int	POC_PRE_EMPTIVE	1
public static final int	POC_EMERGENCY	2

7.2.9.7 NetType

Package Name: com.hytera.call.param

Include all network types

NetType Constant fields

Qualifier and Type	Constant fields	Value
public static final int	NET_TYPE_INVALID	0
public static final int	NET_TYPE_PHONE	1
public static final int	NET_TYPE_DMR_C	2
public static final int	NET_TYPE_DMR_T	3
public static final int	NET_TYPE_TETRA_D	4
public static final int	NET_TYPE_TETRA_T	5
public static final int	NET_TYPE_ROIP_TETRA	6
public static final int	NET_TYPE_ROIP_DMR	7
public static final int	NET_TYPE_PSLTE	8
public static final int	NET_TYPE_THIRD_PARTY_VOICE	9

public static final int	NET_TYPE_THIRD_PARTY_VIDEO	10
-------------------------	-----------------------------------	----

NetType Methods

isNarrowBandCall

```
public static boolean isNarrowBandCall(int netType)
```

Return true if the call is NarrowBand call, else return false

Parameters:

netType—network type

Reference: [NetType](#)

7.2.9.8 CallingPartyIDType

Package Name: com.hytera.call.param

Include all CallingPartyID Type

CallingPartyIDType Constant fields

Qualifier and Type	Constant fields	Value
public static final int	CALLING_PARTY_TSI_NOT_PRESENT	0
public static final int	CALLING_PARTY_PRESENT_TRUE_SSI_IS_USED	1
public static final int	CALLING_PARTY_PRESENT_PSEUDO_SSI_IS_USED	2

7.2.9.9 CallRecord

Package Name: com.hytera.call.param.callrecord

CallRecord.RecordError Constant fields

Qualifier and Type	Constant fields	Value
public static final int	RECORD_ERROR_NO_ERROR	0
public static final int	RECORD_ERROR_NO_SDCARD	1
public static final int	RECORD_ERROR_SDCARD_FULL	2
public static final int	RECORD_ERROR_CREATE_FILE_FAIL	3
public static final int	RECORD_ERROR_ADD_DB	4
public static final int	RECORD_ERROR_UPDATE_DB	5
public static final int	RECORD_ERROR_SDCARD_FULL_WHEN_RECORDING	6
public static final int	RECORD_ERROR_TIME_TOO_SHORT	7

CallRecord.RecordState Constant fields

Qualifier and Type	Constant fields	Value
public static final int	RECORD_STATE_INVALID	0
public static final int	RECORD_STATE_READING	1
public static final int	RECORD_STATE_START	2
public static final int	RECORD_STATE_STOP	3
public static final int	RECORD_STATE_PASUE	4
public static final int	RECORD_STATE_SUCESS	5

7.2.9.10 HytKeyCode

Package Name:com.hytera.call.listener

Include hyt key code

HytKeyCode Constant fields

Qualifier and Type	Constant fields	Value
public static final int	KEY_CODE_BACK	0x02
public static final int	KEY_CODE_G2_PTT	0x5A
public static final int	KEY_CODE_EXTERNAL_PTT_Key	0x5B
public static final int	KEY_CODE_BT_PTT	0x5C

7.2.9.11 ENCRYPTION_TYPE

Package Name: com.hytera.call.param.roip

enum:ENCRYPTION_TYPE
NO_ENCRYPTION
HYTERA_SOFT_ENCRYPTION
MTU_HARD_ENCRYPTION
TF_HARD_ENCRYPTION
TETRA_ENCRYPTION

7.2.9.12 DATA_TYPE

Package Name: com.hytera.call.param.roip

enum:DATA_TYPE

ISO_7

ISO_8859_1

ISO_8859_2

ISO_8859_5

ISO_8859_6

ISO_8859_9

UNICODE

7.2.9.13 CC_DISCONNECT_CAUSE

Package Name: com.hytera.call.param.roip

enum:CC_DISCONNECT_CAUSE

CAUSE_UNKNOWN

USER_REQUESTED_DISCONNECT

CALLED_PARTY_BUSY

CALLED_PARTY_NOT_REACHABLE

CALLED_PARTY_DOES_NOT_SUPPORT_ENCRYPTION

CONGESTION_IN_INFRASTRUCTURE

NOT_ALLOWED_TRAFFIC_CASE

INCOMPATIBLE_TRAFFIC_CASE

REQUESTED_SERVICE_NOT_AVAILABLE**PRE_EMPTIVE_USE_OF_RESOURCE****INVALID_CALL_IDENTIFIER****CALL_REJECTED_BY_CALLED_PARTY****NO_IDLE_CC_ENTITY****EXPIRY_OF_TIMER****DMX_REQUESTED_DISCONNECT****ACKNOWLEDGED_SERVICE_NOT_COMPLETED****RESOURCE_FAILED****UNKNOWN_RESOURCE****INACTIVE_RESOURCE****RESOURCE_USED_IN_OTHER_CALL****CELL_RESELECTION****CALL_SETUP_REPETITION****IC_LEAVING_CALL_NO_CALL_END****UNKNOWN_TETRA_IDENTITY****SS_SPECIFIC_DISCONNECT****UNKNOWN_EXTERNAL_SUBSCRIBER_IDENTITY****CALL_RESTORATION_OF_OTHER_USER_FAILED**

WRONG_COMMUNICATION_TYPE**CALL_TRANSFER_SUBSCRIBE_ERROR****UNKNOWN_VIRTUAL_DEST****ROUTING_FAILURE_TO_VIRTUAL_DEST****OFFLINE_CIRCUIT_MODE_MONITORING_GATEWAY****UNKNOWN_CALL_IDENTIFIER****MON_TO_SAME_VIRTUAL_DEST_ALREADY_ACTIVE****UNKNOWN_MONITORING_HANDLE****CALL_IN_WRONG_STATE****LOSS_OF_MAIN_COS_CONNECTION****TRANSFER_OF_MAIN_COS_FAILED****LOSS_OF_COS_CONNECTION****CALL_FORWARDING****MAX_NUM_OF_DUPLEX_INTERCEPTIONS_EXCEEDED****CURRENTLY_NOT_AVAILABLE****PREMATURE_DISCONNECT****TAP_REQUESTED_DISCONNECT****NO_PERMISSION****INVALID_PAYLOAD_TYPE**

RESOURCE_TYPE_NOT_SUPPORTED**SUCCESS****PTTC_DISCONNECTED**

7.2.9.14 PROTOCOL_RESULT

Package Name: com.hytera.call.param.roip

enum:PROTOCOL_RESULT**SUCCESS****FAILED****UNKNOWN_ERROR****IO_ERROR****TIMEOUT****AUTHENTICATION_ERROR****INVALID_ACCOUNT****REGISTRATION_REJECTED****REGISTRATION_DENIED****PROTOCOL_ERROR****NO_CONNECTION****SERVICE_NOT_OPEN****STILL_OPENING**

STILL_CLOSING

ALREADY_OPEN

SENT

SYSTEM_CONFIRMED

USER_CONFIRMED

ACTIVE_CALL

DUPLEX_CALL

INACTIVE_CALL

CALL_WRONG_STATE

TX_NOT_GRANTED

TX_GRANTED

TX_GRANTED_TO_ANOTHER_USER

TX_CEASED

TX_QUEUED

NO_AUDIO_DESTINATION

UNKNOWN_GROUP

SUBSCRIBER_ERROR

DENIED

7.2.9.15 SDS_RESULT

Package Name: com.hytera.call.param.roip

enum:SDS_RESULT

SUCCESS

GENERAL_FAILURE

SUBSCRIBER_ERROR

SUBSCRIBER_OFFLINE

SUBSCRIBER_QUEUE_OVERLOAD

SUBSCRIBER_DISABLED

PERMISSION_DENIED

TRANSMISSION_FAILURE

DELIVIRY_MODE_NOT_PERMITTED

7.2.9.16 GROUP_ATTACH_RESULT

Package Name: com.hytera.call.param.roip

enum:GROUP_ATTACH_RESULT

GROUP_ATTACH_OK

GROUP_ATTACH_DENIED

SUBSCRIBER_ERROR

ALREADY_ATTACHED**UNKNOWN_GROUP****NO_CONNECTION**

7.2.9.17 SDS_STATUS_CODES

Package Name: com.hytera.call.param.roip

enum:SDS_STATUS_CODES**CALLBACK_REQUEST****URGENT_CALLBACK_REQUEST****SS_AL_EMERGENCY_CALL_REQUEST****SS_AL_CALL_REQUEST****TX_INHIBIT_OFF****TX_INHIBIT_ON****STATUS_VALUE_OUT_OF_RANGE****DESTINATION_BUSY****DESTINATION_NOT_AUTHORISED****DESTINATION_NOT_REACHABLE****DESTINATION_DOES_NOT_EXIST****UNRECOGNISED_ADDRESS**

NOT_AUTHORIZED

GENERAL_STATUS_NEGATIVE_ACKNOWLEDGEMENT

BS_FALLBACK

EMERGENCY_STATUS

UNDEFINED

7.2.9.18 SDS_TYPE

Package Name: com.hytera.call.param.roip

enum:SDS_TYPE

NULL

STATUS

TYPE2

TYPE3

TYPE4

7.2.9.19 SERVICE_TYPE

Package Name: com.hytera.call.param.roip

enum:SERVICE_TYPE

SINGLE

CALL**SHORT_DATA****GROUP**

7.2.9.20 CALL_TYPE

Package Name: com.hytera.call.param.roip

enum:CALL_TYPE**INDIVIDUAL_CALL****GROUP_CALL****BROADCAST_CALL**

7.2.10 Other

7.2.10.1 BasePhysicalPttCallback

Package: com.hytera.call.listener

BasePhysicalPttCallback Methods

getOutgoingCallParam

OutgoingCallParam getOutgoingCallParam()throws android.os.RemoteException

Get call param.TP implements the abstract class BasePhysicalPttCallback and passes it to Hytera Call Application with the method CallManager.requestPhysicalPtt,then HyteraCall Application gets outgoing call param to set up call with the BasePhysicalPttCallback implemented by TP if physical PTT down.

Throws:

`android.os.RemoteException`

Reference: [OutgoingCallParam](#)

7.2.10.2 RoipProxy

Package: com.hytera.callapi.roip

RoipProxy Methods

createDataMessage

```
public RoipToBeEncryptShortData createDataMessage(TetraRoipEncryptMmsInfo
                                                    iTetraRoipEncryptMmsInfo)
```

Reference: [RoipToBeEncryptShortData](#), [TetraRoipEncryptMmsInfo](#)

sendEncryptShortData

```
public int sendEncryptShortData(RoipToBeEncryptShortData
                                roipToBeEncryptShortData)
```

Send encrypt short data , if failed return -1

Reference: [RoipToBeEncryptShortData](#)

sendEncryptedMessage

```
public int sendEncryptedMessage(SDS_DataSequenceTypeSDS_DataSequenceType)
```

Send Roip Msg

Reference: [SDS_DataSequenceType](#)

sendRoipMsg

```
public int sendRoipMsg(String destSsi, int callType, byte[] msg, DATA_TYPE
                        dataType, int msgType)
```

Send Roip Msg

Parameters:

destSsi - number of the party

callType - Sms Type, refer to CallType.ROIP_GROUP_CALL etc

msg - content of sms

dataType -type of sms data, r

efer to RoipConst.SDS_DATA_TYPE_UTF8 etc

msgType-refer to RoipProxy.MsgType

Reference: [DATA_TYPE](#)

sendRoipStatus

```
public int sendRoipStatus(String destSsi, int callType, int statusCode)
```

Send Roip Msg

Parameters:

destSsi - number of the party

callType - Sms Type, refer to CallType.ROIP_GROUP_CALL etc

statusCode- status code

Returns:

msgID - Flag of Sms.if ROIP module is uninitialized , msgID may be equal to

RoipConst.INVALID_SDS_ID

addRoipMsgListener

```
public void addRoipMsgListener(IRoipMsgListener listener)
```

Add listener of RoipMsg

Parameters:

listener—refer to IRoipMsgListener

removeRoipMsgListener

```
public void removeRoipMsgListener(IRoipMsgListener listener)
```

Remove listener of RoipMsg

Parameters:

listener—refer to IRoipMsgListener

isConnected

```
public boolean isConnected()
```

Whether to be connected to Roip Server

7.3 BasicSettingManager

7.3.1 BasicSettingManager Introduction

BasicSettingManager contains several methods to get basic information about BasicSetting module, such as radio information, device scene mode, device type etc.

Package: com.hytera.api.base.common

7.3.2 BasicSettingManager Method

getRadioAlias

```
java.lang.String getRadioAlias()
```

throws [HyteraSDKException](#)

Obtain radio alias

Returns:

Radio alias

Throws:

HyteraSDKException

Example:

```
mCommonManager = HyteraManager.getCommonManager(getApplicationContext());

try {
    mBasicSettingManager = mCommonManager.getBasicSettingManager();
    String alias = mBasicSettingManager.getRadioAlias();
} catch (HyteraSDKException e) {
    e.printStackTrace();
}
```

getRadioInfomation

```
void getRadioInfomation(int operate,
    BasicSettingListener.GetRadioInfoListener listener)
```

throws [HyteraSDKException](#)

Get device information accordinly by specific value.

Parameters:

operate—Different value represent different module.

0x0000 Model Number

0x0001 Firmware Version V1.0.10.000.01

0x0003 Boot Version
0x0004 ES704 Version
0x0005 CPLD Version
0x0006 TuningData Version
0x0007 FixedString Version
0x0008 Font Version
0x0009 Image Version
0x000A TEI/ESN information
0x000B SN Information
0x000C Battery Version

listener - `BasicSettingListener.GetRadioInfoListener`

Throws:

`HyteraSDKException`

Example:

```
try {  
    // operate = 0 ~ 12  
    mBasicSettingManager.getRadioInfomation(operate,mGetRadioInfoListener);  
} catch (HyteraSDKException e1) {  
    e1.printStackTrace();  
}  
  
GetRadioInfoListener mGetRadioInfoListener = new GetRadioInfoListener() {  
    @Override
```

```
        public void onCallback(int result, int operate, String info) {  
            }  
    };
```

getSceneMode

int getSceneMode() throws [HyteraSDKException](#)

Get scene mode information. Returns 0 if current mode is Ring Mode, returns 1 if current mode is Silent Mode.

Notice: This method have a further encapsulation based on android API. Using android native API also can get current scene mode, but it returns a different integer value. It will be a better choice to call this method to get scene mode value on Hytera Device.

Returns:

0x00 Ring mode

0x01 Mute mode

getDeviceType

int getDeviceType() throws [HyteraSDKException](#)

Returns:

0x00 DMR Device

0x01 TETRA Device

7.3.3 BasicSettingListener Class

7.3.3.1 BasicSettingListener Introduction

BasicSettingListener contains one inner static interface, it is used as a callback method when retrieving radio

information.

Package: com.hytera.api.base.common

7.3.3.2 BasicSettingListener Inner Class

Type	Class
static interface	BasicSettingListener.GetRadioInfoListener

public static interface

BasicSettingListener.GetRadioInfoListener

onCallback

void onCallback(int result, int operate,String info)

Parameters:

result-

0x0000 Success

0x0001 Failure

operate-

0x0000 Model Number

...

0x000C Battery Version

info-radio information

7.4 BatteryManager

7.4.1 BatteryManager Introduction

BatteryManager provides one method to getremaining battery information.

Package: com.hytera.api.base.common

7.4.2 BatteryManager Method

getBatteryLife

```
void getBatteryLife(  
    BatteryListener.GetBatteryLevelListener listener)
```

throws [HyteraSDKException](#)

This message is used by peripheral equipment to obtain remaining battery information (percentage of total), it contains methods like unregistering listener process, so when battery level changes the callback method would not response.

Notice: This method can get the same result with nativeandroid battery API. The difference from native API is that this method can return current battery level anytime when it is called.

Parameters:

listener—[BatteryListener.GetBatoryLevelListener](#)

Throws:

[HyteraSDKException](#)

Example:

```
GetBatteryLevelListener mGetBatteryLevelListener =  
    new GetBatteryLevelListener() {  
        @Override  
        public void onCallBack(int level) {  
            // return the percentage of total level.  
        }  
    };
```

```
//get battery life

try {

    mCommonManager.getBatteryManager().getBatteryLife(

        mGetBatteryLevelListener);

} catch (Exception e) {

    e.printStackTrace();

}
```

7.4.3 BatteryListener Class

7.4.3.1 BatteryListener Introduction

BatteryListener provides a static interface which is used as the callback method of obtaining remaining battery information.

Package: com.hytera.api.base.common

7.4.3.2 BatteryListener Inner Class

Type	Class
static interface	BatteryListener.GetBatteryLevelListener

public static interface **BatteryListener.GetBatteryLevelListener**

onCallBack

void onCallBack(int level)

Parameters:

`level-percentage`

7.5 GpsManager

7.5.1 GpsManager Introduction

GpsManager contains three methods: one is used to obtain location information and the other two are register/unregister methods.

Package: com.hytera.api.base.common

7.5.2 GpsManager Method

getLocationInfo

`void getLocationInfo()` throws [HyteraSDKException](#)

Get location

Notice: This method has a different process with native android GPS API, it pulls some data from narrowband system. Calling this method to get location information is a better choice.

Throws:

[HyteraSDKException](#)

registerListener

`void registerListener(GpsManagerListener listener)`
throws [HyteraSDKException](#)

Register listener

Parameters:

`listener` - [android.dsp.proxy.GpsManagerListener](#)

Throws:

[HyteraSDKException](#)

unregisterListener

```
void unregisterListener(GpsManagerListener listener)  
    throws HyteraSDKException
```

Unregister listener, call this method when you do not need to request location information.

Parameters:

listener - `android.dsp.proxy.GpsManagerListener`

Throws:

`HyteraSDKException`

Example:

```
GPSManagerListener mGPSManagerListener = new GPSManagerListener() {

    @Override

    public void onLocationInfo(DuGPSLocationInfo info)

    throws RemoteException {

        Log.d(TAG, "Longitude: "+SDKUtil.getGpsLongitude(info.Longitude)+

            " Latitude: "+SDKUtil.getGpsLatitude(info.Latitude));

    }

    @Override

    public void onGPSStateAck(int result, int enable)

    throws RemoteException {

    }

    @Override

    public void onGPSState(int enable) throws RemoteException {

    }

    @Override

    public void getGPSVersionAck(int result, String version)

    throws RemoteException {

    }

};
```



```
//get location information

try {

    mCommonManager.getGpsManager().registerListener(mGPSManagerListener);

    mCommonManager.getGpsManager().getLocationInfo();

} catch (HyteraSDKException e) {

    e.printStackTrace();

}
```

The callback method `onLocationInfo(info)` returns the location. All the information returned are coded with ASCII code. `SDKUtil` provides methods to transform the information to readable.

7.5.3 GpsManager Data

7.5.3.1 DuGPSLocationInfo

DuGPSLocationInfo class introduction:

Filed	Type	Description
result	int	0x0000 Success, 0x0001 Failure
status	int	'A' Available, 'V' Unavailable
Time	byte[]	Time (HHMMSS) UTC Time
Date	byte[]	Date (DDMMYY) UTC Time
NSFlag	int	'N' North, 'S' South
Latitude	byte[]	DDMM.MMMM dd: Degree (00 ~ 90) mm.mmmm: Minute (00.0000 ~ 59.9999)
EWFlag	int	'E' East, 'W' West
Longitude	byte[]	DDDMM.MMMM DDD: Degree (000 ~ 180) MM.MMMM: Minute (00.0000 ~ 59.9999)
Speed	int	0.0 - 999, Knots
Direction	int	Azimuth (000 ~359) The north is 000 degree, clockwise for every increase of 1 degree, Azimuth plus 1.
Satelites	int	Satellites (00-12)
Altitude	byte[]	Altitude (-9999.9 - 99999.9)

7.6 LedVibrationManager

7.6.1 LedVibrationManager Introduction

LedVibrationManager contains methods to control LED and vibrator.

Package: com.hytera.api.base.common

7.6.2 LedVibrationManager Method

controlLed

```
void controlLed(int ledIndex,  
                int operation,  
                int ledColor,  
                int ledCycle,  
                int onTime,  
                int offTime,  
                LedVibrationListener.ControlLedListener listener)  
    throws HyteraSDKException
```

The Device has 2 led lights, LED 1 and LED 2.

The figure below describes the position of the two LEDs in the radio.



LED 1 have 3 colors(red, green, orange) and LED 2 have 2 colors(red, blue).

Parameters:

ledIndex - 0 -LED 1(3 colors: red, green, orange),

1 - LED 2(2 colors: red, blue)

operation - 0 turn off, 1 turn on

when operation=0,parameters below are unusable

ledColor - 0red,1green,2orange,3blue

ledCycle - 0 infinite loop, 1~10 loop several(1~10) times

onTime - 50~60000 light duration(ms)

offTime - 50~60000 off duration(ms)

listener - `LedVibrationListener.ControlLedListener`

Throws:

`HyteraSDKException`

unregisterControlLed

void unregisterControlLed() throws [HyteraSDKException](#)

UnregisterControlLedListener, call this method when you needn't to monitor led switch status.

Throws:

[HyteraSDKException](#)

controlVibration

```
void controlVibration(int operation,
                     int vibrationCycle,
                     int onTime,
                     int offTime,
                     int level,
                     LedVibrationListener.ControlVibrationListener listener)
    throws HyteraSDKException
```

Notice: It is a further encapsulation method based on android native vibration API, call native vibration API can also get the relative result.

Parameters:

operation - 0 close vibration, 1 open vibration

when operation=0, parameters below are unusable

vibrationCycle - 0 infinite loop, 1~10 loop several times

onTime - 50~60000 (ms)

offTime - 0~60000 (ms)

level - 0~7 vibrate level, 0 means low and 7 means high

listener - [LedVibrationListener.ControlVibrationListener](#)

Throws:

[HyteraSDKException](#)

unregisterControlVibration

`void unregisterControlVibration()` throws [HyteraSDKException](#)

UnregisterControlVibrationListener, call this method when you don't need to monitor vibration switch status.

Throws:

[HyteraSDKException](#)

7.6.3 LedVibrationListener Class

7.6.3.1 LedVibrationListener Introduction

LedVibrationListener contains 2 static interfaces about LED, vibrate respectively.

Package: com.hytera.api.base.common

7.6.3.2 LedVibrationListener Inner Class

Type	Class
static interface	<code>LedVibrationListener.ControlLedListener</code>
static interface	<code>LedVibrationListener.ControlVibrationListener</code>

```
public static interface LedVibrationListener.ControlLedListener
```

Callback of LED listener

onCallback

```
void onCallback(int result)
```

Parameters:

result - 0x0000 Succeeded,

0x0001 Failed,
0x8000 Invalid parameter

public static interface

LedVibrationListener.ControlVibrationListener

Callback of vibration listener

onCallback

void onCallback(int result)

Parameters:

result - 0x0000 Succeeded,
0x0001 Failed,
0x8000 Invalid parameter

7.7 RadioManager

7.7.1 RadioManager Introduction

RadioManager contains methods to manage radio related modules, such as obtaining RSSI value of current timeslot, querying radio current status.

Package: com.hytera.api.base.common

7.7.2 RadioManagerMethod

getRssiValue

int getRssiValue(int type) throws [HyteraSDKException](#)

Get the RSSI value of current timeslot.

Parameters:

type– 0x00 Level
0x01 dB Value

Returns:

when Type=0, Value 0-5, means Level 0-5;
when Type=1, Value means dB value, is a signed Integer

Throws:

[HyteraSDKException](#)

Example:

```
try {

    mCommonManager = HyteraManager.getCommonManager(getApplicationContext());

    mRadioManager = mCommonManager.getRadioManager();

    mRadioManager.registerRadioManagerListener(mRadioCommonListener);

    int rssiValue = mCommonManager.getRadioManager ().getRssiValue(0);

} catch (HyteraSDKException e) {

    e.printStackTrace();

}
```

```
RadioListener.RadioCommonListener mRadioCommonListener = new
    RadioListener.RadioCommonListener(){

        @Override

        public void onRssiValue(int type, int value) {
```



```
        super.onRssiValue(type, value);

        //type, your input parameter

        //value, returned value

    }

};
```

queryDisableStatus

```
void queryDisableStatus(

    RadioListener.QueryDisableStatusListener listener)

    throws HyteraSDKException
```

Check the stun or kill status of the device.

Parameters:

listener -RadioListener.QueryDisableStatusListener

Throws:

HyteraSDKException

registerRadioManagerListener

```
void registerRadioManagerListener(

    RadioListener.RadioCommonListener listener)

    throws HyteraSDKException
```

Register RSSI listener to monitor the value change. When RSSI value changes, RadioCommonListener callback method is responded.

Parameters:

listener - `RadioListener.RadioCommonListener`

Throws:

`HyteraSDKException`

unRegisterRadioManagerListener

```
void unRegisterRadioManagerListener(  
    RadioListener.RadioCommonListener listener)  
  
    throws HyteraSDKException
```

Unregister RSSI listener, call the method when you don't need to watch the change of RSSI.

Parameters:

listener - `RadioListener.RadioCommonListener`

Throws:

`HyteraSDKException`

BroadcastDisableStatus

Action: `ACTION_DISABLE_STATUS`

Status: `DISABLE_STATUS`

Broadcast the device status, the returned value is integer.

status:

- 0TETRA normal/active or DMR normal/active
- 1 TETRA radio temporarily disabled or DMR stunned
- 2 TETRA radio permanently disabled or DMR killed
- 3 TETRA subscription temporarily disabled
- 4TETRA subscription permanently disabled

5TETRA radio temporarily disabled and subscription permanently disabled

6 TETRA radio permanently disabled and subscription temporarily disabled

7TETRA radio and subscription temporarily disabled

8 TETRA radio and subscription permanently disabled

7.7.3 RadioListener

7.7.3.1 RadioListener Introduction

PowerListener contains two static interfaces.

Package: com.hytera.api.base.common

7.7.3.2 RadioListenerInner Class

Type	Class
static interface	RadioListener.QueryDisableStatusListener
static class	RadioListener.RadioCommonListener

public static interface

RadioListener.QueryDisableStatusListener

Check the decice status

onCallBack

void onCallBack(int result, int status)

Parameters:

result-

0x0000 Succeed

0x0001 Failed

0x0002-0x7FFF Specific protocol parsing error codes

0x8000 Invalid parameter

0x8001-0xAFFF Extended for error codes

status-

0TETRA normal/active or DMR normal/active

1 TETRA radio temporarily disabled or DMR stunned

2 TETRA radio permanently disabled or DMR killed

3 TETRA subscription temporarily disabled

4TETRA subscription permanently disabled

5TETRA radio temporarily disabled and subscription
permanently disabled

6 TETRA radio permanently disabled and subscription temporarily
disabled

7TETRA radio and subscription temporarily disabled

8 TETRA radio and subscription permanently disabled

Public static class

RadioListener.RadioCommonListener

onRssiValue

public void onRssiValue(int type,int value)

[Parameters:](#)

type–

0x00 Level

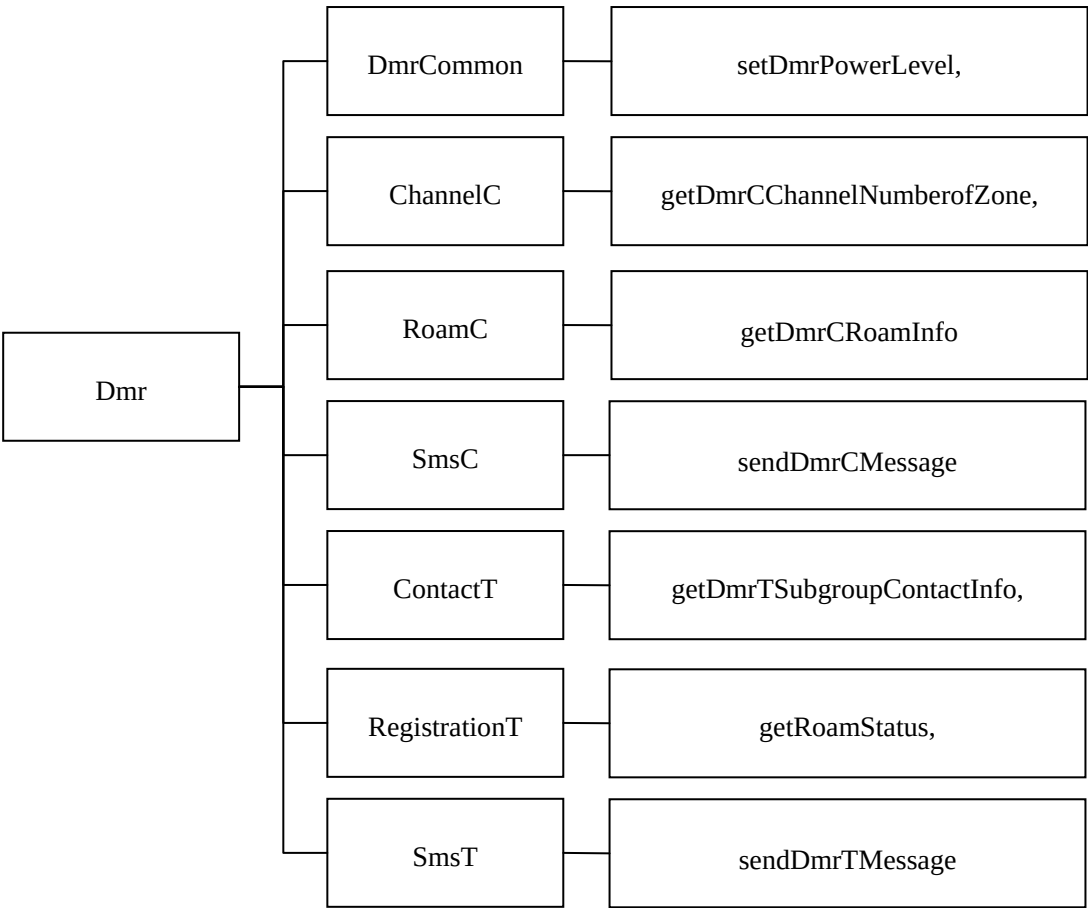
0x01 dB Value

value–

when Type=0, Value 0-5, means Level 0-5;

when Type=1, Value means dB value, is a signed Integer

8. DmrManager Overview



The picture shows the overview of DmrManager. It contains 7 sub managers to manage different modules.

DmrCommonManager contains 2 methods to set/get Dmr device power.

ChannelCManager, RoamCManager and SmsCManager are used to manage 3 different Dmr Conventional modules.

RegistrationTManager, ContactTManager and SmsTManager are used to manage 3 different Dmr Trunking modules.

8.1 DmrManager

8.1.1 DmrManager Introduction

DmrManager contains 7 methods to get different manager instances.

Package: com.hytera.api.base.dmr

8.1.2 DmrManager Method

getDmrCommonManager

```
public DmrCommonManager getDmrCommonManager() throws  
    HyteraSDKException
```

Returns: [DmrCommonManager](#) instance.

Throws: [HyteraSDKException](#)

getChannelCManager

```
public ChannelCManager getChannelCManager()  
    throws HyteraSDKException
```

Returns: [ChannelCManager](#) instance.

Throws: [HyteraSDKException](#)

getRoamCManager

```
public RoamCManager getRoamCManager()throws HyteraSDKException
```

Returns: [RoamCManager](#) instance.

Throws: [HyteraSDKException](#)

getSmsCManager

```
public SmsCManager getSmsCManager()throws HyteraSDKException
```

Returns: [SmsCManager](#) instance.

Throws: [HyteraSDKException](#)

getContactTManager

```
public ContactTManager getContactTManager()
```

```
    throws HyteraSDKException
```

Returns: ContactTManager instance.

Throws: HyteraSDKException

getRegistrationTManager

```
public RegistrationTManager getRegistrationTManager()
```

```
    throws HyteraSDKException
```

Returns: RegistrationTManager instance.

Throws: HyteraSDKException

getSmsTManager

```
public SmsTManager getSmsTManager()throws HyteraSDKException
```

Returns: SmsTManager instance.

Throws: HyteraSDKException

8.2 DmrCommonManager

8.2.1 DmrCommonManagerIntroduction

DmrCommonManagercontains two methods to get power level and set power level tospecific valueof DMR device.

Package: com.hytera.api.base.dmr

8.2.2 DmrCommonManagerMethod

getDmrPowerLevel

`int getDmrPowerLevel()` throws [HyteraSDKException](#)

Get power level of DMR devices and returns an integer value.

Returns:

0x00 High Power level

0x02 Low Power level

Throws:

[HyteraSDKException](#)

setDmrPowerLevel

`void setDmrPowerLevel(int level,
 DmrCommonListener.SetDmrPowerLevelListener listener)`
throws [HyteraSDKException](#)

Set power level of DMR device with a specific integer value.

Parameters:

level–

0x00 High Power level

0x02 Low Power level

listener - [DmrCommonListener.SetDmrPowerLevelListener](#)

Example:

```
//get DmrManager instance  
  
private DmrManager mDmrManager;  
  
mDmrManager = HyteraManager.getDmrManager(getApplicationContext());
```

```
//set dmr power level to low level

try {

    mDmrManager.getDmrCommonManager().setDmrPowerLevel(2,

    new SetDmrPowerLevelListener(){

        @Override

        public void onCallback(int result, int value) {

        }

    });

} catch (HyteraSDKException e) {

    e.printStackTrace();

}
```

BroadcastDmrPowerLevel

Action:ACTION_SET_DMR_POWER_LEVEL_REPLY

Level:POWER_LEVEL

Broadcast the power level when it changes.

Level:

0x00 High Power level

0x02 Low Power level

Example:

```
private static final String ACTION_DMR_POWER_LEVEL =
    DmrCommonManager.ACTION_SET_DMR_POWER_LEVEL_REPLY;
```

```
private static final String LEVEL = DmrCommonManager.POWER_LEVEL;

// onReceive

if (action.equals(ACTION_DMR_POWER_LEVEL)) {

    Log.d(TAG, LEVEL+"="+intent.getIntExtra(LEVEL, -1));

}
```

8.2.3 DmrCommonListener

Package: com.hytera.api.base.dmr

8.2.3.1 DmrCommonListener Introduction

Type	Class
static interface	DmrCommonListener.SetDmrPowerLevelListener

public static interface DmrCommonListener.SetDmrPowerLevelListener

onCallback

void onCallback(int result, int value)

Parameters:

- result - Result Description
 - 0x0000 Succeeded
 - 0x0001 Failure
 - 0x8000 Invalid parameter
- value - Number Description

0x00 High Power level

0x02 Low Power level

8.3 ChannelCManager

8.3.1 ChannelCManager Introduction

This interface contains methods about how to set or get zone/channel in Dmr Conventional mode.

Package: com.hytera.api.base.dmr

8.3.2 ChannelCManager Method

getCurrentZoneChannlInfo

DSDmrCZoneChannelInfo getCurrentZoneChannelInfo()

throws [HyteraSDKException](#)

This message is used to obtain current zone channel information.

Returns:

Throws:

[HyteraSDKException](#)

getDmrCChannelNumberOfZone

int getDmrCChannelNumberOfZone(int zoneNum)

throws [HyteraSDKException](#)

This message is used to obtain the total number of channels in the specified zone.

Parameters:

zoneNum - 0x01~0x40zone number

Returns:

Throws:

[HyteraSDKException](#)

getDmrCZoneChannelInfo

```
DSDmrCChannelInfo getDmrCZoneChannelInfo(int zoneNum, int channelNum)  
    throws HyteraSDKException
```

Retrieve channel information

Parameters:

zoneNum - 0x01~0x40 zone number

channelNum - 0x01~0x80 channel number

Throws:

[HyteraSDKException](#)

Example:

```
//getDmrCZoneChannelInfo()  
  
try {  
    DSDmrCChannelInfo channelInfo =  
    mDmrManager.getChannelManager().getDmrCZoneChannelInfo(  
        zoneNum, channelNum);  
  
} catch (HyteraSDKException e) {  
    e.printStackTrace();  
}
```



setDmrCZoneChannel

```
void setDmrCZoneChannel(int zoneNum,int channelNum)  
    throws HyteraSDKException
```

Switch to a specific zone and channel. You need to register a listener and the callback method will return you the target channel information.

Parameters:

zoneNum - 0x01~0x40 zone number

channelNum - 0x01~0x80 channel number

Throws:

[HyteraSDKException](#)

Example:

```
try {  
    mDmrManager.getChannelCManager().registerListener(  
        mDmrCChannelManagerListen  
er);  
    mDmrManager.getChannelCManager().setDmrCZoneChannel(  
        zoneNum, channelNum);  
} catch (HyteraSDKException e) {  
    e.printStackTrace();  
}  
  
DmrCChannelManagerListener mDmrCChannelManagerListener = new  
    DmrCChannelManagerListener() {
```

```
@Override

    public void onsetZoneChannelAck(int result) {

        super.onsetZoneChannelAck(result);

        //result - 0x0000 Success, 0x0001 Failure, 0x8000 Invalid
parameter
    }

    @Override

    public void onUnsolCurrentZoneChannel(DSDmrCZoneChannelInfo
data) {

        super.onUnsolCurrentZoneChannel(data);

    }

};
```

registerListener

```
void registerListener(
    android.dsp.proxy.DmrCChannelManagerListener listener)
throws HyteraSDKException
```

Parameters:

listener - [android.dsp.proxy.DmrCChannelManagerListener](#)

Throws:

[HyteraSDKException](#)

DmrCChannelManagerListener has 4 callback methods and 2 of them should be implemented.

Callback method: [onUnsolCurrentZoneChannel](#)

This method will respond when current channel changes.

Callback method: [onsetZoneChannelAck](#)

This method will respond when you call setDmrCZoneChannelmethod.

```
DmrCChannelManagerListener mDmrCChannelManagerListener = new
DmrCChannelManagerListener() {

    @Override

    publicvoid onSetChannelParamAck(intresult, Bundle request) {

    }

    @Override

    publicvoid onUnsolCurrentZoneChannel(DSDmrCZoneChannelInfo
data) {          //this method will be resposeded when current channel
switched

    }

    @Override

    publicvoid ongetZoneChannelAck() {

    }

    @Override
```



```
public void onsetZoneChannelAck(int result) {  
  
    // this method will be responded when set zone channel  
  
}  
  
};
```

unregisterListener

```
void unregisterListener(  
    android.dsp.proxy.DmrCChannelManagerListener listener)  
  
throws HyteraSDKException
```

Parameters:

listener - `android.dsp.proxy.DmrCChannelManagerListener`

Throws:

`HyteraSDKException`

8.3.3 ChannelCManager Data

8.3.3.1 DsDmrCChannelInfo

DsDmrCChannelInfo class member introduction:

Filed	Type	Description
channelAlias	int	0x01~0x80 channel index in zone
channelIndex	int	1~1024 channel index in channel list
channelType	int	0x00 Digital 0x01 Analog

8.3.3.2 DsDmrCZoneChannelInfo

DsDmrCZoneChannelInfo class member introduction:

Filed	Type	Description
zoneNum	int	0x01~0x40 zone index
channelNum	int	0x01~0x80 channel index in a zone
channelType	int	0x00 Digital 0x01 Analog
channelIndex	int	1~1024 channel index in channel list
zoneAlias	String	
channelAlias	String	

8.4 RoamCManager

8.4.1 RoamCManager Introduction

RoamCManager interface contains one method to monitor device scan and roaming switch on or off.

Package: com.hytera.api.base.dmr

8.4.2 RoamCManager Method

registerListener

```
public void registerListener(  
    android.dsp.proxy.DmrCRoamScanListener listener)  
    throws HyteraSDKException
```

Register this listener to get callback method values. Roaming information, both roaming switch status and scan switch status can be returned in this listener.

Parameters:

listener –**android.dsp.proxy.DmrCRoamScanListener**

Throws:

HyteraSDKException

unregisterListener

```
public void unregisterListener(  
    android.dsp.proxy.DmrCRoamScanListener listener)  
    throws HyteraSDKException
```

Call this method when you don't need to watch something above anymore.

Parameters:

listener - `android.dsp.proxy.DmrCRoamScanListener`

Throws:

`HyteraSDKException`

android.dsp.proxy.DmrCRoamScanListener

This listener contains abstract methods, you only need to implement 4 methods which are listed below.

```
public void unsolDmrCScanSwitch(int status)
```

Parameters:

status - 0 Off, 1 On

```
public void unsolDmrCRoamSwitch (int status)
```

Parameters:

status - 0 Off, 1 On

Callback method: [unsolDmrCScanSwitch](#)

This method is used to monitor device scan switch on or off.

Callback method: [unsolDmrCRoamSwitch](#)

This method is used to monitor device roaming switch on or off.

8.4.3 RoamCManager Data

8.4.3.1 DsDmrCScanOrRoamInfo

DsDmrCScanOrRoamInfo class member introduction:

Filed	Type	Description
status	int	0x00 roaming off 0x01 detecting 0x02 roaming stay
channelType	int	0x00 Digital Channel This value is invalid if the Status is "roaming off".
channelIndex	int	1-1024 This value is invalid if the Status is "roaming off".
channelAlias	String	This value is invalid if the Status is "roaming off".

8.5 SmsCManager

8.5.1 SmsCManager Introduction

This interface defines some methods about how to send or receive a DMR Conventional message. In order to receive message in background process, messages(DmrC/DmrT/TetraD/TetraT) are received through broadcast method. For example, when you receive one DmrC message through a broadcast receiver, then you can start one

service to handle it.

Package: com.hytera.api.base.dmr

8.5.2 SmsCManager Method

sendDmrCMessage

```
void sendDmrCMessage(int requestID,  
                     int callType,  
                     int targetID,  
                     java.lang.String msg,  
                     SmsCListener.SendDmrCMessageListener listener)  
    throws HyteraSDKException
```

This message is used to send a text message in DMR Conventional mode.

Parameters:

requestID - message id, can be an arbitrary integer value

callType - 0x00 Private Call, 0x01 Group Call

targetID - 1-16776415The target contact ID.

msg-message text

listener -[SmsCListener.SendDmrCMessageListener](#)

Throws:

[HyteraSDKException](#)

registerDmrCMessagePendingIntent

```
void registerDmrCMessagePendingIntent ()throwsHyteraSDKException
```

Register PendingIntent to receive DmrC sending report broadcast and message broadcast.In order to receive

broadcast information in background service, you need to call this method when booting completes or starting application.

Throws:

[HyteraSDKException](#)

BroadcastReceiveDmrCSendReport

This message is used to broadcast message sending result. Below Example code shows how to get report and receive message.

Action: `ACTION_DMR_C_SEND_REPORT` // broadcast action

Report: `DMR_C_SEND_REPORT` // flag, used to retrieve send report from intent

BroadcastReceiveDmrCMessage

This message is used to broadcast received text message content.

Action: `ACTION_DMR_C_REV_MESSAGE`

Message: `DMR_C_REV_MESSAGE` // flag, used to retrieve message from intent

Example: register broadcast receiver in androidmanifest file

```
<receiver android:name="com.hytera.sdktest.SmsReceiverSDK">

    <intent-filter>

        <action android:name="android.intent.action.BOOT_COMPLETED"/>

        <action android:name="com.hytera.action.dmr_c_send_report_sdk"/>

    </intent-filter>

    <action android:name="com.hytera.action.dmr_c_rev_message_sdk"/>

</receiver>
```

```
<actionandroid:name="com.hytera.action.dmr_t_send_report_sdk"/>

<actionandroid:name="com.hytera.action.dmr_t_rev_message_sdk"/>

</intent-filter>

</receiver>
```

Example: handle received broadcast

```
publicclass SmsReceiverSDK extends BroadcastReceiver {

    @Override

    publicvoid onReceive(Context context, Intent intent) {

        String action = intent.getAction();

        if(Intent.ACTION_BOOT_COMPLETED.equals(intent.getAction())){

            try {

                mDmrManager = HyteraManager.getDmrManager(context);

                //register PendingIntent to receive Dmr message broadcast

                mDmrManager.getSmsCManager().registerDmrCMessagePendingIntent();

                mDmrManager.getSmsTManager().registerDmrTMessagePendingIntent();

            } catch (HyteraSDKException e) {

                e.printStackTrace();

            }

        }elseif(SmsCManager.ACTION_DMR_C_SEND_REPORT.equals(action)){
```

```
        //DmrC message send report

        CInt5Pracel data =

        (CInt5Pracel)intent.getParcelableExtra(SmsCManager.DMR_C_SEND_REPO
RT);

        }elseif(SmsCManager.ACTION_DMR_C_REV_MESSAGE.equals(action)){

            //receive DmrC message

            DSDmrCReceiveMessage mDSDmrCReceiveMessage =

            intent.getParcelableExtra(SmsCManager.DMR_C_REV_MESSAGE);

            }elseif(SmsTManager.ACTION_DMR_T_SEND_REPORT.equals(action)){

                //DmrT message send report

                DSDmrTMsgReport report =

                intent.getParcelableExtra(SmsTManager.DMR_T_SEND_REPORT);

                }elseif(SmsTManager.ACTION_DMR_T_REV_MESSAGE.equals(action)){

                    //receive DmrT message

                    DSDmrTReceiveMessage data =

                    intent.getParcelableExtra(SmsTManager.DMR_T_REV_MESSAGE);

                }

            }

        }
```


sendDmrCShortData

```
void sendDmrCShortData(DSDmrShortData dsDmrShortData)  
    throws HyteraSDKException
```

This message is used to send short data in DMR Conventional mode.

Parameters:

dsDmrShortData – short data

Throws:

[HyteraSDKException](#)

registerDmrCShortDataListener

```
void registerDmrCShortDataListener (SmsCListener.ShortDataListener listener)  
    throws HyteraSDKException
```

This message is used to register a listener in DMR Conventional mode.

Parameters:

listener – [SmsCListener.ShortDataListener](#)

Throws:

[HyteraSDKException](#)

unRegisterDmrCShortDataListener

```
void unRegisterDmrCShortDataListener (SmsCListener.ShortDataListener listener)  
    throws HyteraSDKException
```

This message is used to unregister a listener in DMR Conventional mode.

Parameters:

listener - `SmsCListener.ShortDataListener`

Throws:

`HyteraSDKException`

8.5.3 SmsCListener Class

8.5.3.1 SmsCListener Introduction

Package: com.hytera.api.base.dmr

8.5.3.2 SmsCListener Inner Class

Type	Class
static interface	<code>SmsCListener.DmrCSendMessageListener</code>
static class	<code>SmsCListener.ShortDataListener</code>

public static interface `SmsCListener.DmrCSendMessageListener`

onCallback

void onCallback(int result)

Parameters:

result -

0x0100 Accepted, the send request is processing;

0x0200 Refused, channel busy;

0x8000 Invalid parameter

public static abstract class `SmsCListener.ShortDataListener`

sendDmrCShortDataReply

void sendDmrCShortDataReply (int result)

Parameters:

result -

0x0100 Accepted, the send request is processing;

0x0200 Refused, channel busy;

0x8000 Invalid parameter

sendDmrCShortDataResult

```
void sendDmrCShortDataResult (DSDmrShortData dsDmrShortData )
```

Parameters:

dsDmrShortData -

receiveDmrCShortData

```
void receiveDmrCShortData (DSDmrShortData dsDmrShortData)
```

Parameters:

dsDmrShortData -

8.5.4 SmsCManager Data

8.5.4.1 CInt5Parcel

CInt5Parcel class introduction:

(This class used to describe DmrC message send report.)

Filed	Type	Description
mInt1	int	result 0x0000 Succeeded, 0x0001 Failed, 0x0002: channel busy

Filed	Type	Description
		0x0003: No ACK 0x0004: authentication failed 0x0065: unactivated (reserved) 0x0066: narrowband network off 0x0067: radio stunned (reserved) 0X0068: service denied (reserved)
mInt2	int	callType 0x00 Private Call, 0x01 Group Call
mInt3	int	sourceId The source radio ID.
mInt4	int	targeted The target contact ID.
mInt5	int	reserved

8.5.4.2 DSDmrCReceiveMessage

DSDmrCReceiveMessage class introduction:

(This class used to describe received DmrC message.)

Filed	Type	Description
callType	int	0x00 Private Call, 0x01 Group Call
sourceID	int	The source radio ID.
targetID	int	The target contact ID.
year	int	The year value of the received time
month	int	

Filed	Type	Description
day	int	
hour	int	
minute	int	
second	int	
msg	String	The received message content

8.5.4.3 DSDmrShortData

DSDmrShortData class introduction:

(This class used to describe received DSDmrShortData.)

Filed	Type	Description
callType	int	0x00 Private ShortData, 0x01 Group ShortData
requestID	int	
destID	String	When callType is 0, destID means receiver radio ID; When callType is 1, destID means group ID.
srcID	String	The source radio ID.
groupID	String	When callType is 0, this parameter is invalid.
dataLen	int	The data length, byte array length.
shortData	byte[]	Received short data
result	int	0x0000 Success; 0x0001 Failure; 0x0002 Channel busy; 0x0003 No ack; 0x0004 Authentication failed; 0x0065 Inactivated;

Filed	Type	Description
		0x0066 Narrowband network off; 0x0067 Device stunned; 0x0068 Service denied

8.6 ContactTManager

8.6.1 ContactTManager Introduction

ContactTManager contains some methods to request the contact information in DMR Trunking mode.

Package: com.hytera.api.base.dmr

8.6.2 ContactTManager Method

getDmrTSubgroupContactInfo

DmrTSubGroupContact

getDmrTSubgroupContactInfo(int subgroupIndex, int contactIndex) throws [HyteraSDKException](#)

This message is used to obtain the subgroup contact information.

Parameters:

subgroupIndex - 0x01~0x40 valid range of group index

contactIndex - 0x01~0x80 valid range of contact index

Returns: [DmrTSubGroupContact](#)

Throws:

[HyteraSDKException](#)

Example:

```
// get the contact information in current group

try {

DmrTSubGroupContactcontactInfo =
mDmrManager.getContactTManager().getDmrTSubgroupContactInfo(

                                subgroupIndex, contactIndex);

} catch (HyteraSDKException e) {

    e.printStackTrace();

    Log.d(TAG,e.getMessage());

}
```

getDmrTCurrentContact

DmrTSubGroupContact getDmrTCurrentContact()

throws [HyteraSDKException](#)

Return current contact iformation.

getDmrTContactSizeOfSubGroup

int getDmrTContactSizeOfSubGroup(int subgroupIndex)

throws [HyteraSDKException](#)

This message is used to get the total number of contacts in a subgroup.

Parameters:

subgroupIndex—group index

Returns: contacts number of one group

Throws:

[HyteraSDKException](#)

Example:

```
try {  
    intsize =  
        mDmrManager.getContactTManager().  
            getDmrTContactSizeOfSubGroup(subgroupIndex);  
    } catch (HyteraSDKException e) {  
        e.printStackTrace();  
    }  
}
```

registerDmrTCurrentContactListener

```
void registerDmrTCurrentContactListener(  
    ContactTListener.DmrTCurrentContactListener listener)  
    throws HyteraSDKException
```

Returns the current contact information

Parameters:

listener - [ContactTListener.DmrTCurrentContactListener](#)

Throws:

[HyteraSDKException](#)

unRegisterDmrTCurrentContactListener

```
void unRegisterDmrTCurrentContactListener()
```


throws [HyteraSDKException](#)

Unregister the current contact listener

Throws:

[HyteraSDKException](#)

8.6.3 ContactTListener Class

8.6.3.1 ContactTListener Introduction

This class has defined a static interface which is used as a callback to verify the operation result.

Package: com.hytera.api.base.dmr

8.6.3.2 ContactTListener Inner Class

Type	Class
static interface	ContactTListener.DmrTCurrentContactListener

```
public static interface ContactTListener.DmrTCurrentContactListener
```

onCallback

```
void onCallback(DmrTCurrentContact currentContact)
```

Parameters:

currentContact -

8.6.4 ContactTManager Data

8.6.4.1 DmrTSubGroupContact

DmrTSubGroupContact class introduction:

Filed	Type	Description
subGroupIndex	int	the group index in one group list

Filed	Type	Description
contactIndex	int	contact index in one group
contactId	String	
contacttype	int	
subGroupAlias	String	group alias
contactAlias	String	contact alias

8.6.4.2 DmrTCurrentContact

DmrTCurrentContact class introduction:

Filed	Type	Description
subGroupIndex	int	0x01~0x40 the group index in one group list
contactIndex	int	0x01~0x80 contact index in one group
subGroupAlias	String	group alias
contactAlias	String	contact alias

8.7 RegistrationTManager

8.7.1 RegistrationTManagerIntroduction

RegistrationTManager contains methods about how to get/set DMR registration information. It is feasible to get registration status, roaming status and single station status through this interface to. Also, this manager provides methods to monitor the change of them.

Package: com.hytera.api.base.dmr

8.7.2 RegistrationTManager Method

sendDmrTRegisterStatusQuery

```
void sendDmrTRegisterStatusQuery(int requestId)
    throws HyteraSDKException
```

This message is used to query registration status.

Parameters:

requestId—Reserve, any integer.

Throws:

[HyteraSDKException](#)

Example:

```
DmrTRegisterManagerListener mDmrTRegisterManagerListener =
    new DmrTRegisterManagerListener() {

        @Override
        public void sendDmrTRegisterStatusQueryAck(int result, int state) {
            //result: 0x0000 succeeded, 0x0001 failed
            //state: 0x00 registering,
                    0x01 registration succeeded,
                    0x02 registration failed
        }
    }

    //Register status query
    try {

        mDmrManager.getRegisterManager().registerListener(mDmrTRegisterManagerListen
```

```
er);

mDmrManager.getRegisterManager().sendDmrTRegisterStatusQuery(0);

} catch (HyteraSDKException e) {

    e.printStackTrace();

}
```

setDmrTSwitchControlChannel

void setDmrTSwitchControlChannel(int type, int number)

throws [HyteraSDKException](#)

This message is used to switch control channel. In order to get process result you need to register DmrTRegisterManagerListener.

Parameters:

type–

0x00 Channel

0x01 Station

0x02 System code

number - 0x0000~0xffff channelnumber/

stationnumber/

systemcode

Throws:

[HyteraSDKException](#)

Example: callback method below will be responded

```
@Override
```

```
public void setSwitchControlChannelInformationAck(int result) {  
    super.setSwitchControlChannelInformationAck(result);  
  
    //result: 0x0000 succeeded;  
  
    0x0001 failed;  
  
    0x0200 refused, busy;  
  
    0x8000 invalid parameter  
  
    For "refused", it means the previous request is in process, current  
    request is refused.  
  
}
```

setDmrTHandoverSwitch

void setDmrTHandoverSwitch(int enable) throws [HyteraSDKException](#)

This message is used to set handover feature on or off, register DmrTRegisterManagerListener to get callback result.

Parameters:

enable–

0x00 Off

0x01 On

Throws:

[HyteraSDKException](#)

Example: callback method below will be responded

```
@Override

    public void setHandoverSwitchAck(Bundle bundle, int result) {

        super.setHandoverSwitchAck(bundle, result);

        //result: 0x0000 succeeded;

                0x0001 failed;

                0x8000 invalid parameter

    }
```

getDmrTSingleStationStatus

```
int getDmrTSingleStationStatus()

    throws HyteraSDKException
```

This message is used to query the status of registered base station, register DmrTRegisterManagerListener to get callback result.

Returns:

```
0x00interconnected site status

0x01 single site status

0x02~0xff Reserved
```

Throws:

[HyteraSDKException](#)

Example: callback method below will be responded

```
@Override

    public void getSingleStationStatusAck(int result, int state) {

        super.getSingleStationStatusAck(result, state);

        //result: 0x0000 succeeded;

                0x0001 failed;

                0x8000 invalid parameter

        //state: 0x00interconnected site status

                0x01 single site status

                0x02~0xff Reserved

    }
```

getRoamStatus

void getRoamStatus() throws [HyteraSDKException](#)

This message is used to query roaming status, register DmrTRegisterManagerListener to get callback result.

Throws:

[HyteraSDKException](#)

Example:

```
DmrTRegisterManagerListener mDmrTRegisterManagerListener =

    new DmrTRegisterManagerListener() {
```

```
@Override

    public void getRoamStatusAck(int result, int state) {

        //result: 0x0000 succeeded,
        0x0001 failed,
        0x8000 invalid parameter

        //state: 0x00 normal status,
                0x01 roaming status,
                0x02~0xff reserved

    }

}

//Register status query

try {

    mDmrManager.getRegisterManager().registerListener(mDmrTRegisterManagerListener);

    mDmrManager.getRegisterManager().getRoamStatus();

} catch (HyteraSDKException e) {

    e.printStackTrace();

}
```

registerListener

```
void registerListener(
    android.dsp.proxy.DmrTRegisterManagerListener listener)
```


throws [HyteraSDKException](#)

Parameters:

listener - `android.dsp.proxy.DmrTRegisterManagerListener`

Throws:

[HyteraSDKException](#)

unregisterListener

```
void unregisterListener(  
    android.dsp.proxy.DmrTRegisterManagerListener listener)  
    throws HyteraSDKException
```

Call this method when you no longer need to listen

Parameters:

listener - `android.dsp.proxy.DmrTRegisterManagerListener`

Throws:

[HyteraSDKException](#)

BroadcastDmrTRegisterState

Register `DmrTRegisterManagerListener` and implement the follow 4 callback methods, you can watch status change about handover switch, registration status, roaming status and single sation status.

```
DmrTRegisterManagerListener mDmrTRegisterManagerListener = new  
DmrTRegisterManagerListener() {  
  
    @Override  
  
    public void unsolHandoverSwitch(int state) {
```

```
        //state: 0x00          Disable

        0x01Enable

    }

    @Override

    publicvoid unsolRegisterStatus(intstate, intchannelNumber) {

        //state:  0x00          Registering

        0x01  Registration succeeded

        0x02 Registrationfailed

        //channelNumber:  0x000~0xffe

    }

    @Override

    publicvoid unsolRoamStatus(intstatus) {

        //status:  0x00 normal status,

        0x01 roaming status,

        0x02~0xff reserved

    }

    @Override

    publicvoid unsolSingleStationStatus(intstatus) {

        //status:  0x00interconnected site status

        0x01 single site status

        0x02~0xff Reserved

    }
```

```
        . . .  
  
};
```

8.8 SmsTManager

8.8.1 SmsTManager Introduction

This interface defines methods about how to send or receive a Dmr Trunking message.

Package: com.hytera.api.base.dmr

8.8.2 SmsTManager Method

sendDmrTMessage

```
void sendDmrTMessage(int requestId,
                    int callType,
                    java.lang.String targetID,
                    int msgType,
                    java.lang.String msg,
                    SmsTListener.SendDmrTMessageListener listener)
```

throws [HyteraSDKException](#)

This message is used to send a text or status messages in DMR Trunking mode.

Parameters:

requestId - message id, can be an arbitrary integer value

callType - 0x00 Private Call, 0x01 Group Call

targetID-target id

msgType - 0x00 Text Message, 0x01 Status Message

msg-if msgType=0, Text Message

if msgType=1, Status Message, e.g. "1" or "0"

listener-[SmsTListener.SendDmrTMessageListener](#)

Throws:[HyteraSDKException](#)**registerDmrTMessagePendingIntent**

```
void registerDmrTMessagePendingIntent ()throws HyteraSDKException
```

Register PendingIntent for receiving DmrT message broadcast. In order to receive message in background service you need to register broadcast receiver in androidmanifest file.

Throws:[HyteraSDKException](#)**BroadcastReceiveDmrTSendReport**

After sending a DmrT message you will receive a report about the sending operation.

Action: [ACTION_DMR_T_SEND_REPORT](#)

Report : [DMR_T_SEND_REPORT](#)

BroadcastReceiveDmrTMessage

This message is used to broadcast received messages

.

Action: [ACTION_DMR_T_REV_MESSAGE](#)

Message:[DMR_T_REV_MESSAGE](#)

Register PendingIntent for receive DmrT send report broadcast and message broadcast.

Example code refer[8.5SmsCManager](#).

sendDmrTShortData

```
void sendDmrTShortData(DSDmrShortData dsDmrShortData)
    throws HyteraSDKException
```

This message is used to send a short data in DMR Trunking mode.

Parameters:

dsDmrShortData – short data

Throws:

[HyteraSDKException](#)

registerDmrTShortDataListener

```
void registerDmrTShortDataListener (SmsTListener.ShortDataListener listener)
    throws HyteraSDKException
```

This message is used to send a text message in DMR Conventional mode.

Parameters:

listener – [SmsTListener.ShortDataListener](#)

Throws:

[HyteraSDKException](#)

unRegisterDmrTShortDataListener

```
void unRegisterDmrTShortDataListener (SmsTListener.ShortDataListener listener)
    throws HyteraSDKException
```

This message is used to send a text message in DMR Conventional mode.

Parameters:

listener – [SmsTListener.ShortDataListener](#)

Throws:

HyteraSDKException

8.8.3 SmsTListener Class

8.8.3.1 SmsTListener Introduction

Package: com.hytera.api.base.dmr

8.8.3.2 SmsTListener Inner Class

Type	Class
static interface	SmsTListener.SendDmrTMessageListener
static class	SmsTListener.ShortDataListener

public static interface **SmsTListener.SendDmrTMessageListener**

onCallback

void onCallback(int result)

Parameters:

- result-
 - 0x0100 Accepted
 - 0x0101-0x01FF Else result, not the end state.
 - 0x0200 Refused
 - 0x8000 Invalid parameter
 - 0x8001-0xAFFF Else error

Note:

For "accepted", it means the request is accepted and is in process.

For "refused", it means that the previous request is in process, current request is refused.

```
public static abstract class SmsTListener.ShortDataListener
```

sendDmrTShortDataReply

```
void sendDmrTShortDataReply (int result)
```

Parameters:

result -

0x0100 Accepted, the send request is processing;

0x0200 Refused

sendDmrTShortDataReport

```
void sendDmrTShortDataReport (DSDmrShortData dsDmrShortData)
```

Parameters:

dsDmrShortData -

receiveDmrTShortData

```
void receiveDmrTShortData (DSDmrShortData dsDmrShortData)
```

Parameters:

dsDmrShortData -

8.8.4 SmsTManager Data

8.8.4.1 DSDmrTMsgReport

DSDmrTMsgReportclass introduction:

(This class used to describe DmrT message send report.)

Filed	Type	Description
Result	int	0x0000 Succeeded, 0x0001 Failed, 0x0002 No Authority, 0x0003 ID Forbidden, 0x0004 Status Forbidden, 0x0005 Incorrect Format, 0x0065 Unregistered, 0x0066 radio networkoff, 0x0067 radio stuned, 0x0068 Service denied. 0x8000 Invalid parameter For "ID forbidden", it means the target ID is not authorized or is the same as the source ID. For "status forbidden", it means the status message code is forbidden. For "service denied", it includes exceptions for example, a higher priority service is ongoing, or the service is not responded.
CallType	String	0x00 Private Call, 0x01 Group Call
SourceID	String	The source radio ID The specific Air Interface ID of the source radio
TargetID	int	The target contact ID The specific Air Interface ID of the source radio
MsgType	int	0X00 Text Message, 0x01 Status Message

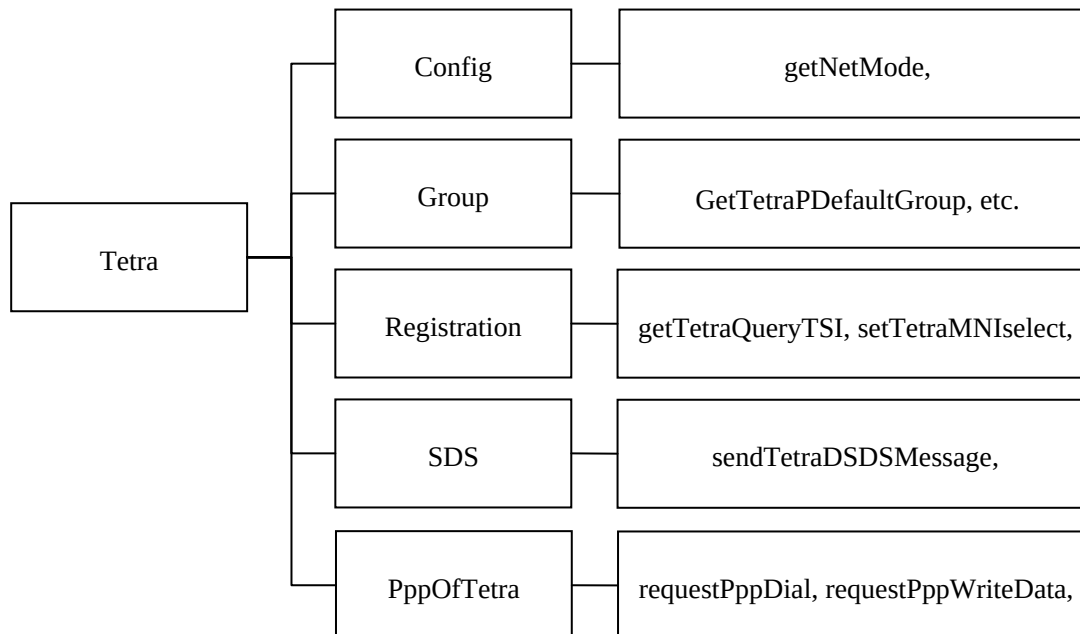
8.8.4.2 DSDmrTReceiveMessage

DSDmrTReceiveMessage class member introduction:

(This class used to describe received DmrT message.)

Filed	Type	Description
CallType	int	0x00 Private Call, 0x01 Group Call
sourceID	String	The source radio ID
targetID	String	The target contact ID
msgType	int	0X00 Text Message, 0x01 Status Message
year	int	The year value of the received time
month	int	
day	int	
hour	int	
minute	int	
second	int	
dataLen	int	The character length of the text message The value is "1" when the msgType is "status message".
message	String	The received message content

9. TetraManager Overview



The picture shows the overview of TetraManager. It contains 5 sub managers to manage different modules.

ConfigManager contains methods to manage Tetra device net mode and duplex mode.

GroupManager is used to manage Tetra group module.

RegistrationManager contains methods to manage registration module.

SDSManager provides methods to manage SDS module, such as how to send or receive a Tetra message in DMO or TMO mode.

PppOfTetraManager shows several methods to manage PPP module, such as how to build PPP connection and how to send data through this connection.

9.1 TetraManager

9.1.1 TetraManager Introduction

TetraManager interface contains 4 methods to get different manager instances (ConfigManager/GroupManager/RegistrationManager/SDSManager).

Package: com.hytera.api.base.tetra

9.1.2 TetraManager Method

getConfigManager

[ConfigManager](#) getConfigManager() throws [HyteraSDKException](#)

Returns: **ConfigManager instance**

Throws: [HyteraSDKException](#)

getGroupManager

[GroupManager](#) getGroupManager()throws [HyteraSDKException](#)

Returns: **GroupManager instance**

Throws: [HyteraSDKException](#)

getRegistrationManager

[RegistrationManager](#) getRegistrationManager()
throws [HyteraSDKException](#)

Returns: **RegistrationManager instance**

Throws: [HyteraSDKException](#)

getSDSManager

[SDSManager](#) getSDSManager()throws [HyteraSDKException](#)

Returns: **SDSManager instance**

Throws: [HyteraSDKException](#)

getPppOfTetraManager

[PppOfTetraManager](#) `getPppOfTetraManager()` throws [HyteraSDKException](#)

Returns: [PppOfTetraManager](#) instance

Throws: [HyteraSDKException](#)

9.2 ConfigManager

9.2.1 ConfigManager Introduction

This interface contains necessary methods about TETRA basic configurations, such as Net mode, E2EE mode call build method and so on.

Package: com.hytera.api.base.tetra

9.2.2 ConfigManager Method

getTetraNetMode

`int getTetraNetMode()` throws [HyteraSDKException](#)

This message is used to query the current work mode.

Returns:

0 TMO, 1DMO, 2 Gateway, 3 Repeater

(Currently, only the values 0 and 1 are available.)

Throws:

[HyteraSDKException](#)

getTetraTDuplexMode

`int getTetraTDuplexMode()` throws [HyteraSDKException](#)

This message is used to query the type of an initiated call.

Returns:

0 duplex, 1 simplex

Throws:[HyteraSDKException](#)**setTetraTDuplexMode**

```
void setTetraTDuplexMode(int mode) throws HyteraSDKException
```

This message is used to set the type of an initiated call. You need to register listener and one of its callback methods `setDefSimDuplexCallAck` will be responded.

Parameters:

mode - 0 duplex, 1 simplex

Throws:[HyteraSDKException](#)*Example:*

```
//set Tetra TMO Duplex mode

try {

    mTetraManager.getConfigManager().registerListener(

        mTetraConfigurationManagerListener);

    mTetraManager.getConfigManager().setTetraTDuplexMode(mode);

} catch (HyteraSDKException e) {

    e.printStackTrace();

}

//listener
```

```
TetraConfigurationManagerListener mTetraConfigurationManagerListener
= new TetraConfigurationManagerListener() {

    @Override

    public void setDefSimDuplexCallAck(int result, int plex) {

        super.setDefSimDuplexCallAck(result, plex);

        // result - 0x0000 Success, 0x0001 Failure, 0x8000 Invalid
parameter

        // plex - 0 duplex, 1 simplex

    }

};
```

registerListener

```
void registerListener(
    android.dsp.proxy.TetraConfigurationManagerListener listener)
    throws HyteraSDKException
```

Register listener to get callback result.

Parameters:

listener–

`android.dsp.proxy.TetraConfigurationManagerListener`

Throws:

`HyteraSDKException`

unregisterListener

```
void unregisterListener(  
    android.dsp.proxy.TetraConfigurationManagerListener listener)  
    throws HyteraSDKException
```

Parameters:

listener–

`android.dsp.proxy.TetraConfigurationManagerListener`

Throws:

`HyteraSDKException`

9.3 GroupManager

9.3.1 GroupManager Introduction

GroupManager contains methods to manage folder and group information(DMO/TMO).

Package: com.hytera.api.base.tetra

9.3.2 GroupManager Method

getTetraPDefaultGroup

```
android.dsp.tetra.bean.DsPDefaultGroupInfo  
getTetraPDefaultGroup()throws HyteraSDKException
```

Get default group information

Returns:`DsPDefaultGroupInfo`

Throws:

`HyteraSDKException`

getTetraDReadFolderInfo

```
android.dsp.tetra.bean.DsDFolderInfo getTetraDReadFolderInfo(  

```


int folderIndex) throws [HyteraSDKException](#)

Read folder information in DMO mode

Parameters:

folderIndex - Number Description 0-49(DMO)

Throws:

[HyteraSDKException](#)

getTetraDReadFolders

java.util.List<android.dsp.tetra.bean.DsDFolderInfo>

getTetraDReadFolders() throws [HyteraSDKException](#)

Read whole folder information in DMO mode

Returns:

Throws:

[HyteraSDKException](#)

getTetraDReadFolderGroupInfo

android.dsp.tetra.bean.DsDGroupInfo getTetraDReadFolderGroupInfo(int folderIndex, int groupIndex) throws [HyteraSDKException](#)

Read the group information of the folder in DMO

Parameters:

folderIndex - 0-49

groupIndex - 0-199 valid value

Throws:

[HyteraSDKException](#)

getTetraDReadFolderGroups

```
java.util.List<android.dsp.tetra.bean.DsDGroupInfo>
```

```
    getTetraDReadFolderGroups(int folderIndex)
```

```
    throws HyteraSDKException
```

Read whole groups of one folder in DMO mode

Parameters:

folderIndex - 0-49

Returns:**Throws:**

[HyteraSDKException](#)

getTetraTReadFolderInfo

```
android.dsp.tetra.bean.DsTFolderInfo getTetraTReadFolderInfo(  
    int folderIndex) throws HyteraSDKException
```

Read folder information in TMO mode. Max folder number in TMO mode is 200.

Parameters:

folderIndex - 0-199(TMO Static) valid value

Returns:**Throws:**

[HyteraSDKException](#)

getTetraTReadFolders

```
java.util.List<android.dsp.tetra.bean.DsTFolderInfo>
```

```
    getTetraTReadFolders() throws HyteraSDKException
```

Read whole folder information in TMO mode

Returns:

Throws:

[HyteraSDKException](#)

getTetraTReadScanInfo

```
android.dsp.tetra.bean.DsTScanListInfo getTetraTReadScanInfo(  
    int folderIndex) throws HyteraSDKException
```

Read scanfolder information in TMO mode. Max scan folder in TMO mode number is 64.

Parameters:

folderIndex - 0-63(TMO Scan) valid value

Returns:

Throws:

[HyteraSDKException](#)

getTetraTReadScanList

```
java.util.List<android.dsp.tetra.bean.DsTScanListInfo>  
    getTetraTReadScanList() throws HyteraSDKException
```

Read whole scan folder information in TMO mode.

Returns:

Throws:

[HyteraSDKException](#)

getTetraTReadFolderGroupInfo

```
android.dsp.tetra.bean.DsTGroupInfo getTetraTReadFolderGroupInfo(int type,
    int folderIndex, int groupIndex)

    throws HyteraSDKException
```

Get TMO group information. Parameter type=0 means static folder, type=1 means scan folder.

Parameters:

type - 0x00 Static Folder
 0x01 Scan List(TMO mode)

folderIndex - 0-199(TMO Static) valid value
0-63(TMO Scan) valid value

groupIndex - 0-199 valid value

Returns:**Throws:**

[HyteraSDKException](#)

getTetraTReadFolderGroups

```
java.util.List<android.dsp.tetra.bean.DsTGroupInfo>
    getTetraTReadFolderGroups(int type, int folderIndex)

    throws HyteraSDKException
```

Read whole groups of one folder in TMO mode. Parameter type=0 means static folder, type=1 means scan folder.

Parameters:

type - 0x00 Static Folder
 0x01 Scan List(TMO mode)

folderIndex - 0-199(TMO Static) valid value

0-63(TMO Scan) valid value

Returns:

Throws:

[HyteraSDKException](#)

registerListener

```
void registerListener(  
    android.dsp.proxy.TetraTalkGroupManagerListener listener)  
    throws HyteraSDKException
```

Register listener to watch the change of group attach/detach status, used folder change and the change of current default group.

Parameters:

listener - `android.dsp.proxy.TetraTalkGroupManagerListener`

Throws:

[HyteraSDKException](#)

unregisterListener

```
void unregisterListener(  
    android.dsp.proxy.TetraTalkGroupManagerListener listener)  
    throws HyteraSDKException
```

Unregister this listener when you don't need to watch the change above.

Parameters:

listener - `android.dsp.proxy.TetraTalkGroupManagerListener`

Throws:

[HyteraSDKException](#)

TetraTalkGroupManagerListener has several callback methods, you need to implement 3 methods like below:

```
TetraTalkGroupManagerListener listener = new
TetraTalkGroupManagerListener() {

    @Override

    publicvoid unsolTetraAttachDeatchStatus(DuTetraPAttachDeatchStatus[]
adutetrapattachdeatchstatus) {

//attach/detach result will be returned in this callback method

    }

    @Override

    publicvoidunsolTetraPDefaultGroup(CInt2Pracel cint2pracel) {

//this callback will be responded when default group changed

    }

    @Override

    publicvoid unsolTetraPUsedFolder(CInt2Pracel cint2pracel) {

//this callback will be responded when used folder changed

    }

};
```

9.3.3 GroupManager Data

9.3.3.1 DsPDefaultGroupInfo

DsPDefaultGroupInfo class member introduction:

Filed	Type	Description
MCC	int	Country id
MNC	int	Network id
GSSI	int	Group id
folderIndex	int	
groupIndex	int	
attachStatus	int	0 Detach, 1 Attach
name	int	Default group name

9.3.3.2 DsDGroupInfo

DsDGroupInfo class member introduction:

Filed	Type	Description
Name	String	Group name
MCC	int	Country id
MNC	int	Network id
GSSI	int	Group id
ChannelOffset	int	
CarrierNumber	int	
FrequencyBand	int	
COU	int	
LifeTime	int	Life time

Filed	Type	Description
Visibility	int	0 invisibility, 1 visibility
OnlyRecv	int	0 not only recv, 1 omly recv
TempGrp	int	
UseScan	int	0 not scan, 1 scan
EncryptionRule	int	

9.3.3.3 DsTGroupInfo

DsTGroupInfo class member introduction:

Filed	Type	Description
staticGroupIndex	int	Static group index
Name	String	Group name
MCC	int	Country id
MNC	int	Network id
GSSI	int	Group id
COU	int	
LifeTime	int	Life time
Visibility	int	0 invisibility, 1 visibility
OnlyRecv	int	0 not only recv, 1 only recv
TempGrp	int	

Filed	Type	Description
UseScan	int	0 not scan, 1 scan

9.3.3.4 DsDFolderInfo

DsDFolderInfo class member introduction:

Filed	Type	Description
Name	String	Folder name
PreDefaultGroup	int	0-200 prev default group index, 0xFFFF no prev default group
Visibility	int	0 invisibility, 1 visibility
GroupNum	int	Group number in this folder

9.3.3.5 DsTFolderInfo

DsTFolderInfo class member introduction:

Filed	Type	Description
Name	String	Folder name
PreDefaultGroup	int	0-200 prev default group index, 0xFFFF no prev default group
Visibility	int	0 invisibility, 1 visibility
GroupNum	int	Group number in this folder
ScanListIndex	int	

9.3.3.6 DuTetraPAttachDeatchStatus

DuTetraPAttachDeatchStatus class member introduction:

Filed	Type	Description
MCC	int	Country id
MNC	int	Network id
GSSI	int	
AttachStatus	int	0 Attachment, 1 Detachment
Cou	int	Class of usage
Lifetime	int	0 Permanent, attachment not needed, 1 Attachment needed for next ITSI attachment, 2 Attachment not allowed afternext ITSI attachment, 3 Attachment needed for next location update
DetachReason	int	0 Unknown group identity, 1 Temporary 1 detached, 2 Temporary 2 detached, 3 Permanently detached, 4 Initiated by user

9.4 RegistrationManager

9.4.1 RegistrationManager Introduction

RegistrationManager contains some methods to manage TETRA registration module.

Package: com.hytera.api.base.tetra

9.4.2 RegistrationManager Method

getTetraQueryTSI

```
int getTetraQueryTSI()throws HyteraSDKException
```

Query device MCC, MNC and SSI. You need to register *TetraRegistrationManagerListener* to get the result. Call back method *getTetraQueryTSIAck* will return you the result.

Returns: result of query process

```
0x0000 Success,  
0x0001 Failure,  
0x8000 Invalid parameter.
```

The result can also be returned by callback method.

Throws:

HyteraSDKException

Example:

```
//query TSI  
  
try {  
    //register listener  
    mTetraManager.getRegistrationManager().registerListener(  
        mTetraRegistrationManagerListene  
r);  
    //call method  
    int result =  
        mTetraManager.getRegistrationManager().getTetraQueryTSI();  
    } catch (HyteraSDKException e) {  
        e.printStackTrace();  
    }  
}
```

```
//listener

TetraRegistrationManagerListener mTetraRegistrationManagerListener =
new TetraRegistrationManagerListener() {

    @Override

    public void getTetraQueryTSIAck(int result, DsTetraPTsiType[]
data) {

        super.getTetraQueryTSIAck(result, data);

        //result - 0x0000 Success, 0x0001 Failure, 0x8000 Invalid
parameter

        // data - data is a array, data[0] will be what you want

    }

};
```

getTetraTRegistrationState

int getTetraTRegistrationState() throws [HyteraSDKException](#)

Query the registration status of the radio in TMO. In order to get the process result, you need to register listener and then the callback method getTetraTRegistrationStateAck will be responded.

Returns:

Throws:[HyteraSDKException](#)**setTetraMNIseselect**

```
void setTetraMNIseselect(int index) throws HyteraSDKException
```

Switch to a specific network. Register listener and callback method setTetraMNIseselectAck will return you the result.

Parameters:

index - 0-31

Throws:[HyteraSDKException](#)**getTetraTQueryTMOMode**

```
int getTetraTQueryTMOMode() throws HyteraSDKException
```

This message is used to broadcast whether the current base station is under fall back mode. Returns 0 for fall back mode, and returns 1 for normal mode. You can also get querying result through listener callback method getTetraTQueryTMOModeAck.

Returns:**Throws:**[HyteraSDKException](#)**registerListener**

```
void registerListener(  
    android.dsp.proxy.TetraRegistrationManagerListener listener)  
    throws HyteraSDKException
```

Parameters:

listener-

`android.dsp.proxy.TetraRegistrationManagerListener`

Throws:

`HyteraSDKException`

TetraRegistrationManagerListener has several abstract methods, you need to implement 5 methods below:

```
TetraRegistrationManagerListener mTetraRegistrationManagerListener =
new TetraRegistrationManagerListener() {

    @Override

    publicvoid getTetraQueryTSIAck(intresult, DsTetraPTsiType[]
data) {

        //result - 0x0000 Success,

                0x0001 Failure,

                0x8000 Invalid parameter

        // data - data is a array, data[0] will be what you want

    }

    @Override

    publicvoid getTetraTRegistrationStateAck(intresult, intstate)
{

        //result - 0x0000 Success,

                0x0001 Failure,
```

```
        0x8000 Invalid parameter

//state - 0 Not registered,

        1 Registered.

}

@Override

publicvoid setTetraMNIseselectAck(Bundle request, intresult) {

        //result - 0x0000 Success,

        0x0001 Failure,

        0x8000 Invalid parameter

}

@Override

publicvoid getTetraTQueryTMOModeAck(intresult, intTMOMode) {

        //result - 0x0000 Success,

        0x0001 Failure,

        0x8000 Invalid parameter

        //TMOMode - 0 FallBackMode,

        1 NormalMode

}
```

```
@Override

    public void unsolTetraTMOMode(int mode) {

        // when TMO mode have changed, this callback method will be
        responded

        //mode -    0 FallBackMode,

                1 NormalMode

    }

};
```

unregisterListener

```
void unregisterListener(
    android.dsp.proxy.TetraRegistrationManagerListener listener)
throws HyteraSDKException
```

Parameters:

listener-

`android.dsp.proxy.TetraRegistrationManagerListener`

Throws:

`HyteraSDKException`

9.5 SDSManager

9.5.1 SDSManager Introduction

SDSManager provides methods to show how to send or receive a tetra message in DMO and TMO mode.

Package: com.hytera.api.base.tetra

9.5.2 SDSManager Method

getTetraPSDSFormat

```
void getTetraPSDSFormat(  
    SDSListener.TetraPGetSDSFormatListener listener)  
  
throws HyteraSDKException
```

Get the messageformat

Parameters:

listener - `SDSListener.TetraPGetSDSFormatListener`

Throws:

`HyteraSDKException`

unregisterGetSDSFormat

```
void unregisterGetSDSFormat ()throws HyteraSDKException
```

Call this method when current activity is finished. If you don't need to watch the changeof SDS format, you can call this method after getTetraPSDSFormat.

Throws:

`HyteraSDKException`

setTetraPSDSFormat

```
void setTetraPSDSFormat(int format,  
    SDSListener.TetraPSetSDSFormatListener listener)  
  
throws HyteraSDKException
```

Set message format

Parameters:

format– 0x07 PEI mode

(User data including SDS-TL header in ASCII Hex format.)

0x08 Text mode

(User data in clear text, no SDS-TL header. In text mode, the element "Length" of other SDS related commands will be ignored.)

listener – `SDSListener.TetraPSetSDSFormatListener`

Throws:

`HyteraSDKException`

sendTetraDMessage

```
void sendTetraDMessage(int requestId,
                        int modeType,
                        int e2eeMode,
                        int callType,
                        int priorityLevel,
                        int destSsi,
                        String msg,
```

```
SDSListener.TetraDSendMessageListener listener)
```

throws `HyteraSDKException`

Send message in DMO

Parameters:

requestId– message id, can be an arbitrary integer value

modeType–

- 9 SDS type 1 (16 bits),
- 10 SDS type 2 (32 bits),
- 11 SDS type 3 (64 bits),
- 12 SDS type 4 (0 - 2047 bits),
- 13 Precoded status

e2eeMode - 0 Clear, 1 Encrypted

callType - 0 Point to Point, 1 Point to multipoint

prioritylevel - 0 Normal priority call,

1 High priority call,

2 Pre-emptive priority call,

3 Emergency priority call

destSsi- identify devices, like target id

msg - if modeType=13, the message will be status message which must be consist of numbers,

if modeType=9~12, it will be text message

listener - **SDSListener.TetraDSendMessageListener**

Throws:

HyteraSDKException

Example: sendTetraDMessage

```
// send DMO text message "hytera" to target 15

try { mSDSManager = mTetraManager.getSDSManager();

    mSDSManager.sendTetraDMessage(121, 12, 0, 0, 0, 15, "hytera",
    new TetraDSendMessageListener() {
```

```
        @Override

        public void onCallback(int result) {

        }

    });

    } catch (HyteraSDKException e) {

        e.printStackTrace();

    }

    // send DMO status message "105" to target 15
    try { mSDSManager = mTetraManager.getSDSManager();

        mSDSManager.sendTetraDMessage(122, 13, 0, 0, 0, 15, "105",

        new TetraDSendMessageListener() {

            @Override

            public void onCallback(int result) {

            }

        });

    } catch (HyteraSDKException e) {

        e.printStackTrace();

    }

}
```

```
}
```

sendTetraTMessage

```
void setTetraTMessage(int requestId,
                      int aiService,
                      int callType,
                      int calledIdtype,
                      int accessPriority,
                      int e2ee,
                      int destSsi,
                      String msg, SDSListener.TetraTSendMessageListener listener)
throws HyteraSDKException
```

Send message in TMO

Parameters:

requestId—request id can be any integer

aiService—

- 9 SDS type 1 (16 bits),
- 10 SDS type 2 (32 bits),
- 11 SDS type 3 (64 bits),
- 12 SDS type 4 (0 - 2047 bits),
- 13 Precoded status

callType—

- 0 Point to Point, 1 Point to multipoint.

calledIdtype—

- 0 SSI,
- 1 TSI,
- 2 SNA,
- 3 PABX external subscriber number,
- 4 PSTN external subscriber number.

accessPriority—0Low, 1High, 2Emergency

e2ee—0Clear, 1Encrypted

destSsi—target id

msg—

if aiService=13, the message will be status message which

mustbe consist of numbers,

if aiService=9~12, it will be text message

listener —**SDSListener.TetraTSendMessageListener**

Throws:

HyteraSDKException

Example: sendTetraTMessage

```
// send TMO text message "hytera" to target 15

try {

    mSDSManager = mTetraManager.getSDSManager();

    mSDSManager.sendTetraTMessage(124, 12, 0, 0, 0, 0, 15, "hytera",

        new TetraTSendMessageListener() {

            @Override

            public void onCallback(int result) {
```

```

        }

    });

    } catch (HyteraSDKException e) {

        e.printStackTrace();

    }

    // send TM0status message "102" to target 15

    try {

        mSDSManager = mTetraManager.getSDSManager();

        mSDSManager.sendTetraTMessage(125, 13, 0, 0, 0, 15, "102",

            new TetraTSendMessageListener() {

                @Override

                public void onCallback(int result) {

                }

            });

    } catch (HyteraSDKException e) {

        e.printStackTrace();

    }

```

registerTetraMessagePendingIntent

void registerTetraMessagePendingIntent ()throws [HyteraSDKException](#)

Register PendingIntent for receiving report and message broadcast. In order to receive message in background

service you need to register broadcast receiver in androidmanifest file.

You will receive a sending report after you send a message, the report contains information about sending status. Messages others sent to you can be received in background process. Details shows in follow example.

Throws:

[HyteraSDKException](#)

BroadcastReceiveTetraDSendReport

This message is used to broadcast the message delivery report in DMO. The example below describes how to get a sending report and how to receive message in more detail.

Action: [ACTION_TETRA_D_SEND_REPORT](#)

Report : [TETRA_SEND_REPORT](#)

BroadcastReceiveTetraDMessage

This message is used to broadcast the received message content in DMO.

Action: [ACTION_TETRA_D_REV_SDS](#)

Message: [TETRA_REV_SDS](#)

BroadcastReceiveTetraTSendReport

This message is used to broadcast the message delivery report in TMO.

Action: [ACTION_TETRA_T_SEND_REPORT](#)

Report : [TETRA_SEND_REPORT](#)

BroadcastReceiveTetraTMessage

This message is used to broadcast the received message content in TMO.

Action: [ACTION_TETRA_T_REV_MESSAGE](#)

Message: TETRA_REV_SDS

Example: receive Tetra message

```
<receiverandroid:name="com.hytera.sdktest.SmsReceiverSDK">

<intent-filter>

<actionandroid:name="android.intent.action.BOOT_COMPLETED"/>

<actionandroid:name="com.hytera.action.tetra_d_send_report_sdk"/>

<actionandroid:name="com.hytera.action.tetra_d_rev_sds_sdk"/>

        <actionandroid:name="com.hytera.action.tetra_t_send_report_
sdksdk"/>

<actionandroid:name="com.hytera.action.tetra_t_rev_sds_sdk"/>

</intent-filter>

</receiver>
```

Example: handle received broadcast

```
publicclass SmsReceiverSDK extends BroadcastReceiver {

    @Override

    publicvoid onReceive(Context context, Intent intent) {

        String action = intent.getAction();

        if(Intent.ACTION_BOOT_COMPLETED.equals(intent.getAction())){

            try {
```

```
        mTetraManager = HyteraManager.getTetraManager(context);

//register PendingIntent to receive Tetra message broadcast

        mTetraManager.getSDSManager().registerTetraMessagePendingIntent();

        } catch (HyteraSDKException e) {

            e.printStackTrace();

        }

    } elseif (SDSManager.ACTION_TETRA_D_SEND_REPORT.equals(action)){

        //receive Tetra DMO message send report

        CInt3Pracel report =

            (CInt3Pracel)intent.getParcelableExtra(SDSManager.TETRA_SEND_REPORT);

    }elseif (SDSManager.ACTION_TETRA_D_REV_MESSAGE.equals(action)){

        // receive Tetra DMO message

        DuTetraDReceiveSDS data =

            intent.getParcelableExtra(SDSManager.TETRA_REV_SDS);

    }elseif (SDSManager.ACTION_TETRA_T_SEND_REPORT.equals(action)){

        // receive Tetra TMO message send report

        DuTetraTSendReport report =

            (DuTetraTSendReport)intent.getParcelableExtra(SDSManager.TETRA_SEND_REPORT);

    }elseif (SDSManager.ACTION_TETRA_T_REV_MESSAGE.equals(action)){

        // receive Tetra TMO message

        DuTetraTReceiveSDS data =

            intent.getParcelableExtra(SDSManager.TETRA_REV_SDS);

    }

}
```

```
}

```

9.5.3 SDSListener Class

9.5.3.1 SDSListener Introduction

Package: com.hytera.api.base.tetra

9.5.3.2 SDSListener Method

Type	Class
static interface	SDSListener.TetraPGetSDSFormatListener
static interface	SDSListener.TetraPSetSDSFormatListener
static interface	SDSListener.TetraDSendMessageListener
static interface	SDSListener.TetraTSendMessageListener

public static interface**SDSListener.TetraPGetSDSFormatListener**

onCallback

void onCallback(int result,int format)

Parameters:

result - 0x0000 Success,
 0x0001 Failure,
 0x8000 Invalid parameter
format - 0x07 PEI mode,0x08 Text mode

```
public static interface SDSListener.TetraPSetSDSFormatListener
```

onCallback

```
void onCallback(int result)
```

Parameters:

result - 0x0000 Success, 0x0001 Failure

```
public static interface SDSListener.TetraDSendMessageListener
```

onCallback

```
void onCallback(int result)
```

Parameters:

result - 0x0000 Success, 0x0001 Failure

```
public static interface SDSListener.TetraTSendMessageListener
```

onCallback

```
void onCallback(int result)
```

Parameters:

result - 0x0000 Success, 0x0001 Failure

9.5.4 SDSManager Data

9.5.4.1 CInt3Pracel

CInt3Pracelclass member introduction:

(This class is used to describe DMO message send report.)

Filed	Type	Description
-------	------	-------------

Filed	Type	Description
mInt1	int	sdsReport 0 Success, 1 Failure, 2 Message content too long, 3 Message process busy, 51 Forbidden send message, 52 Calling on the line, 101 Unregistered, 102 Narrowband net is closed, 103 Device temporarily disabled, 104 Service Denied.
mInt2	int	messageStatus 0 Send SDS message confirm, 1 Send SDS message confirm with User Data, 2 SDS report.
mInt3	int	deliveryStatus 1 Changeover or pre-emption failure, 2 Changeover or pre-emption rejected, 3 Short data transfer completed, 4 Transfer failed, 5 Response to late, 6 Channel is busy, 7 Link to DM-REP or DM-GATE not established or failed, 8 Transmit capability and authorization lost, 9 Called mobile rejected short data, 10 Called mobile does not support short data, 11 DM-MS is present and within range, short data transmission in progress,

Filed	Type	Description
		12 Short data transger in progress, 13 Called DM-GATE does not support short data, 14 Called DM-GATE rejected short data, 16 Security parameter mismatch, 64 Message not allowed/unsupported, 65 Out of time for response message, 66 Wrong destination device address, 67 Device data unknown, 68 Minimum sending power higher than allowed, 69 Unable to send, 70 Incorrect formatted request, 71 Channel-state changed 72 Restricted service.

9.5.4.2 DuTetraDReceiveSDS

DuTetraDReceiveSDSclass member introduction:

(This class is used to describe received DMO message.)

Filed	Type	Description
year	int	Received UTC time
month	int	
day	int	
hour	int	
minute	int	
second	int	
E2EEncryption	int	0 Not use End-to-end Encryption,

Filed	Type	Description
		1 Use End-to-end Encryption
modeType	int	9 SDS type 1(16 bits), 10 SDS type 2(32 bits), 11 SDS type 3(64 bits) 12 SDS type 4(0~2047 bits) 13 Precoded status
mccCalled	int	
mncCalled	int	
issi_gssiCalled	int	
callingTSIType	int	
mccCalling	int	
mncCalling	int	
issi_gssiCalling	int	
numCallingEsn	int	
callingESN	String	
ack	int	0 Not requested, 1 Requested
priorityLevel	int	0 Normal priority call, 1 High priority call, 2 Pre-emptive priority call, 3 Emergency priority call.
AIEncryption	int	
communicationType	int	0 Direct MS-MS operation, 1 Operation with repeater, 2 Operation with gateway(*),

Filed	Type	Description
		3 Operation with a repeater/gateway(*)
getwayMacc	int	10Bit, 001 (3 Digits) ... 1023 (4 Digits)
getwayMnc	int	14 Bit, 0001 (4 Digits) ... 16383 (5 Digits)
getwayAdd	int	10 Bit, 001 (3 Digits) ... 1023 (4 Digits)
dataLength	int	ASCII Decimal 4 Digits
data	String	User message, Unicode-16 Big-endian

9.5.4.3 DuTetraTSendReport

DuTetraTSendReportclass member introduction:

(This class is used to describe TMO message send report.)

Filed	Type	Description
sdsReport	int	
sdsInstance	int	
localDeliverStatus	int	
aiService	int	9 SDS type 1 (16 bits) 10 SDS type 2 (32 bits) 11 SDS type 3 (64 bits) 12 SDS type 4 (0~2047 bits)
callingIdType	int	
mccCallingParty	int	
mncCallingParty	int	
ssiCallingParty	int	
snaCallingParty	int	
numOfCalledEsnCallingParty	int	

Filed	Type	Description
esnCallingParty	String	
calledIdType	int	
mccCalledParty	int	
mncCalledParty	int	
ssiCalledParty	int	
snaCalledParty	int	
numOfCalledEsnCalledParty	int	
esnCalledParty	String	
length	int	
statusMsgNum	int	
protocolIdentifier	int	
messageType	int	
ackRequired	int	
storage	int	
networkDeliveryStatus	int	
messageReference	int	
validityPeriod	int	
forwardAddressType	int	
mccForwardAddressParty	int	
mncForwardAddressParty	int	
ssiForwardAddressParty	int	
snaForwardAddressParty	int	
numOfCalledEsnForwardAddressParty	int	
esnForwardAddressParty	String	

9.5.4.4 DuTetraTReceiveSDS

DuTetraTReceiveSDSclass member introduction:

(This class is used to describe received TMO message.)

Filed	Type	Description
year	int	Received UTC time
month	int	
day	int	
hour	int	
minute	int	
second	int	
E2EEncryption	int	0 Not use End-to-end Encryption, 1 Use End-to-end Encryption
sequenceNum	int	Use for long message, the message number of a long message.
maxNumOfMessage	int	Use for long message, max number of a long mesage
aiService	int	9 SDS type 1(16 bits), 10 SDS type 2(32 bits), 11 SDS type 3(64 bits), 12 SDS type 4(0~2047 bits), 13 Status (16 bits, some values are reserved in EN 300 392-2 [4])
callingPartIdentity	int	0 SSI, 1 TSI, 3 PABX external subscriber number, 4 PSTN external subscriber number.

Filed	Type	Description
ssiCalling	int	
mccCalling	int	
mncCalling	int	
snaCalling	int	
numCallingEsn	int	
ExternalSubscriberNumber Calling	String	
calledPartIdentity	int	
ssiCalled	int	
mccCalled	int	
mncCalled	int	
snaCalled	int	
ExternalSubscriberNumber Called	String	
dataLength	int	ASCII Decimal 4 Digits
data	String	User message, Unicode-16 Big-endian

9.6 PppOfTetraManager

9.6.1 PppOfTetraManager Introduction

PppOfTetraManager provides methods to show how to build a PPP connection and how to send and receive data through the net .

Package: com.hytera.api.base.tetra

9.6.2 PppOfTetraManager Method

requestPppDial

```
void requestPppDial(int userAuthFlag,  
                   int nameLen,  
                   byte[] name,  
                   int passwdLen,  
                   byte[] passwd)
```

throws [HyteraSDKException](#)

The Dial method is used to get information including IP address, subnet mask, DNS server address.

Parameters:

userAuthFlag - user authentication flag

nameLen -name length

name -

passwdLen - password length

passwd -

Throws:

[HyteraSDKException](#)

requestPppTcpServerInit

```
void requestPppTcpServerInit (int port)throws HyteraSDKException
```

Server initialization for TCP connection.

Parameters:

port -

Throws:

[HyteraSDKException](#)

requestPppTcpClientInit

```
void requestPppTcpClientInit (byte[] ipAddr, int port)throws HyteraSDKException
```

Client initialization for TCP connection.

Parameters:

ipAddr – IP Address

port –

Throws:

[HyteraSDKException](#)

requestPppWriteData

```
void requestPppWriteData (int dataLen, byte[] data)throws HyteraSDKException
```

Send data to Server or Client.

Parameters:

dataLen – data length

data –

Throws:

[HyteraSDKException](#)

requestPppUndial

```
void requestPppUndial ()throws HyteraSDKException
```

Undial

Throws:

[HyteraSDKException](#)

registerPppListener

```
void registerPppListener (PppOfTetraListener listener)  
    throws HyteraSDKException
```

Register PPP listener, used to get dial result and receive data.

Parameters:

listener - PppOfTetraListener

Throws:

HyteraSDKException

registerPppListener

```
void registerPppListener (PppOfTetraListener listener, Loopers loopers)  
    throws HyteraSDKException
```

Register PPP listener, used to get dial result and receive data. If you call this method in subthread, you need to transfer the subthread loopers into second parameter.

Parameters:

listener - PppOfTetraListener

loopers -

Throws:

HyteraSDKException

unRegisterPppListener

```
void unRegisterPppListener (PppOfTetraListener listener)  
    throws HyteraSDKException
```

Unregister PPP listener, call this method when you needn't listen callback methods any more.

Parameters:

listener - PppOfTetraListener

Throws:

HyteraSDKException

9.6.3 PppOfTetraListener

9.6.3.1 PppOfTetraListener Method

pppDialResult

```
void pppDialResult (int i, byte abyte0[], byte abyte1[], byte abyte2[], byte abyte3[])
```

This callback method returned you some information about dial result.

Parameters:

i – error code

abyte0 – IP address

abyte1 – Subnet mask

abyte2 – DNS1 address

abyte3 – DNS2 address

pppReadData

```
void pppReadData (int i, byte abyte0[])
```

This callback method is used to read received data.

Parameters:

i – the length of received byte array

abyte0 – received data

10. Example Application

Two example applications are provided based on HyteraSDK:

- SmsExample Application

SmsExample describes a simple scene about Common and Dmr Conventional/Tetra DMO. At first, it shows how to get a CommonManager instance and DmrManager/TetraManager instance, and then uses them to call relative methods to get radio information and sends or receives a Dmr Conventional/Tetra DMO message.

- CallExample Application

CallExample describes a basic scene about Call process. For example, how to set up a individual or group call, how to hang up a call or how to simulate PTT function, all of these have a detailed description in it.